

2

Introduction

BIG DATA PROCESSING

Part II: Time Series Analysis

Advanced Neural Networks

Neural Networks

Jun.-Prof. Dr.-Ing. Stefan Lüdtke

Institute for Visual & Analytic Computing
University of Rostock

Backpropagation with Tensors

Scalar vs. Matrix Operations

So far we have considered computations on **scalars**:

$$y = \sigma(w_1x + w_0)$$

We now consider computations on **vectors** and **matrices**:

$$\mathbf{y} = \sigma(\mathbf{A}\mathbf{x} + \mathbf{b})$$

- ▶ Matrix $\mathbf{A}$ and vector $\mathbf{b}$ are objects with attributes `value` and `grad`
- ▶ `A.grad` stores $\nabla_{\mathbf{A}} L$ and `b.grad` stores $\nabla_{\mathbf{b}} L$
- ▶ `A.grad` has the same shape/dimensions as `A.value` (since L is scalar)

Backpropagation on Loops

The matrix/vector computation

$$y = \sigma(\underbrace{\mathbf{Ax}}_{=u} + \mathbf{b})$$

can be written as **loops over scalar operations**:

```
for i  u.value[i] = 0
for i,j  u.value[i] += A.value[i, j] * x.value[j]
for i  y.value[i] =  $\sigma$ (u.value[i] + b.value[i])
```

Backpropagation on Loops

The backpropagated gradients for

for i $y.\text{value}[i] = \sigma(u.\text{value}[i] + b.\text{value}[i])$

are:

for i $u.\text{grad}[i] += y.\text{grad}[i] * \sigma'(u.\text{value}[i] + b.\text{value}[i])$

for i $b.\text{grad}[i] += y.\text{grad}[i] * \sigma'(u.\text{value}[i] + b.\text{value}[i])$

► **Red:** back-propagated gradients

► **Blue:** local gradients

Backpropagation on Loops

The backpropagated gradients for

```
for i,j  u.value[i] += A.value[i, j] * x.value[j]
```

are

```
for i,j  A.grad[i, j] += u.grad[i] * x.value[j]
```

```
for i,j  x.grad[j] += u.grad[i] * A.value[i, j]
```

► **Red:** back-propagated gradients

► **Blue:** local gradients

0 Backpropagation on Loops

In practice, all deep learning operations can be written using loops over scalar assignments. Example for a **higher-order tensor**:

```
for h,i,j,k  U.value[h, i, j] += A.value[h, i, k] * B.value[h, j, k]
for h,i,j    Y.value[h, i, j] =  $\sigma$ (U.value[h, i, j])
```

Backpropagation loops:

```
h,i,j  U.grad += Y.grad[h, i, j] *  $\sigma'$ (U.value[h, i, j])
h,i,j,k A.grad += U.grad[h, i, j] * B.value[h, j, k]
h,i,j,k B.grad += U.grad[h, i, j] * A.value[h, i, k]
```

1 Minibatching

Source code has two components:

- ▶ Slow part: Sequential operations (Python)
- ▶ Fast part: Vector/matrix operations (NumPy, BLAS, CUDA)

Goal:

- ▶ Fast part should dominate computation (wall clock time)
- ▶ Reduce the number of slow sequential operations (e.g., Python loops) by running the fast vector/matrix operations on several data points jointly
- ▶ This is called **minibatching** and used in stochastic gradient descent

1

2 Minibatching

Affine + Sigmoid: (applied to N data points simultaneously)

$$\mathbf{Y} = \sigma(\underbrace{\mathbf{XA}}_{=\mathbf{U}} + \mathbf{B})$$

► Each row in $\mathbf{X} \in \mathbb{R}^{N \times D}$ is a data point, bias $\mathbf{b} \in \mathbb{R}^M$ is broadcast to $\mathbf{B} \in \mathbb{R}^{N \times M}$

Loops now include **batch index b** :

```
for b,i    U.value[b, i] = 0
for b,i,j  U.value[b, i] += X.value[b, j] * A.value[j, i]
for b,i    Y.value[b, i] =  $\sigma$ (U.value[b, i] + B.value[i])
```

- Only inputs and outputs depend on batch index b , not the parameters (e.g., A, B)
- By convention, the gradients are averaged over the batch

The XOR Problem

Logistic Regression Model:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x}) \quad \text{with} \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

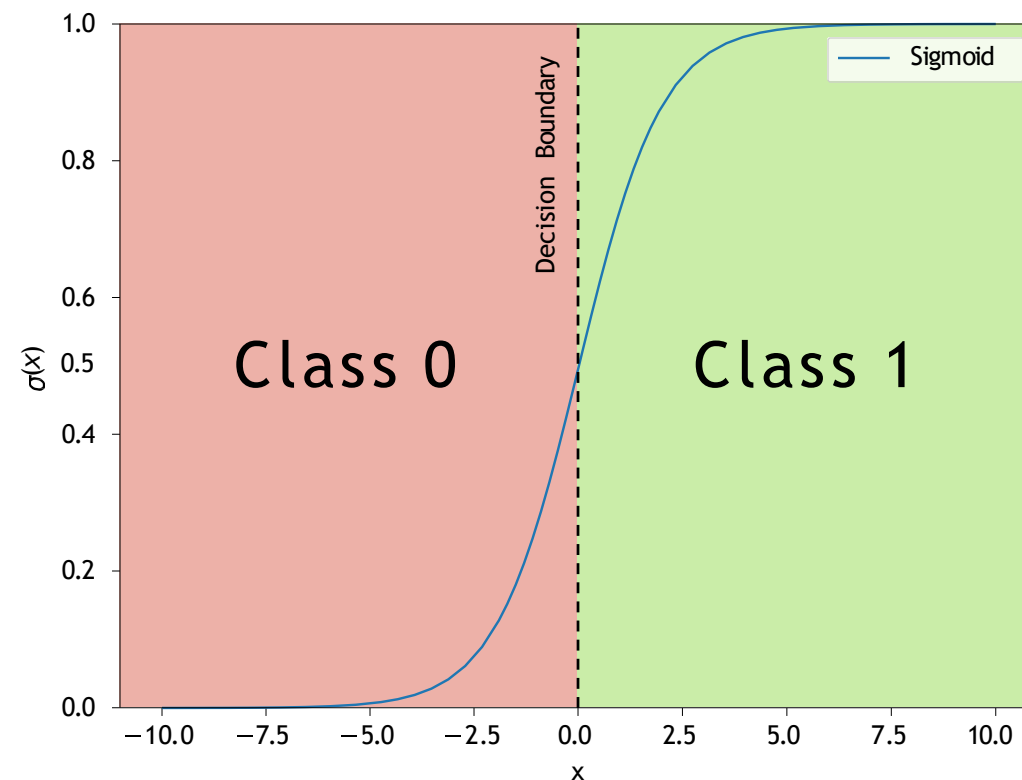
- Which problems can we solve with such a simple linear classifier?

Example: 2D Logistic Regression

- ▶ Let $\mathbf{x} \in \mathbb{R}^2$

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + w_0) \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

- ▶ Decision boundary: $\mathbf{w}^T \mathbf{x} + w_0 = 0$
- ▶ Decide for class 1 $\Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$
- ▶ Decide for class 0 $\Leftrightarrow \mathbf{w}^T \mathbf{x} < -w_0$



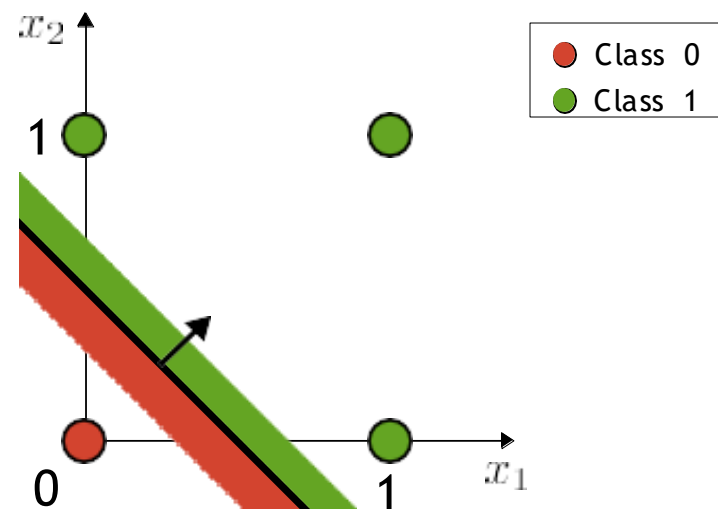
7 The XOR Problem

Linear Classifier:

$$\text{Class 1} \Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$$

$$\underbrace{\begin{pmatrix} 1 & 1 \end{pmatrix}}_{\mathbf{w}^T} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{\mathbf{x}} > \underbrace{0.5}_{-w_0}$$

x_1	x_2	OR(x_1, x_2)
0	0	0
0	1	1
1	0	1
1	1	1



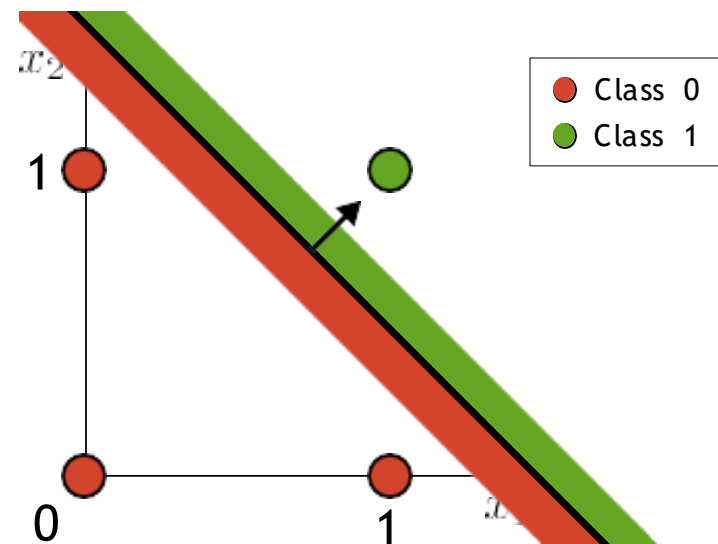
8 The XOR Problem

Linear Classifier:

$$\text{Class 1} \Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$$

$$\underbrace{\begin{pmatrix} 1 & 1 \end{pmatrix}}_{\mathbf{w}^T} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{\mathbf{x}} > \underbrace{0.5}_{-w_0}$$

x_1	x_2	$\text{AND}(x_1, x_2)$
0	0	0
0	1	0
1	0	0
1	1	1



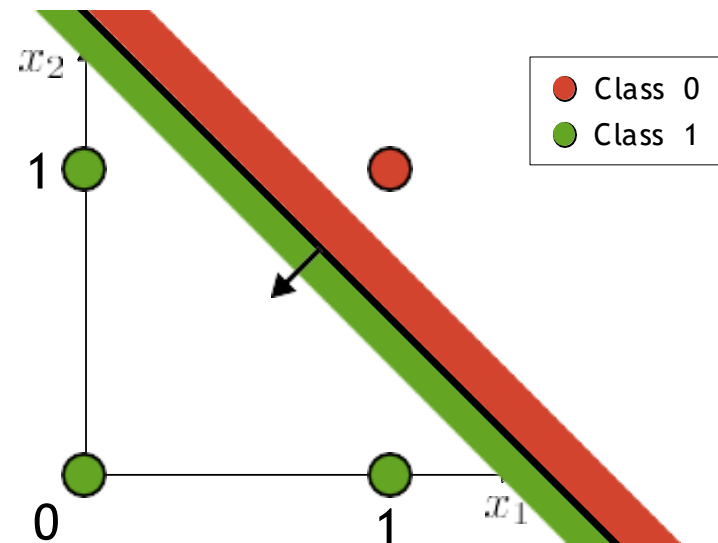
9 The XOR Problem

Linear Classifier:

$$\text{Class 1} \Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$$

$$\underbrace{\begin{pmatrix} 1 & 1 \end{pmatrix}}_{\mathbf{w}^T} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{\mathbf{x}} > \underbrace{0.5}_{-w_0}$$

x_1	x_2	NAND(x_1, x_2)
0	0	1
0	1	1
1	0	1
1	1	0

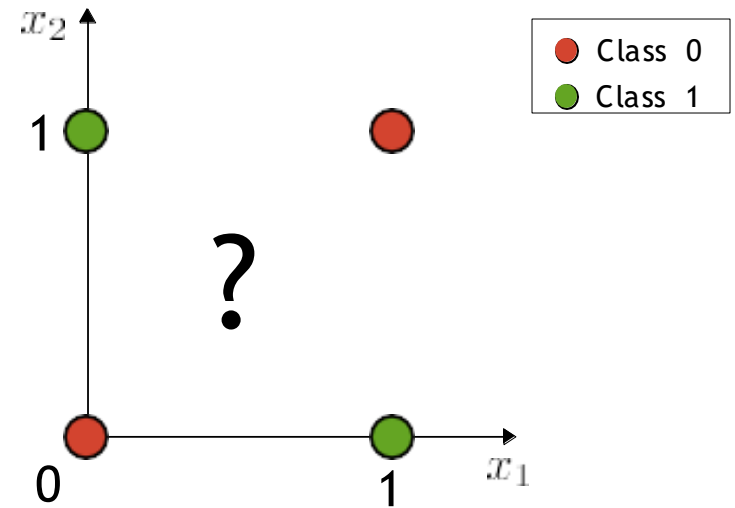


2 0 The XOR Problem

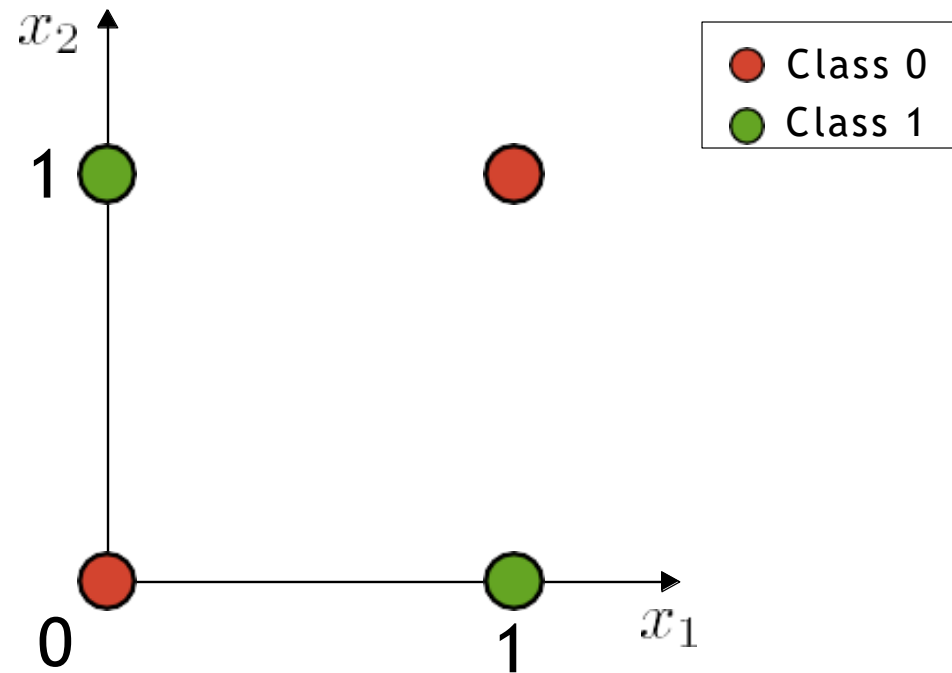
Linear Classifier:
Class 1 $\Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$

$$\underbrace{\begin{pmatrix} 1 & 1 \end{pmatrix}}_{\mathbf{w}^T} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{\mathbf{x}} > \underbrace{0.5}_{-w_0}$$

x_1	x_2	XOR(x_1, x_2)
0	0	0
0	1	1
1	0	1
1	1	0

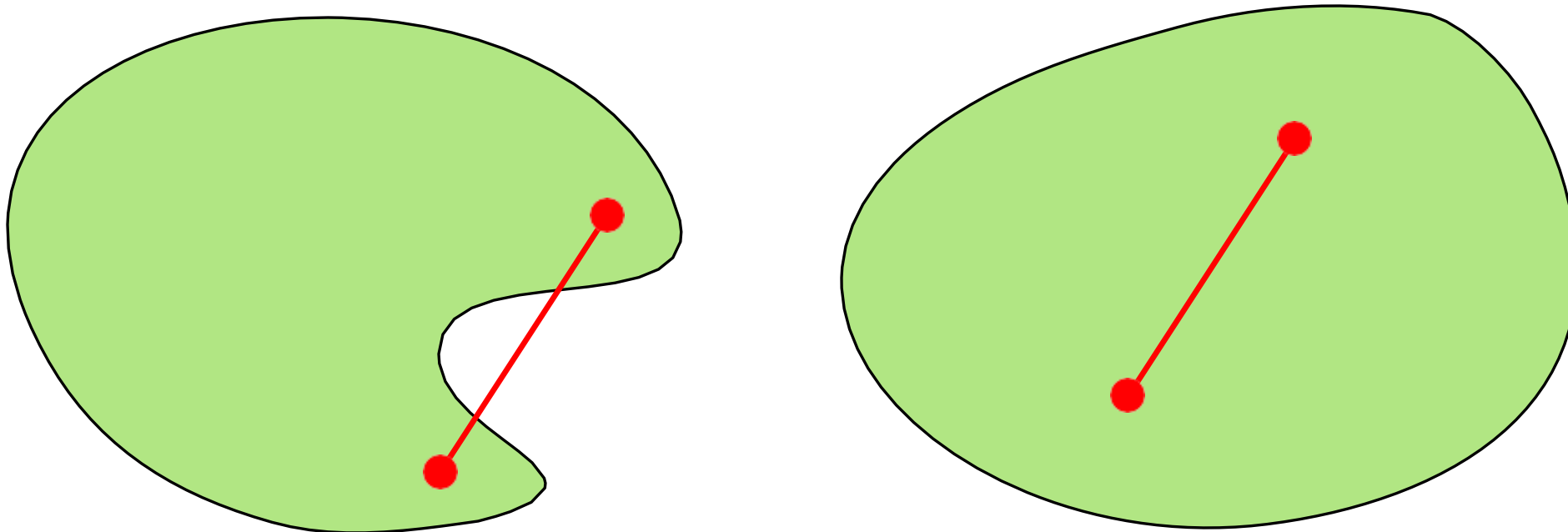


² ¹The XOR Problem



- Visually it is obvious that XOR is not linearly separable
- How can we formally prove this?

Convex Sets

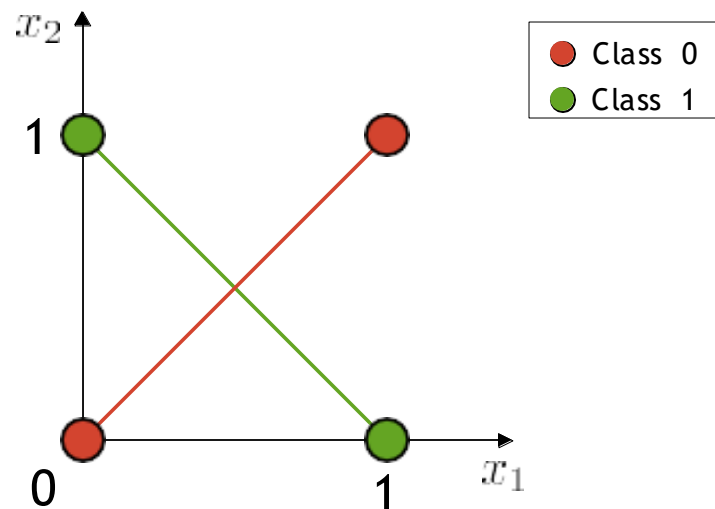


► A set S is **convex** if any line segment connecting 2 points in S lies entirely within S :

$$\mathbf{x}_1, \mathbf{x}_2 \in S \quad \Rightarrow \quad \lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2 \in S \quad \text{for } \lambda \in [0, 1]$$

3 The XOR Problem

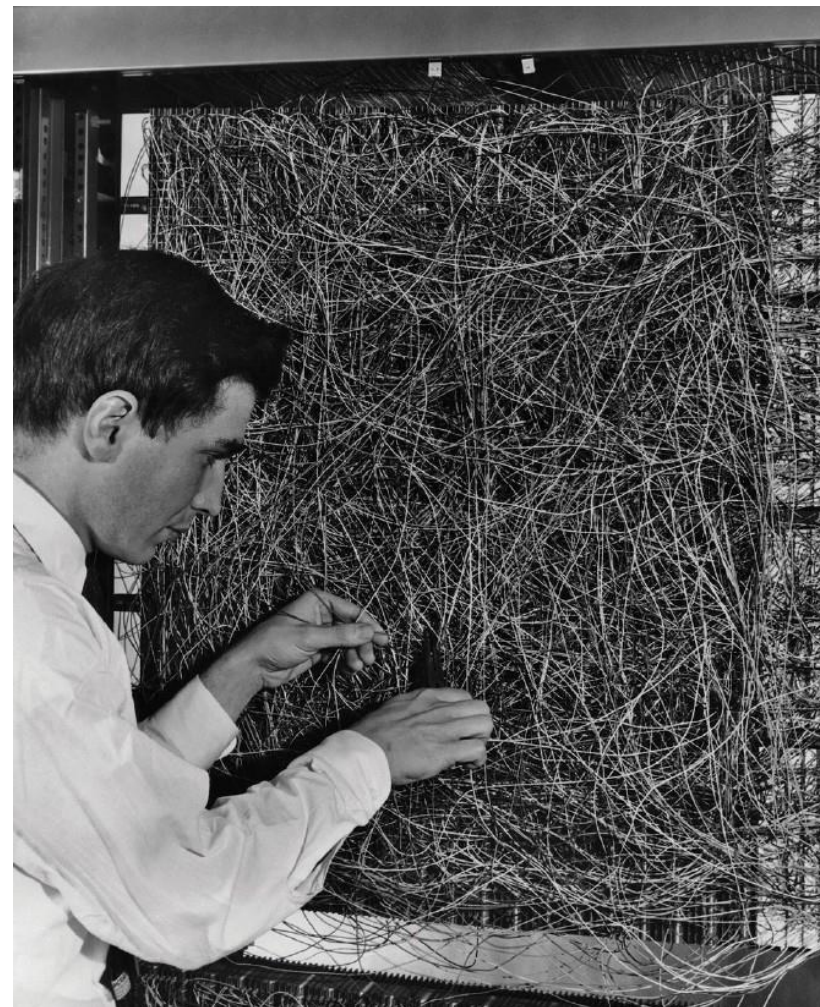
- ▶ Half-spaces (e.g., decision regions) are convex sets
- ▶ Suppose there was a feasible hypothesis. If the positive examples are in the positive half-space, then the green line segment must be as well.
- ▶ Similarly the red line must lie within the negative half-space.
- ▶ But the intersection can't lie in both half-spaces. Contradiction!



4 The XOR Problem

Some Historical Remarks:

- ▶ While linear classification showed some promising results in the 50s and 60s on simple image classification problems (Perceptron)
- ▶ But limitations became clear very soon (e.g., Minsky and Papert book “Perceptrons”, 1969)
- ▶ **XOR problem** is simple but cannot be solved as model capacity limited to linear decisions
- ▶ Led to decline in neural net research in the 70s
- ▶ How can we solve non-linear problems?

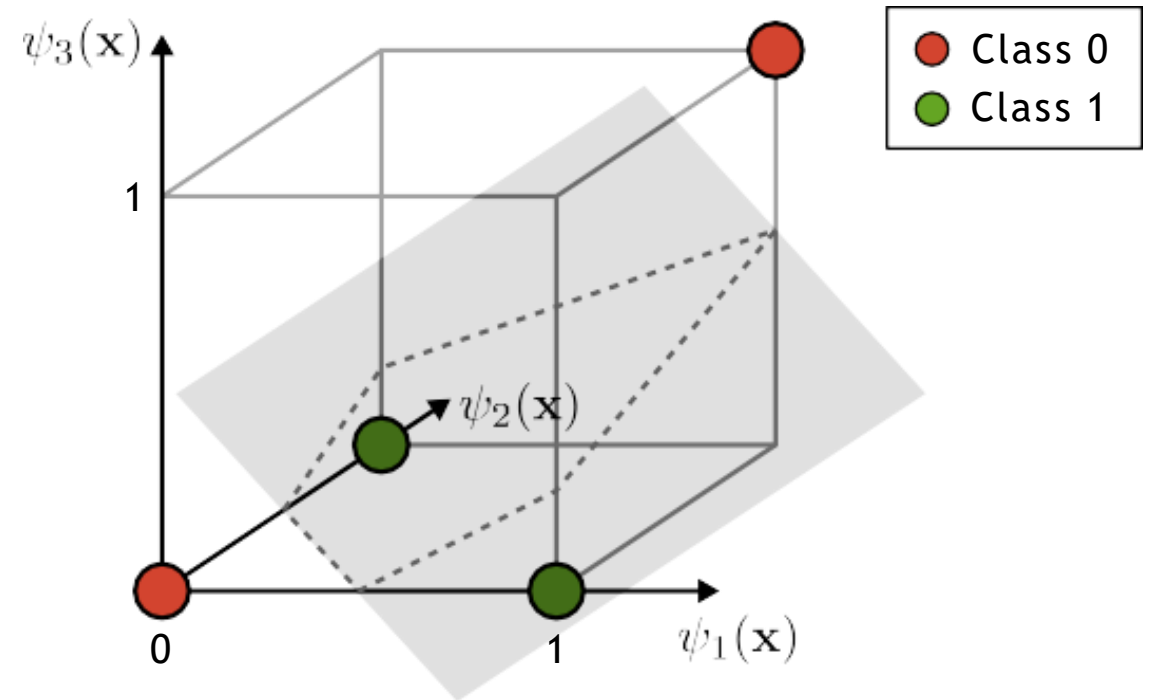


2 5 The XOR Problem

**Linear classifier with
non-linear features ψ :**

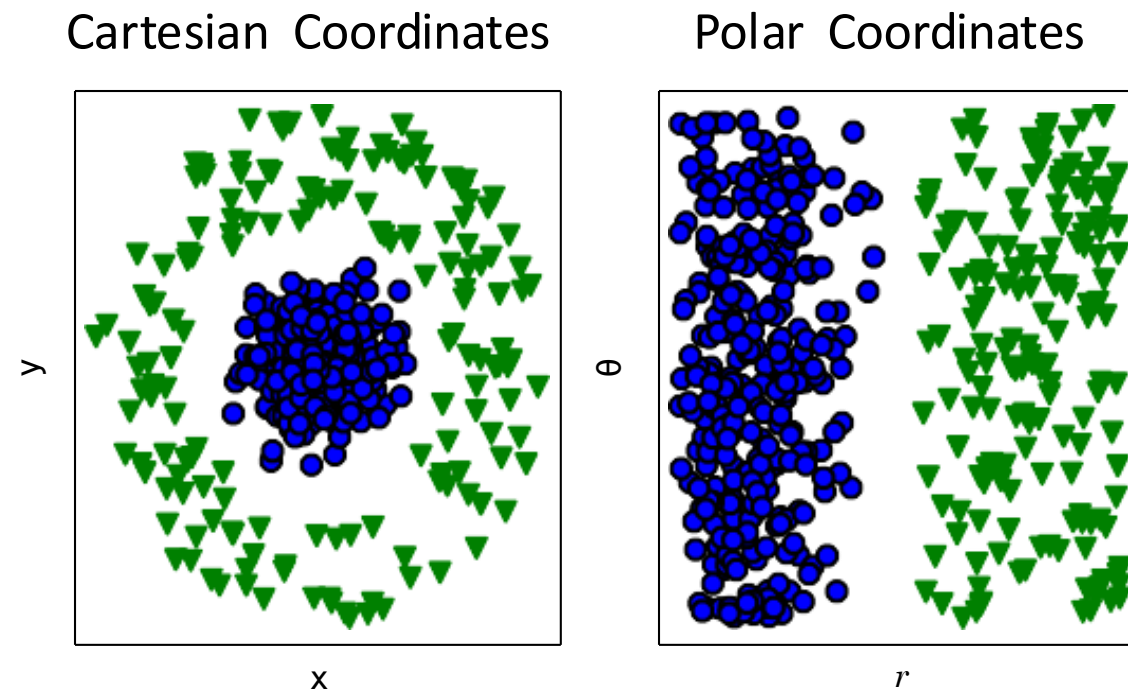
$$\mathbf{w}^\top \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{pmatrix}}_{\psi(\mathbf{x})} > -w_0$$

$x_1 x_2$	$\psi_1(\mathbf{X})$	$\psi_2(\mathbf{X})$	$\psi_3(\mathbf{X})$	XOR
0 0	0	0	0	0
0 1	0	1	0	1
1 0	1	0	0	1
1 1	1	1	1	0



- Non-linear features allow linear classifier to solve non-linear classification problems!
- Analogous to polynomial curve fitting

Representation Matters



- ▶ But how to choose the transformation? Can be very hard in practice.
- ▶ Yet, this was the dominant approach until the 2000s (vision, speech, ..)
- ▶ In this class we want to learn them $\Rightarrow$ **Representation learning**
- ▶ Human needs to choose the right function family rather than the correct function

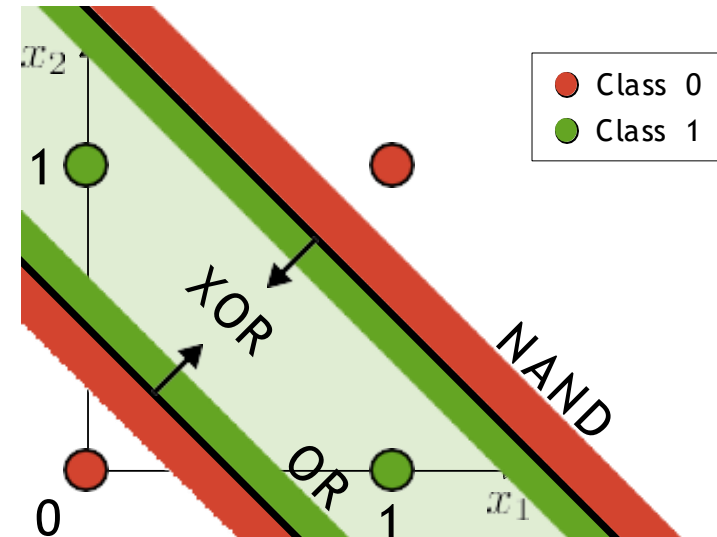
² 7 The XOR Problem

Linear Classifier:

$$\text{Class 1} \Leftrightarrow \mathbf{w}^T \mathbf{x} > -w_0$$

x_1	x_2	$\text{XOR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$$



The XOR Problem

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$$

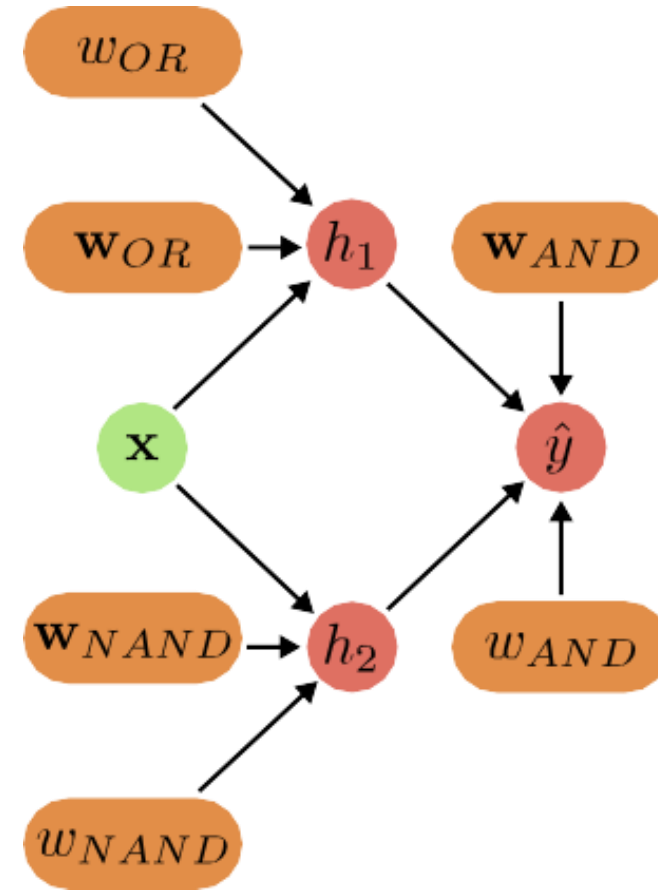
The above expression can be rewritten as a **program of logistic regressors**:

$$h_1 = \sigma(\mathbf{w}_{OR}^T \mathbf{x} + w_{OR})$$

$$h_2 = \sigma(\mathbf{w}_{NAND}^T \mathbf{x} + w_{NAND})$$

$$\hat{y} = \sigma(\mathbf{w}_{AND}^T \mathbf{h} + w_{AND})$$

Note that $\mathbf{h}(\mathbf{x})$ is a non-linear feature of $\mathbf{x}$.
We call $\mathbf{h}(\mathbf{x})$ a **hidden layer**.

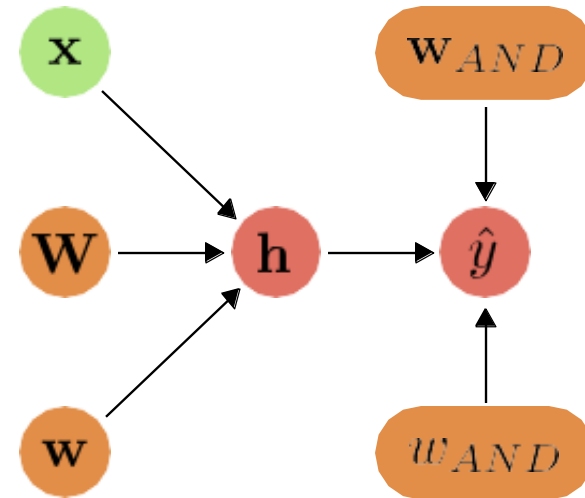


The XOR Problem

$$\text{XOR}(x_1, x_2) = \text{AND}(\text{OR}(x_1, x_2), \text{NAND}(x_1, x_2))$$

Writing the two 1D mappings $h_1(\mathbf{x})$ and $h_2(\mathbf{x})$ as a **single 2D mapping $\mathbf{h}(\mathbf{x})$** yields:

$$\mathbf{h} = \sigma \left(\underbrace{\begin{pmatrix} \mathbf{w}_{OR}^\top \\ \mathbf{w}_{NAND}^\top \end{pmatrix}}_{\mathbf{W}} \mathbf{x} + \underbrace{\begin{pmatrix} w_{OR} \\ w_{NAND} \end{pmatrix}}_{\mathbf{w}} \right)$$
$$\hat{y} = \sigma(\mathbf{w}_{AND}^\top \mathbf{h} + w_{AND})$$



Parameters can be learned using backprop. This is our first **Multi-Layer Perceptron!**

Multi-Layer Perceptrons

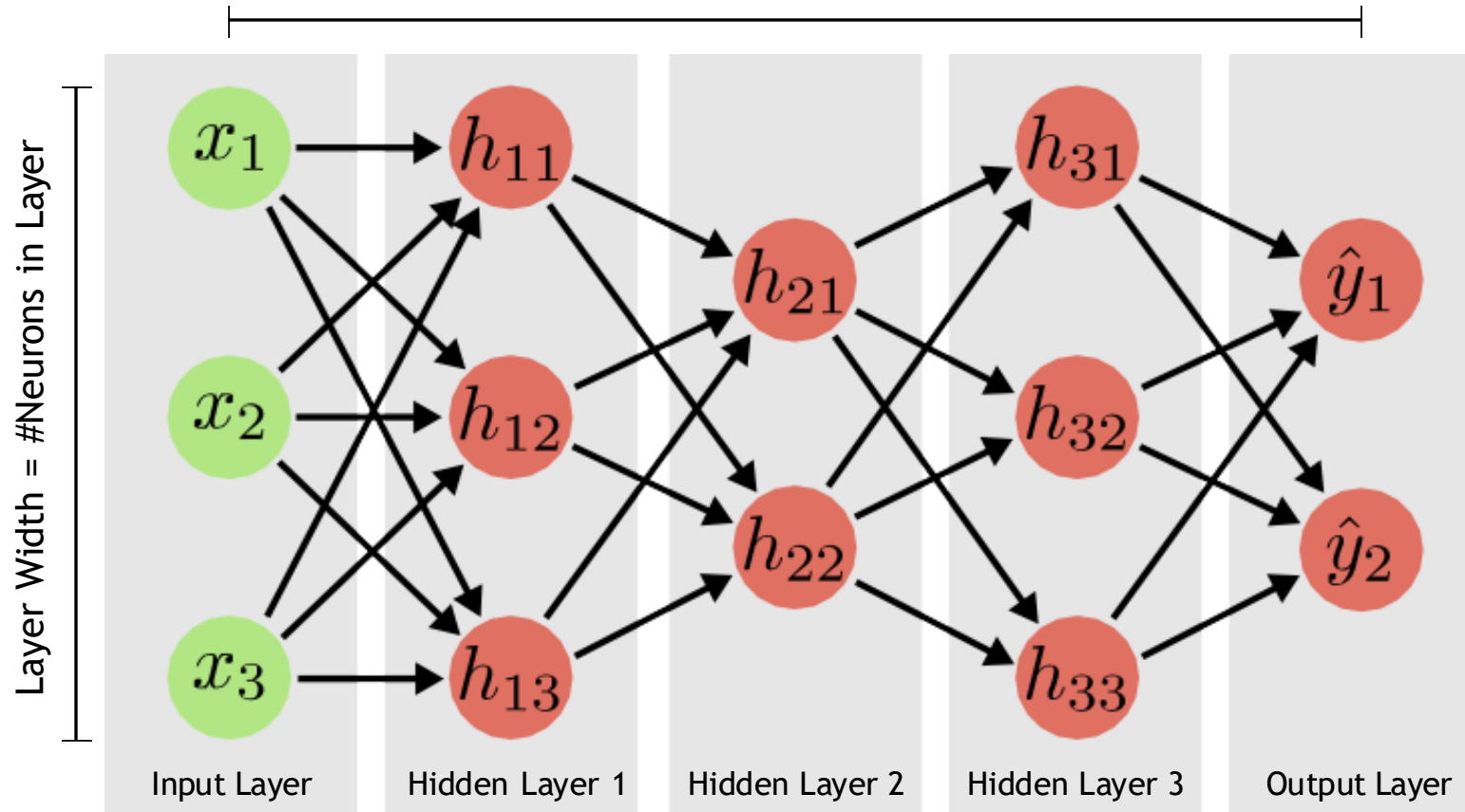
Multi-Layer Perceptrons

- ▶ MLPs are **feedforward** neural networks (no feedback connections)
- ▶ They **compose** several non-linear functions $\mathbf{f}(\mathbf{x}) = \hat{\mathbf{y}}(\mathbf{h}_3(\mathbf{h}_2(\mathbf{h}_1(\mathbf{x}))))$ where $\mathbf{h}_i(\cdot)$ are called **hidden layers** and $\hat{\mathbf{y}}(\cdot)$ is the **output layer**
- ▶ The data specifies only the behavior of the output layer (thus the name “hidden”)
- ▶ Each layer i comprises multiple **neurons** j which are implemented as **affine transformations** ($\mathbf{a}_{ij}^T \mathbf{x} + \mathbf{b}_{ij}$) followed by non-linear **activation functions** (g):

$$h_{ij} = g(\mathbf{a}_{ij}^T \mathbf{h}_{i-1} + \mathbf{b}_{ij})$$

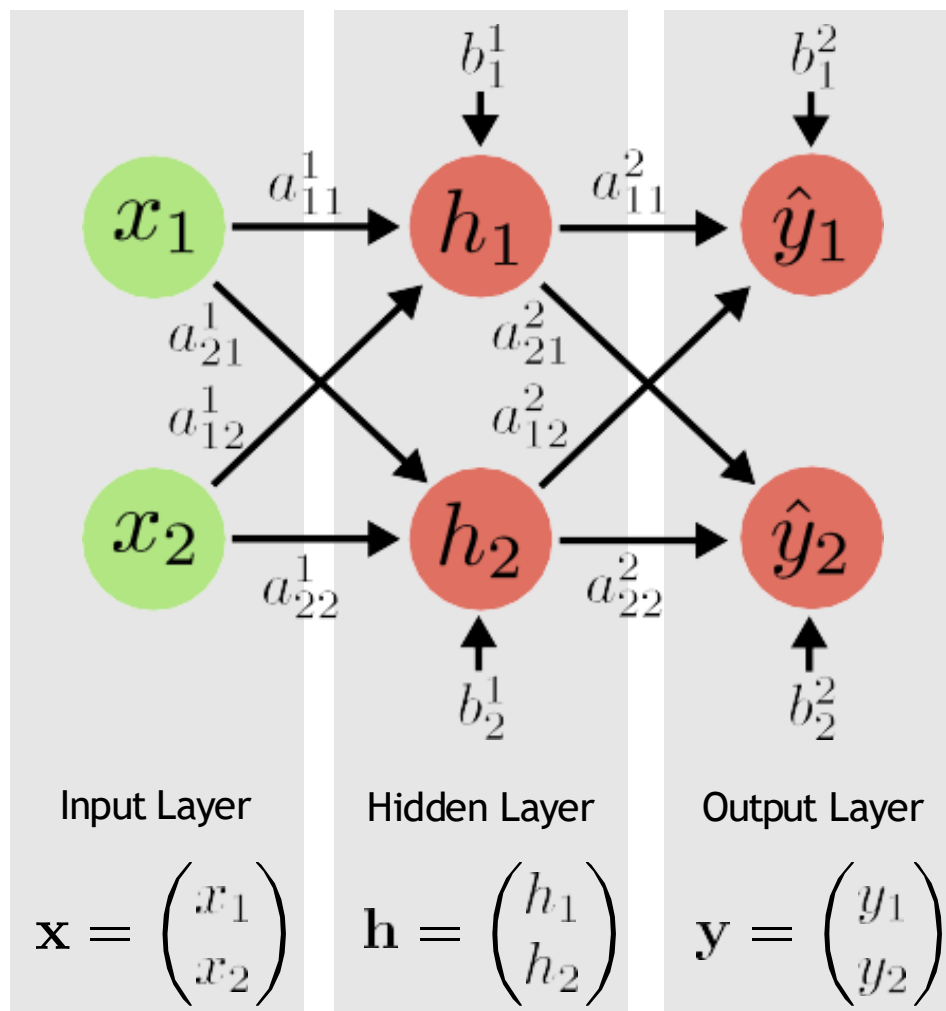
- ▶ Each neuron in each layer is **fully connected** to all neurons of the previous layer
- ▶ The overall length of the chain is the **depth** of the model $\Rightarrow$ “Deep Learning”
- ▶ The name MLP is misleading as we don’t use threshold units as in Perceptrons

Multi-Layer Perceptrons



- Neurons are grouped into layers, each neuron **fully connected** to all prev. ones
- **Hidden layer** $\mathbf{h}_i = g(\mathbf{A}_i \mathbf{h}_{i-1} + \mathbf{b}_i)$ with **activation function** $g(\cdot)$ and weights $\mathbf{A}_i, \mathbf{b}_i$
- **Output layer** $\mathbf{y} = h(\mathbf{A}_L \mathbf{h}_{L-1} + \mathbf{b}_L)$ with **activ. function** $h(\cdot)$ and weights $\mathbf{A}_L, \mathbf{b}_L$

Multi-Layer Perceptrons



$$\mathbf{h} = g(\mathbf{A}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{y} = h(\mathbf{A}_2 \mathbf{h} + \mathbf{b}_2)$$

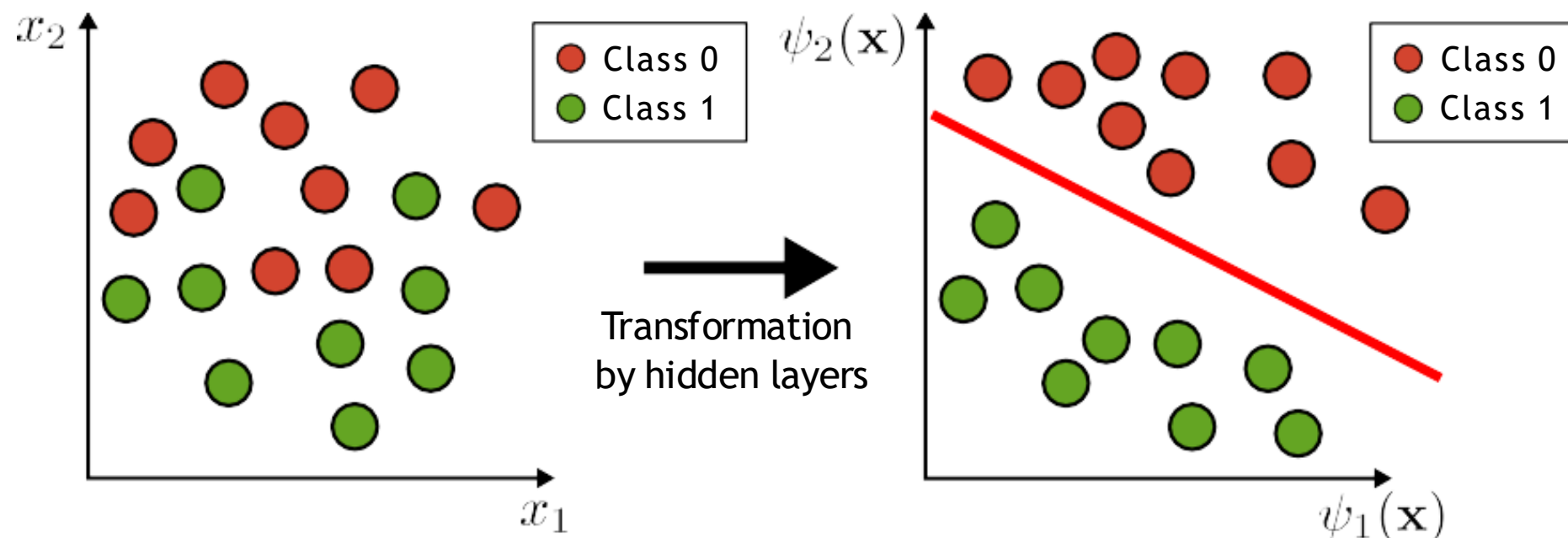
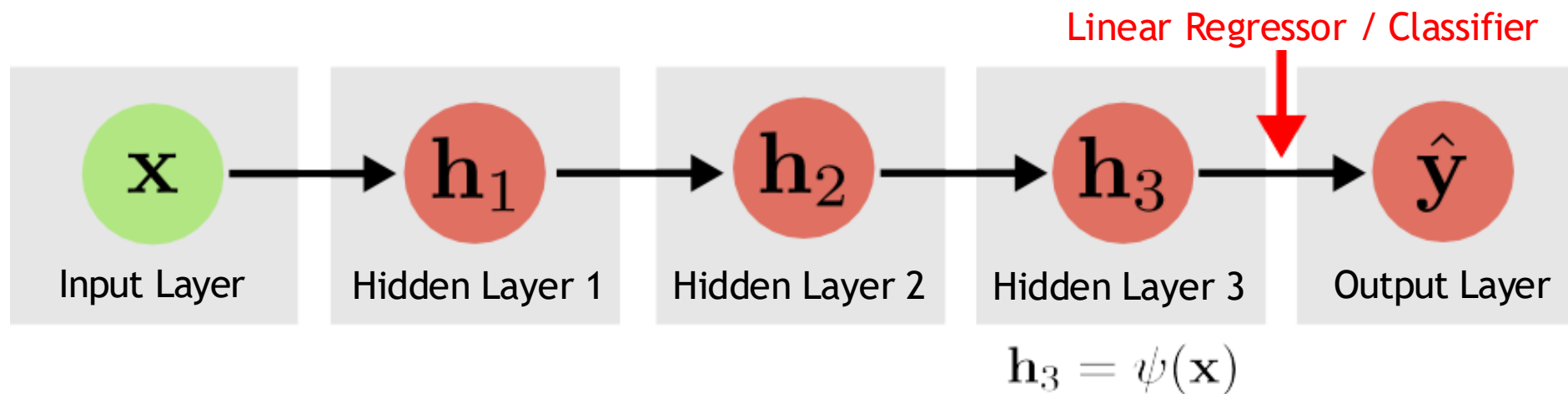
$$\mathbf{A}_l = \begin{pmatrix} a_{11}^l & a_{12}^l \\ a_{21}^l & a_{22}^l \end{pmatrix}$$

$$\mathbf{b}_l = \begin{pmatrix} b_1^l \\ b_2^l \end{pmatrix}$$

► Example MLP with $L = 2$ layers of width 2 and parameters $\{\mathbf{A}_1, \mathbf{A}_2, \mathbf{b}_1, \mathbf{b}_2\}$

3

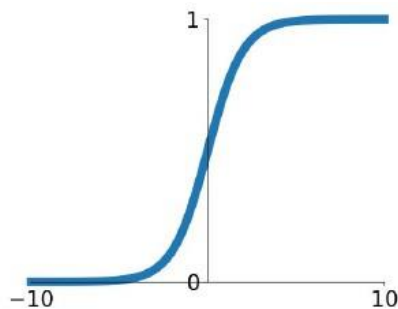
2 Feature Learning Perspective



3 Activation Functions $g(\cdot)$

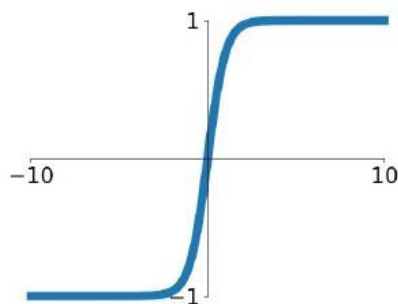
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



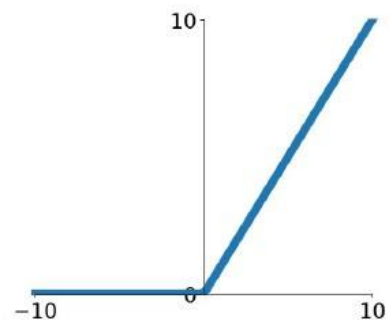
tanh

$$\tanh(x)$$



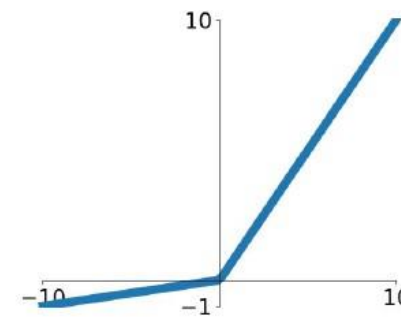
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

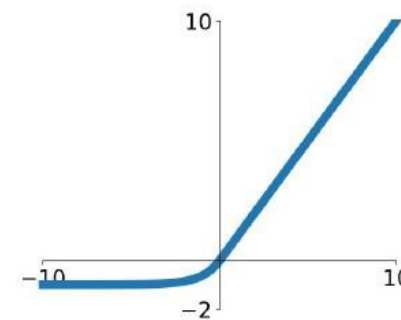


Maxout

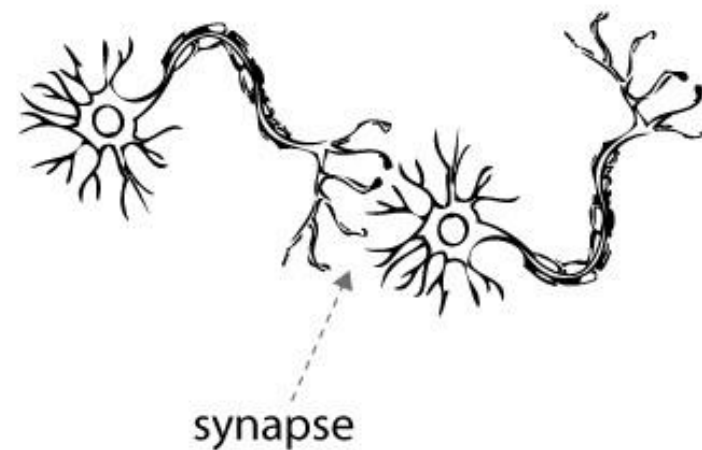
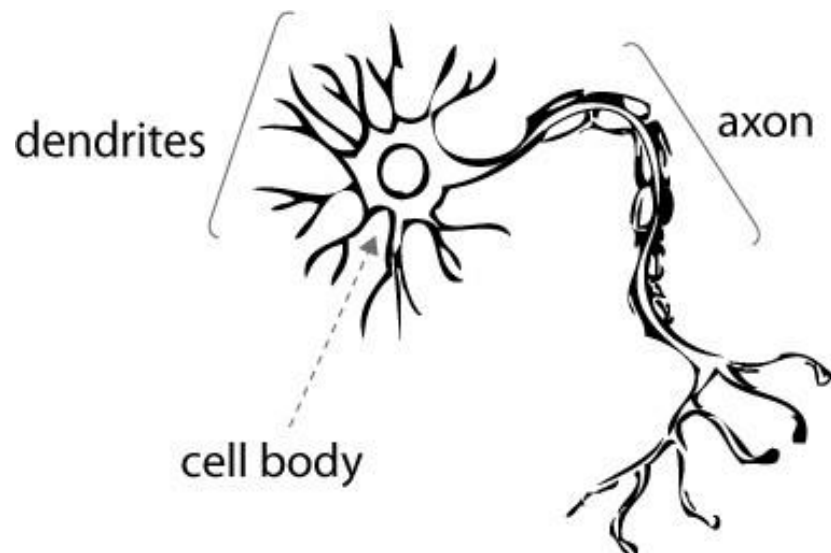
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



4 Neural Motivation



- ▶ Neurons in the brain are structured in layers
- ▶ They receive input from many other units and compute their own activation
- ▶ The sigmoid activation function is guided by neuroscientific observations
- ▶ However, the architecture and training of modern networks differs radically
- ▶ Our main goal is not to model the brain, but to achieve statistical generalization

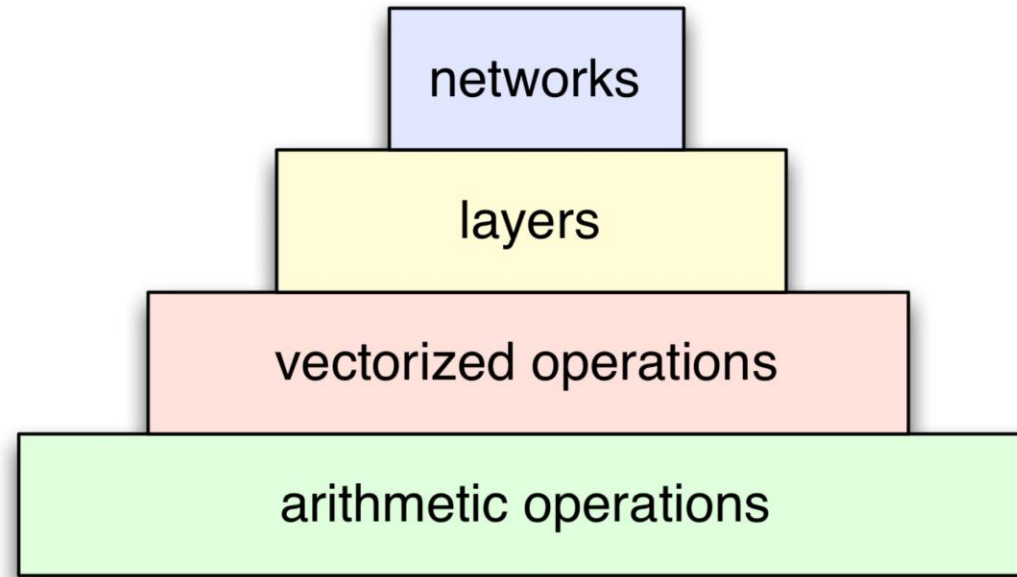
Algorithm for training an MLP using (stochastic) gradient descent:

1. Initialize weights $\mathbf{w}$, pick learning rate η and minibatch size $|X_{\text{batch}}|$
2. Draw (random) minibatch $X_{\text{batch}} \subseteq X$
3. For all elements $(\mathbf{x}, \mathbf{y}) \in X_{\text{batch}}$ of minibatch (in parallel) do:
 1. Forward propagate $\mathbf{x}$ through network to calculate $\mathbf{h}_1, \mathbf{h}_2, \dots, \hat{\mathbf{y}}$
 2. Backpropagate gradients through network to obtain $\nabla_{\mathbf{w}} L(\hat{\mathbf{y}}, \mathbf{y})$
4. Update gradients: $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \frac{1}{|X_{\text{batch}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in X_{\text{batch}}} \nabla_{\mathbf{w}} L(\hat{\mathbf{y}}, \mathbf{y})$
5. If validation error decreases, go to step 2, otherwise stop

Remarks:

- Large datasets typically do not fit into GPU memory $\Rightarrow |X_{\text{batch}}| < |X|$
- Our examples on the next slides are small $\Rightarrow |X_{\text{batch}}| = |X|$

Levels of Abstraction

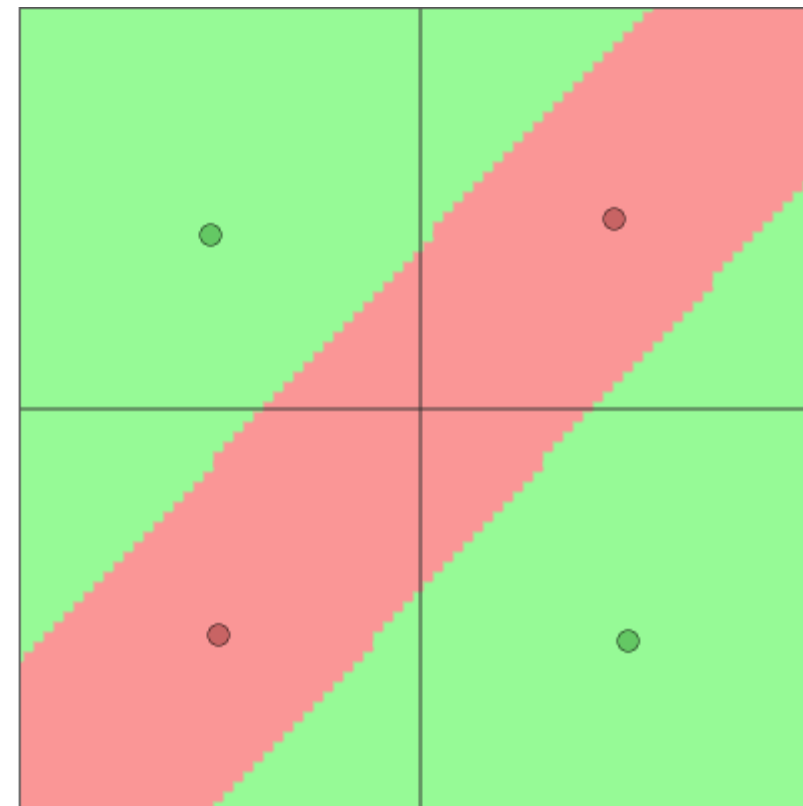


- ▶ When designing neural networks and machine learning algorithms, you'll need to simultaneously think at multiple level's of abstraction
- ▶ “The psychological profiling [of a programmer] is mostly the ability to shift levels of abstraction, from low level to high level. To see something in the small and to see something in the large.” [Donald E. Knuth]

7 The XOR Problem

```
layer_defs = [];  
layer_defs.push({type:'input', out_sx:1, out_sy:1, out_depth:2});  
layer_defs.push({type:'fc', num_neurons:2, activation:'tanh'});  
layer_defs.push({type:'fc', num_neurons:1, activation:'tanh'});  
layer_defs.push({type:'softmax', num_classes:2});  
  
net = new convnetjs.Net();  
net.makeLayers(layer_defs);  
  
trainer = new convnetjs.SGDTrainer(net, {learning_rate:0.01,  
momentum:0.1, batch_size:10, l2_decay:0.001});
```

change network

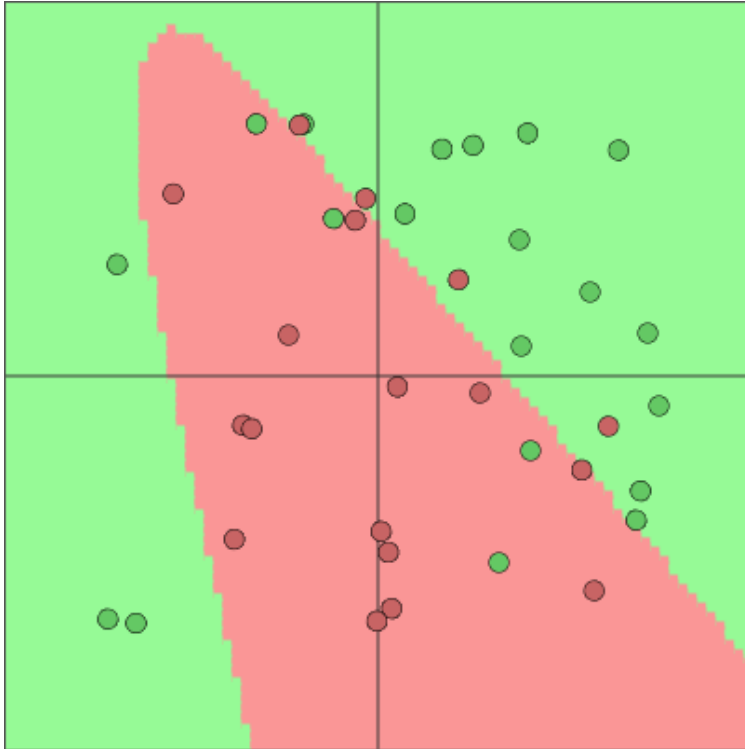


► Note that we have learned a boolean circuit! $\Rightarrow$ differentiable programming

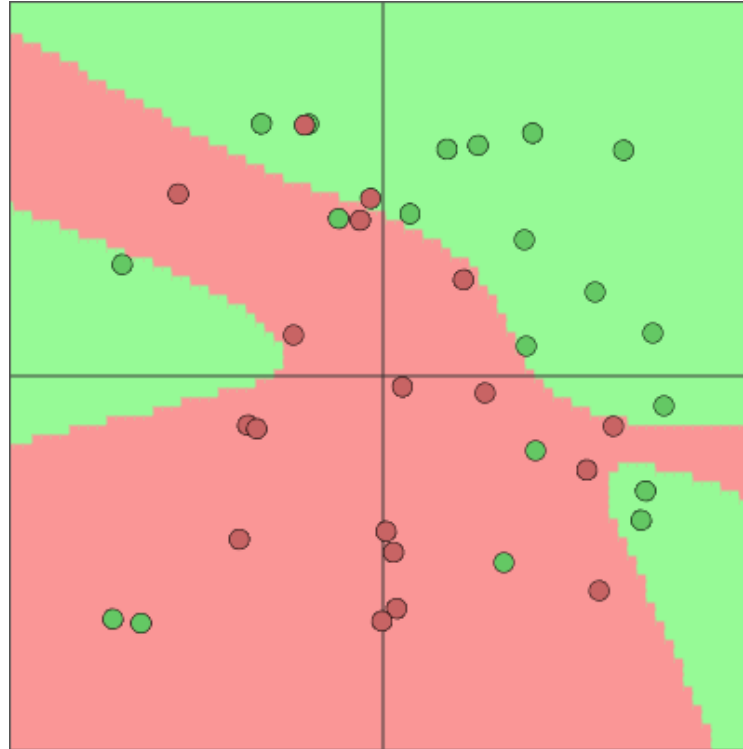
<https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

A More Challenging Problem

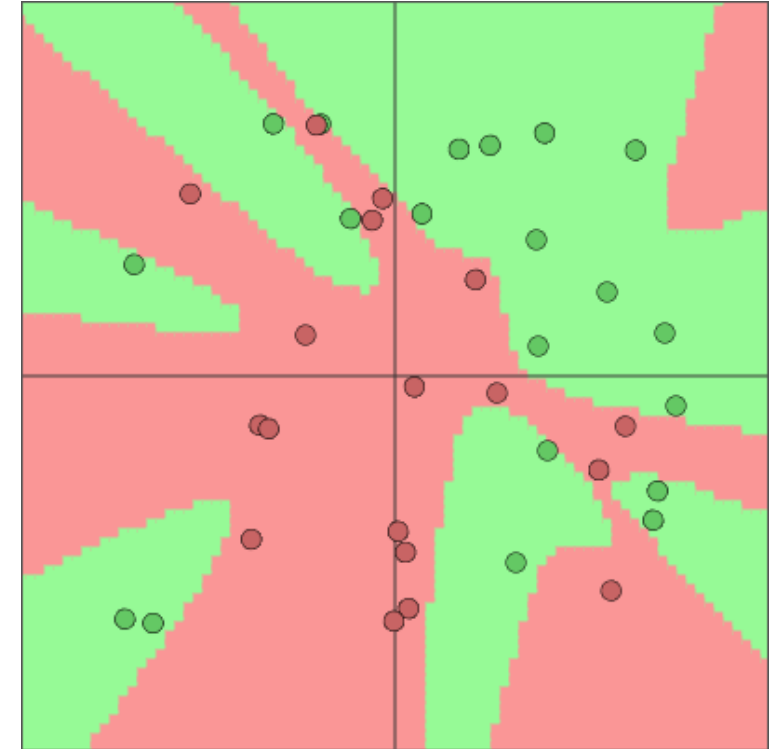
2 Hidden Neurons



5 Hidden Neurons



15 Hidden Neurons



<https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html>

Expressiveness

This following two-layer MLP

$$\mathbf{h} = g(\mathbf{A}_1 \mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{y} = g(\mathbf{A}_2 \mathbf{h} + \mathbf{b}_2)$$

can be written as

$$\mathbf{y} = g(\mathbf{A}_2 g(\mathbf{A}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2)$$

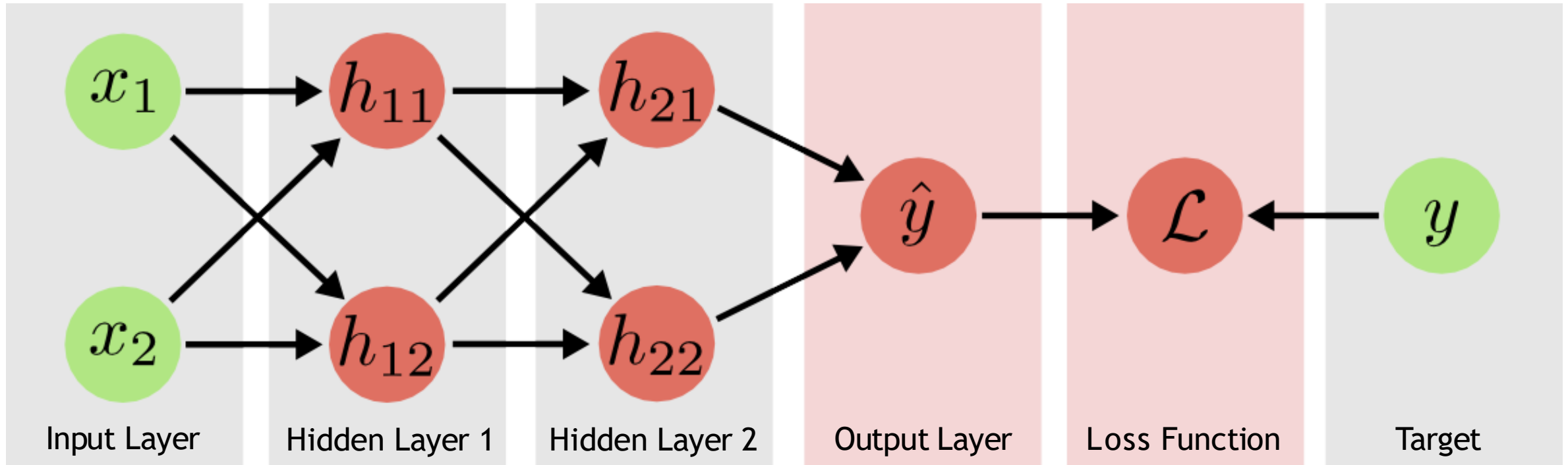
What if we would be using a linear activation function $g(\mathbf{x}) = \mathbf{x}$?

$$\mathbf{y} = \mathbf{A}_2 (\mathbf{A}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \mathbf{A}_2 \mathbf{A}_1 \mathbf{x} + \mathbf{A}_2 \mathbf{b}_1 + \mathbf{b}_2 = \mathbf{A} \mathbf{x} + \mathbf{b}$$

- ▶ With linear activations, a multi-layer network can only express linear functions
- ▶ What is the model capacity of MLPs with non-linear activation functions?

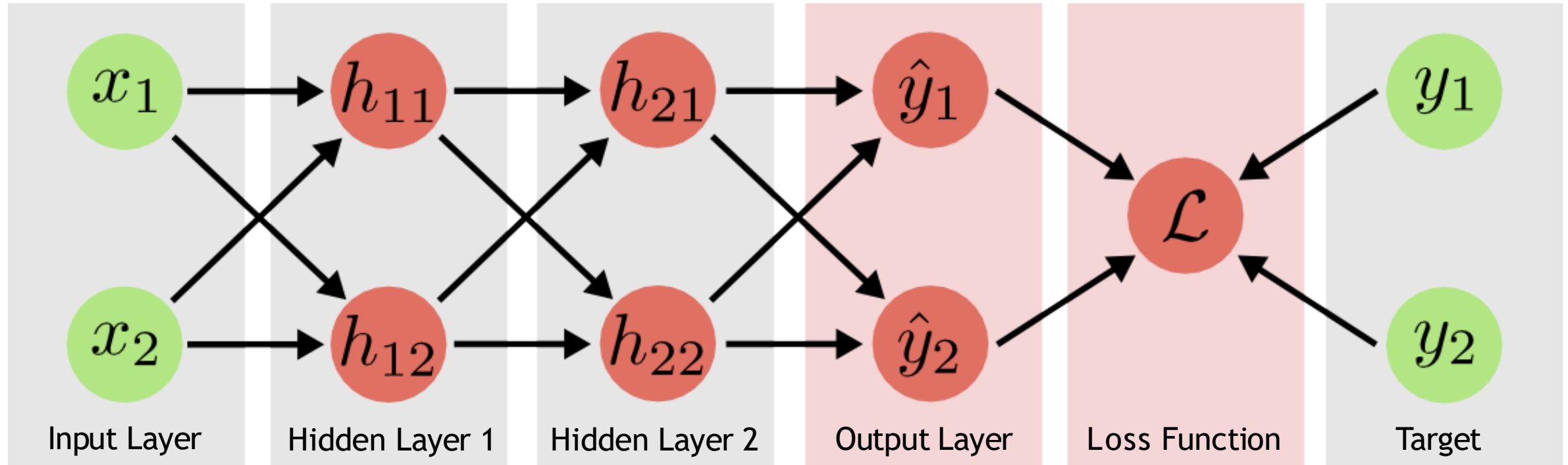
Output and Loss Functions

Output and Loss Functions



- ▶ The **output layer** is the last layer in a neural network which computes the output
- ▶ The **loss function** compares the result of the output layer to the target value(s)
- ▶ Choice of output layer and loss function depends on task (discrete, continuous, ..)

Output and Loss Functions

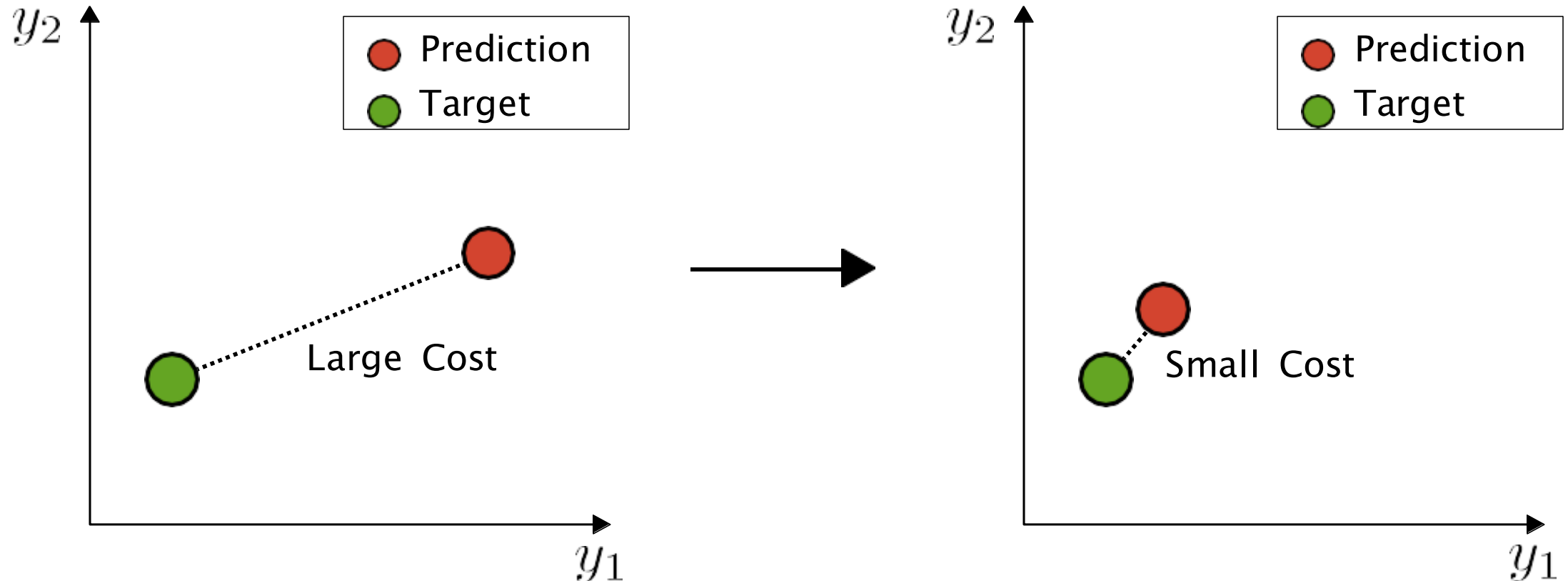


- ▶ The **output layer** is the last layer in a neural network which computes the output
- ▶ The **loss function** compares the result of the output layer to the target value(s)
- ▶ Choice of output layer and loss function depends on task (discrete, continuous, ..)

Loss Functions

What is the goal of optimizing the loss function?

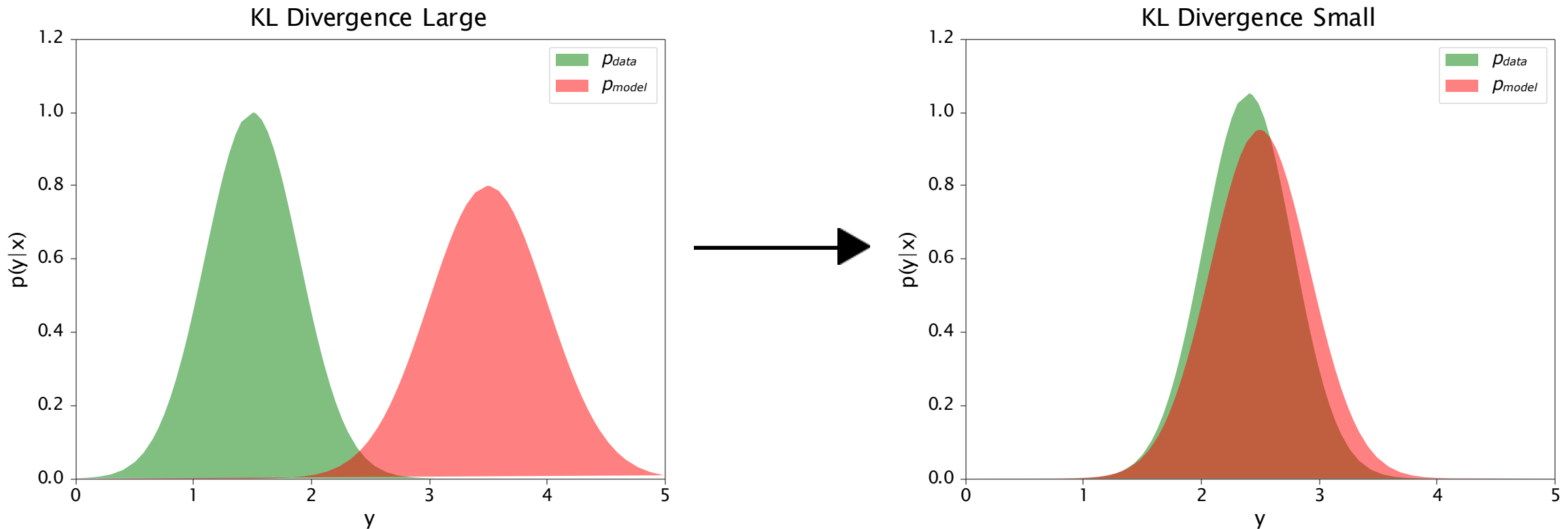
- Tries to make the **model output** (=prediction) similar to the **target** (=data)
- Think of the loss function as a **measure of cost** being paid for a prediction



Loss Functions

What is the goal of optimizing the loss function?

- Tries to make the **model output** (=prediction) similar to the **target** (=data)
- Think of the loss function as a **measure of cost** being paid for a prediction



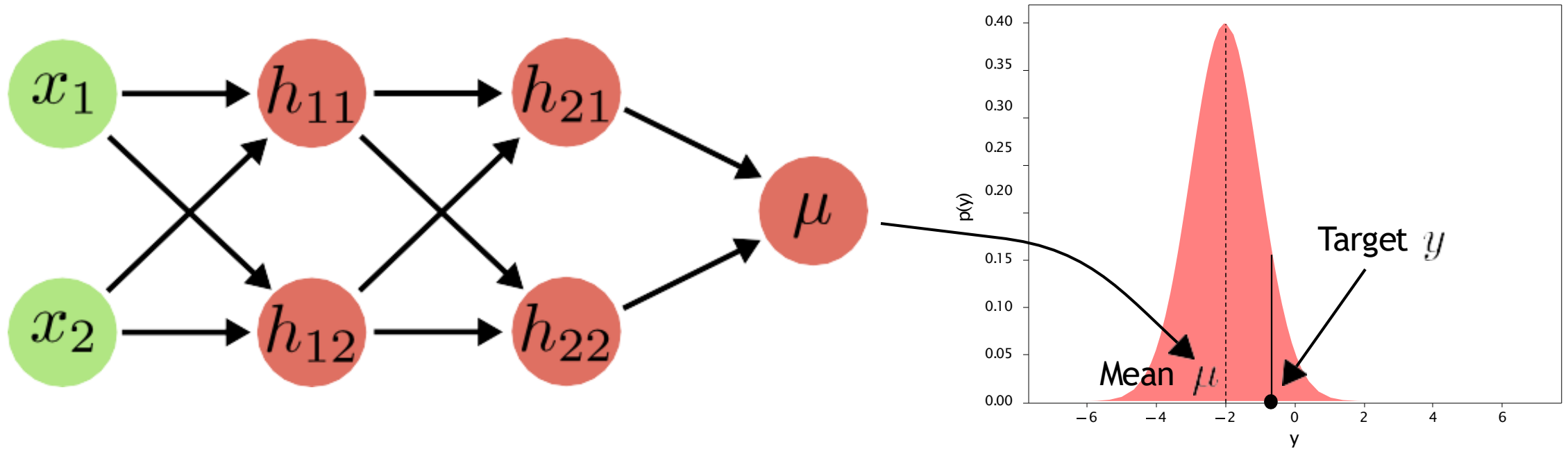
Loss Functions

How to design a good loss function?

- ▶ A loss function can be any differentiable function that we wish to optimize
- ▶ Deriving the cost function from the **maximum likelihood principle** removes the burden of manually designing the cost function for each model
- ▶ Consider the output of the neural network as **parameters of a distribution** over y_i

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} p_{model}(\mathbf{y}|\mathbf{X}, \mathbf{w}) \\ &\stackrel{\text{iid}}{=} \operatorname{argmax}_{\mathbf{w}} \prod_{i=1}^N p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} \underbrace{\sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w})}_{\text{Log-Likelihood}}\end{aligned}$$

7 Loss Functions



Example:

- Neural network $f_{\mathbf{w}}(\mathbf{x})$ predicts mean μ of Gaussian distribution over y :

$$p(y|\mathbf{x}, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(y - f_{\mathbf{w}}(\mathbf{x}))^2}{2\sigma^2} \right)$$

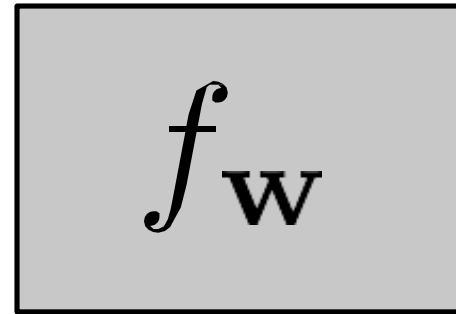
- We want to maximize the probability of the target y under this distribution

Recap: Regression

Input



Model

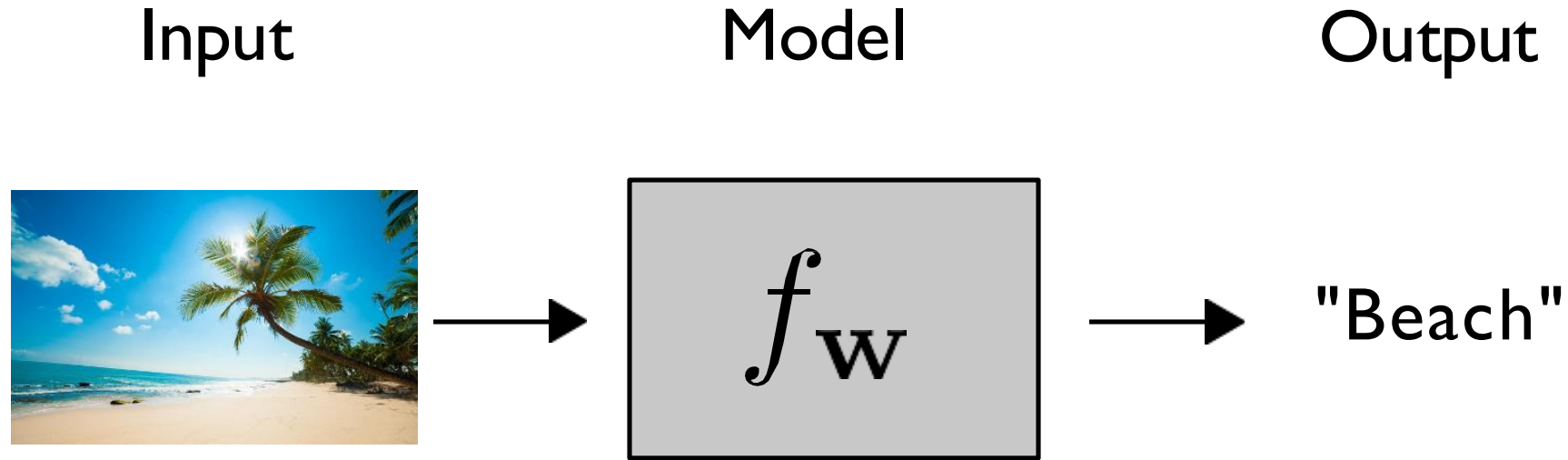


Output

143,52 €

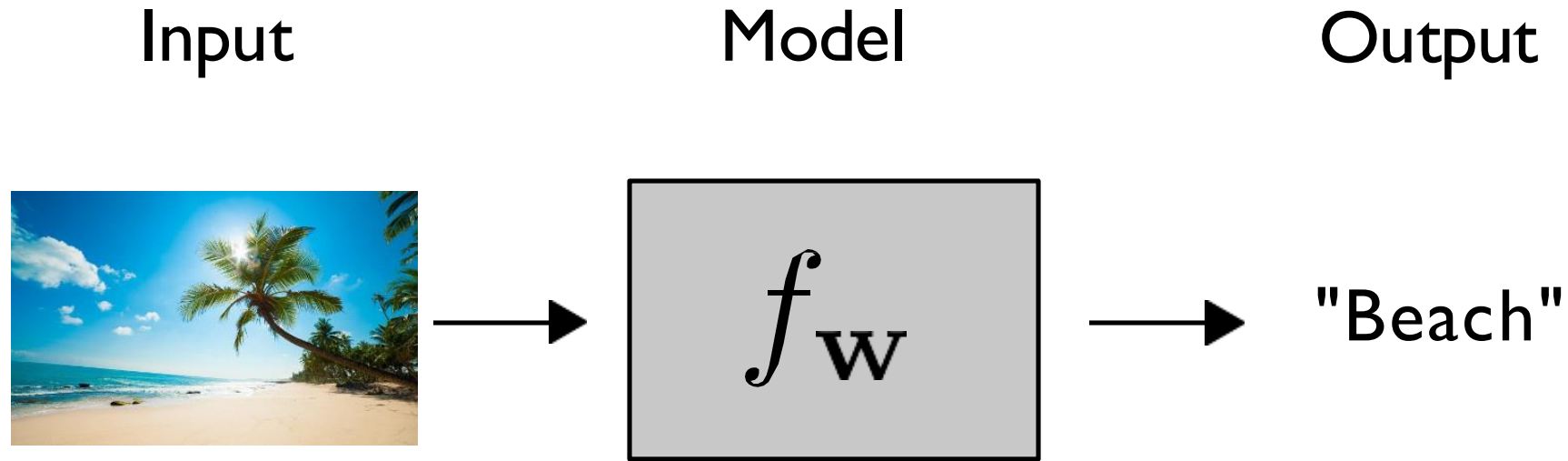
► Mapping: $f_w : \mathbb{R}^N \rightarrow \mathbb{R}$

Recap: Binary Classification



► **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{\text{"Beach"}, \text{"No Beach"}\}$

Recap: Multi-Class Classification



► **Mapping:** $f_{\mathbf{w}} : \mathbb{R}^{W \times H} \rightarrow \{\text{"Beach", "Mountain", "City", "Forest"}\}$

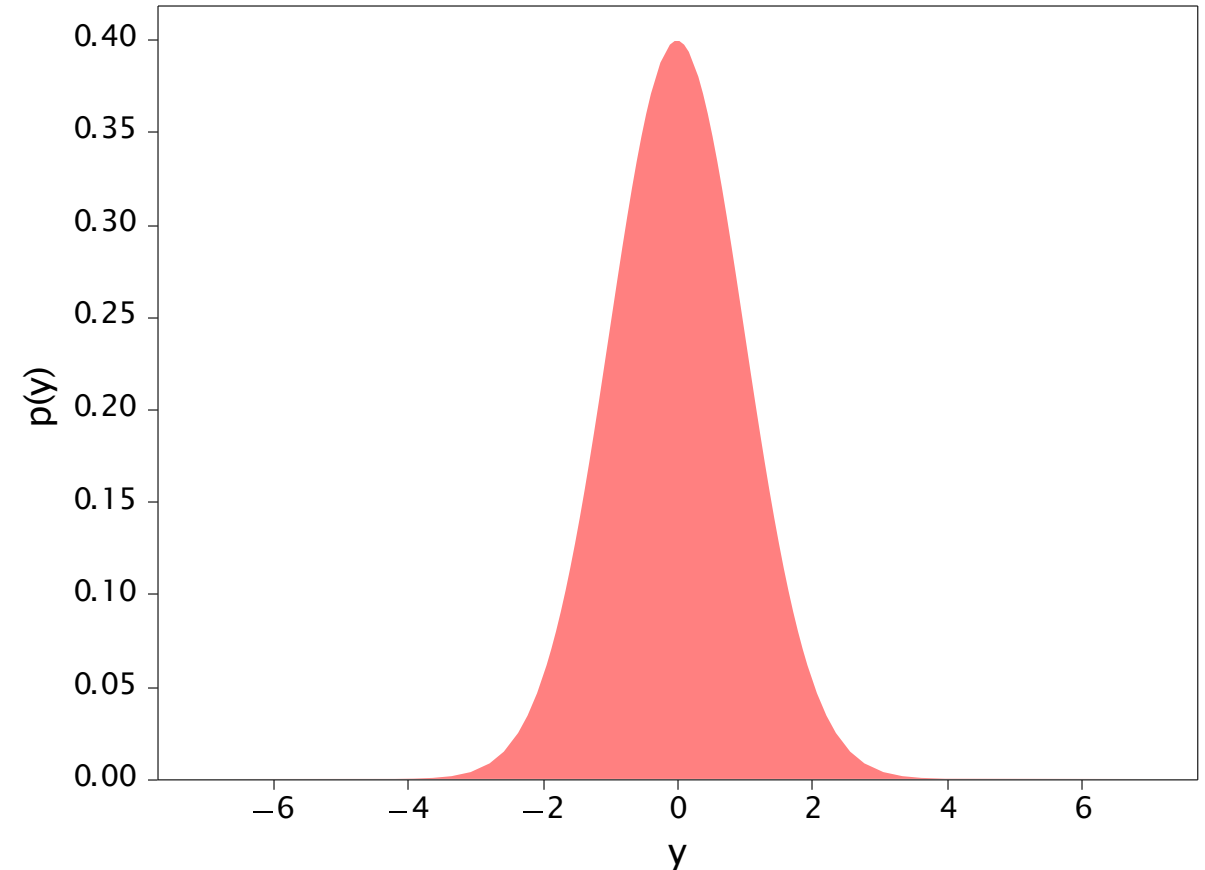
Regression Problems

Gaussian Distribution

Gaussian distribution:

$$p(y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - \mu)^2}{2\sigma^2}\right)$$

- ▶ μ : mean
- ▶ σ : standard deviation
- ▶ The distribution has thin “tails”:
 $p(y) \rightarrow 0$ quickly as $y \rightarrow \infty$
- ▶ It thus penalizes outliers strongly



Gaussian Distribution

Let $p_{model}(y|\mathbf{x}, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-f_{\mathbf{w}}(\mathbf{x}))^2}{2\sigma^2}\right)$ be a **Gaussian distribution**. We obtain:

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N \frac{1}{2} \log(2\pi\sigma^2) - \sum_{i=1}^N \frac{1}{2\sigma^2} (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 \\ &= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 \\ &= \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^N \underbrace{(f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2}_{L_2 \text{ Loss}}\end{aligned}$$

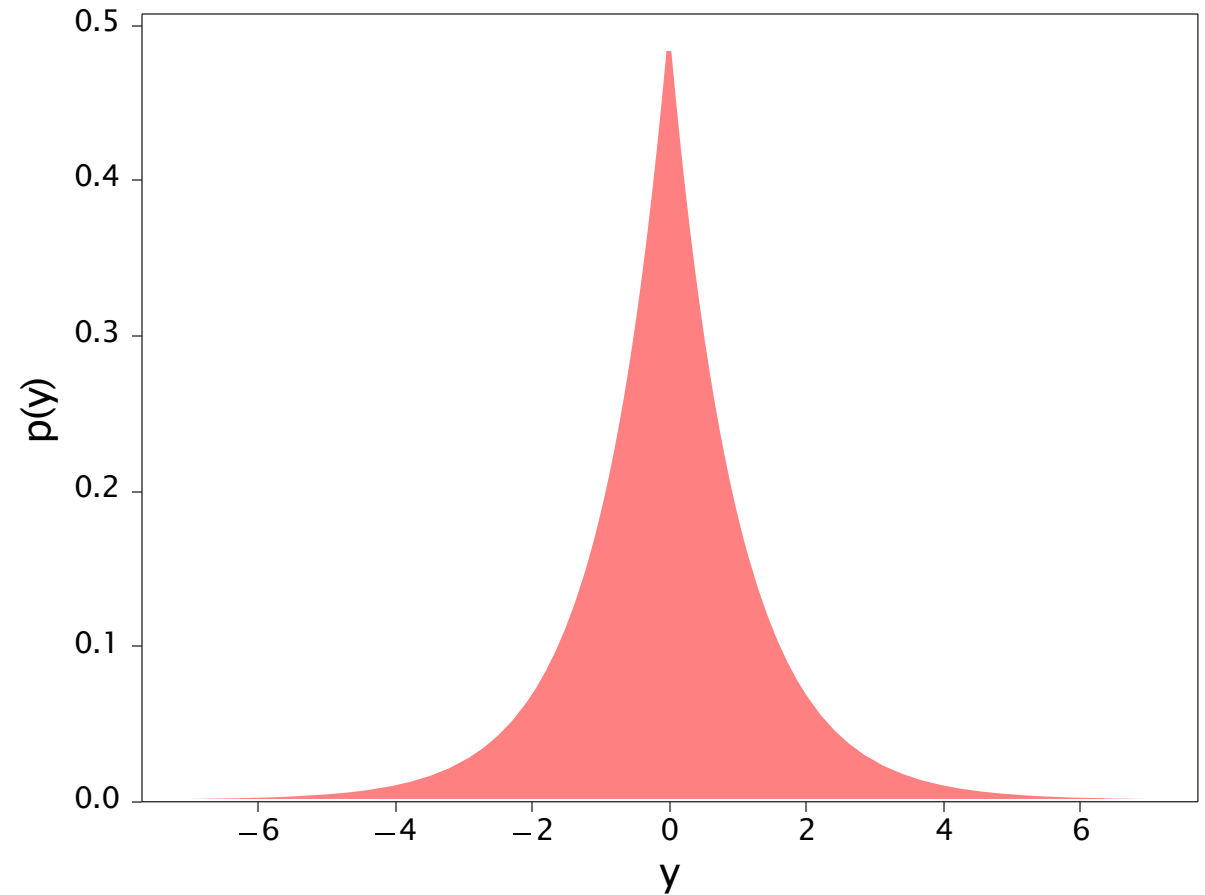
In other words, we minimize the **squared loss** ($=L_2$ loss), affected strongly by outliers.

Laplace Distribution

Laplace distribution:

$$p(y) = \frac{1}{2b} \exp\left(-\frac{|y - \mu|}{b}\right)$$

- ▶ μ : location
- ▶ b : scale
- ▶ The distribution has heavy “tails”:
 $p(y) \rightarrow 0$ more slowly as $y \rightarrow \infty$
- ▶ Penalizes outliers less strongly
- ▶ Thus often preferred in practice



Laplace Distribution

Let $p_{model}(y|\mathbf{x}, \mathbf{w}) = \frac{1}{2b} \exp\left(-\frac{|y-f_{\mathbf{w}}(\mathbf{x})|}{b}\right)$ be a **Laplace distribution**. We obtain:

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N \log(2b) - \sum_{i=1}^N \frac{1}{b} |f_{\mathbf{w}}(\mathbf{x}_i) - y_i| \\ &= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N |f_{\mathbf{w}}(\mathbf{x}_i) - y_i| \\ &= \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^N \underbrace{|f_{\mathbf{w}}(\mathbf{x}_i) - y_i|}_{L_1 \text{ Loss}}\end{aligned}$$

We minimize the **absolute loss** ($=L_1$ loss) which is more robust than L_2 .

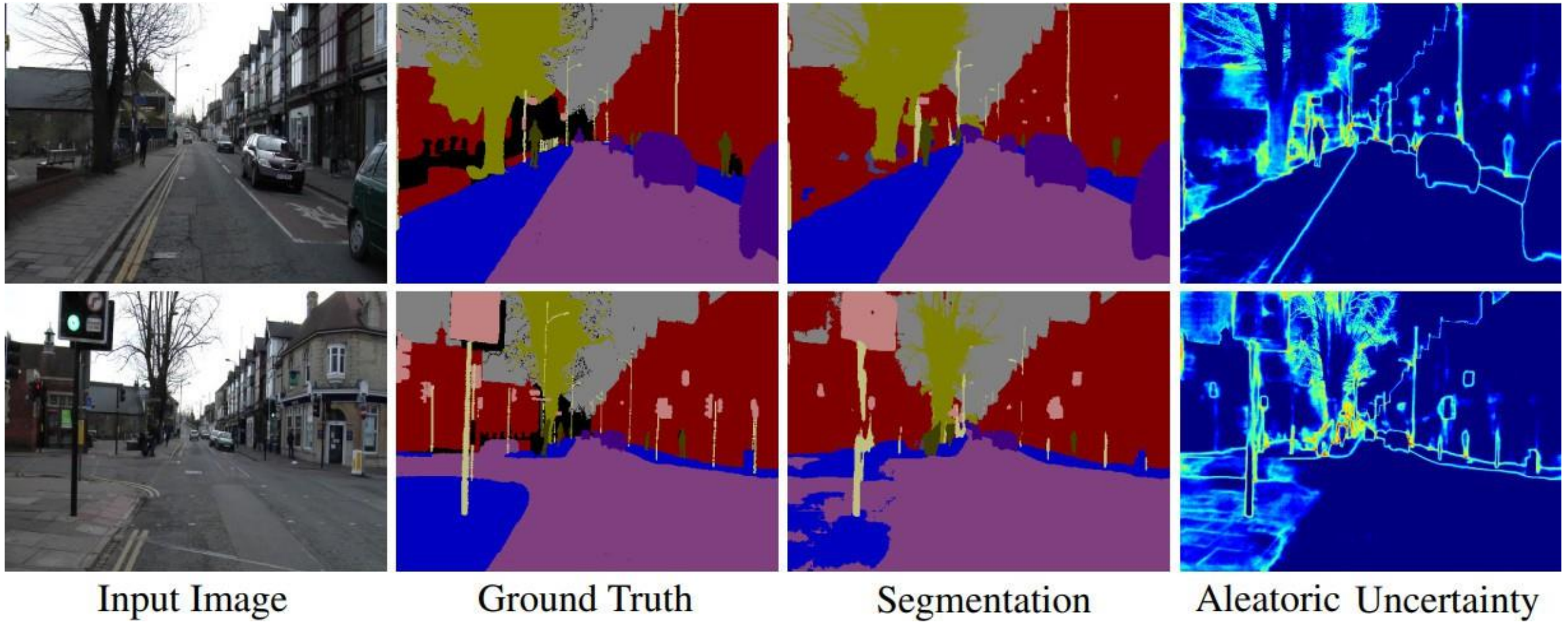
4 Predicting all parameters

Let $p_{model}(y|\mathbf{x}, \mathbf{w}) = \frac{1}{2 g_{\mathbf{w}}(\mathbf{x})} \exp\left(-\frac{|y - f_{\mathbf{w}}(\mathbf{x})|}{g_{\mathbf{w}}(\mathbf{x})}\right)$ be a **Laplace distribution**. We obtain:

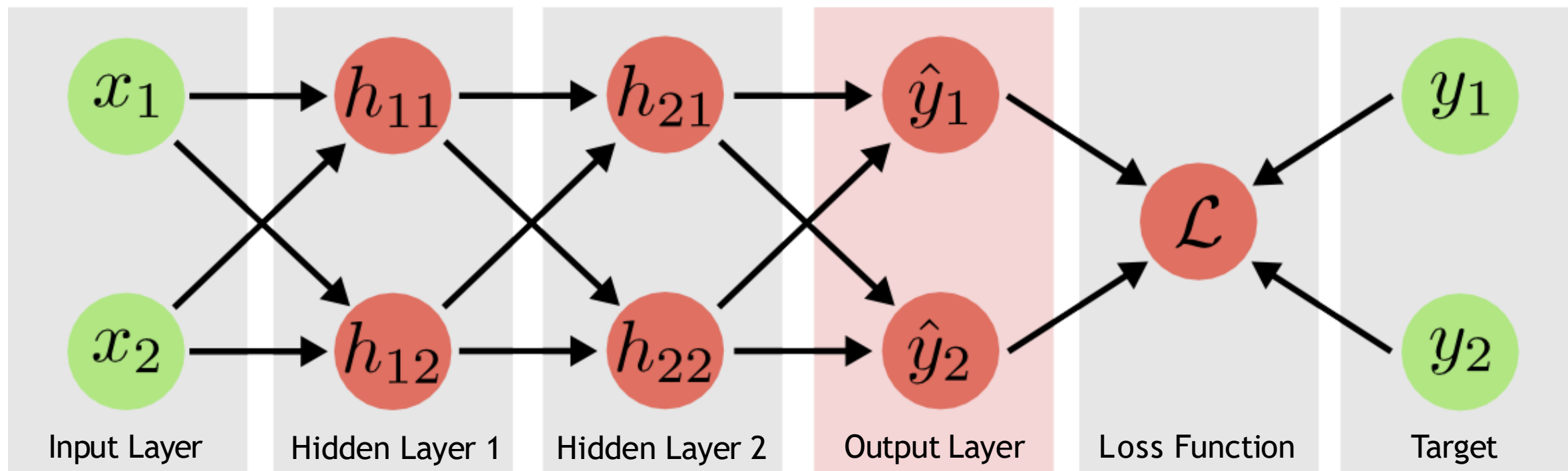
$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} - \sum_{i=1}^N \log(2 g_{\mathbf{w}}(\mathbf{x}_i)) - \sum_{i=1}^N \frac{1}{g_{\mathbf{w}}(\mathbf{x}_i)} |f_{\mathbf{w}}(\mathbf{x}_i) - y_i|\end{aligned}$$

In this case, we predict both the **location** μ and the **scale** b with a neural network. $f_{\mathbf{w}}(\mathbf{x}_i)$ and $g_{\mathbf{w}}(\mathbf{x}_i)$ are typically the same except for the last layer. This allows for estimating the **aleatoric uncertainty** (observation noise) with the neural network itself.

1 5 Predicting all Parameters

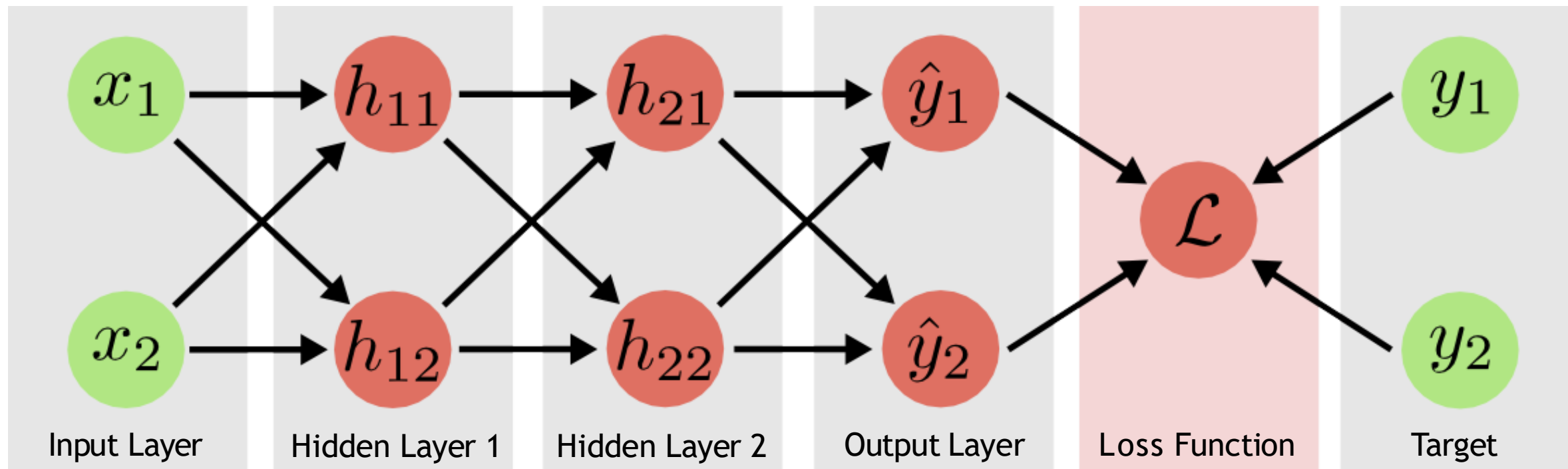


Output Layer for Regression Problems



- For most outputs (e.g., $\mu \in \mathbb{R}$), the output layer is linear (no non-linearity)
- For some outputs (e.g., $b \in \mathbb{R}^+$), we need a squashing function (ReLU, softplus)

8 Loss Function for Regression Problems



- Gaussian/Laplacian model distribution correspond to L_2 and L_1 loss functions
- It is also possible to predict uncertainty (variance/scale) or multiple modes (MDN)

Classification Problems

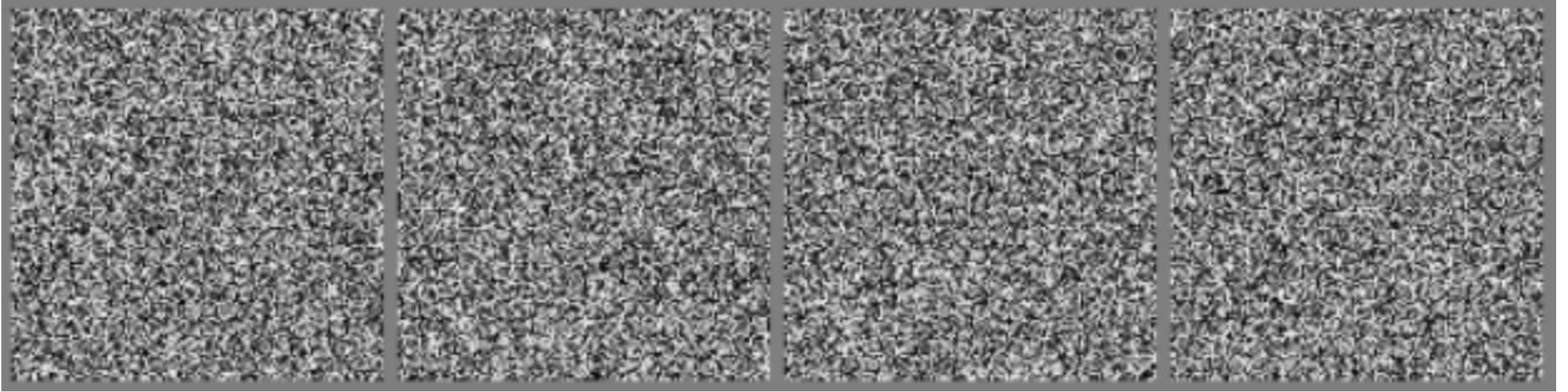
Image Classification



MNIST Handwritten Digits:

- ▶ One of the most popular datasets in ML (many variants, still in use today)
- ▶ Based on a data from the National Institute of Standards and Technology
- ▶ Hand written by Census Bureau employees and high-school children
- ▶ Resolution: 28 x 28 pixels, 60k training samples with labels, 10k test samples

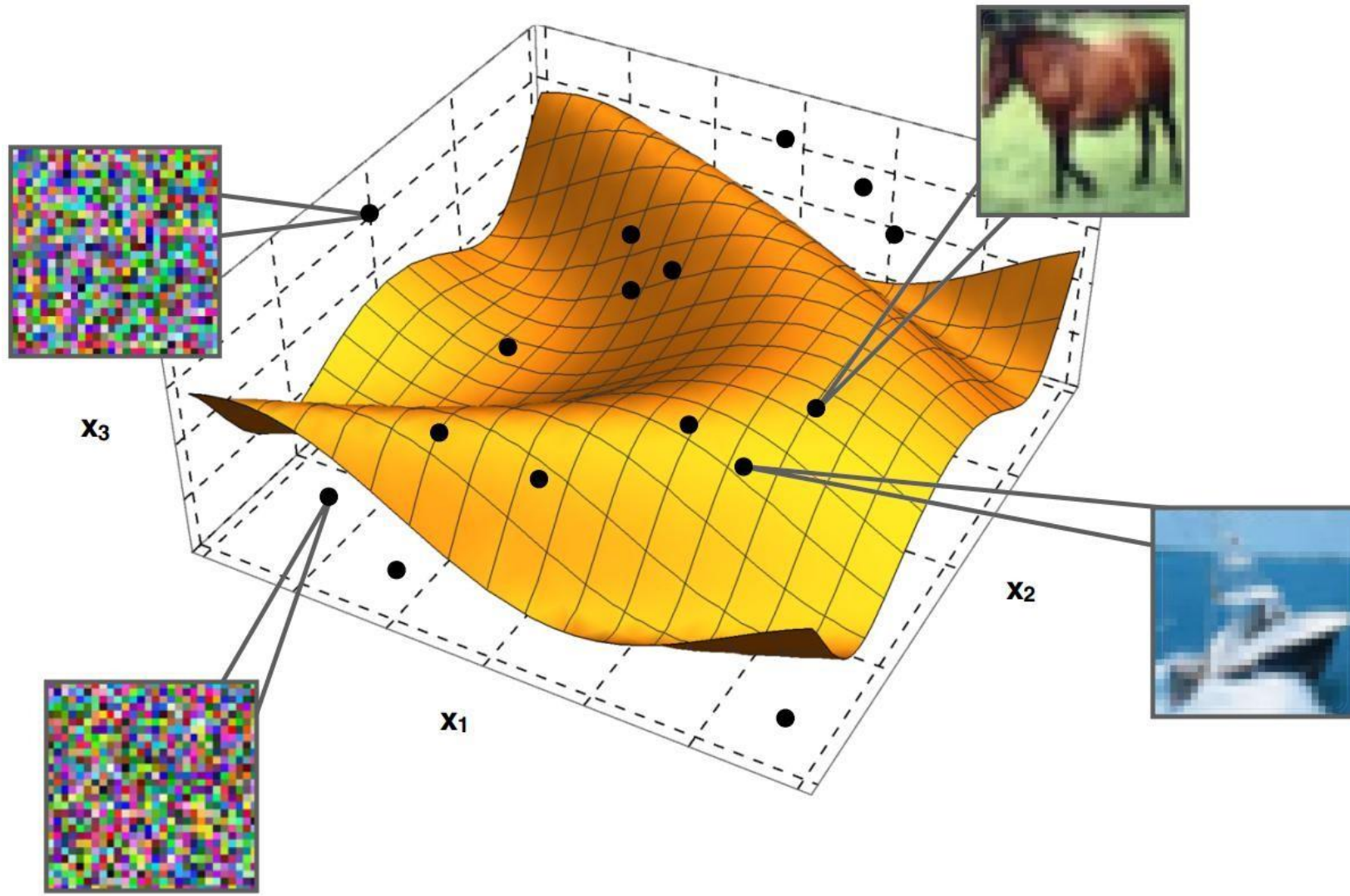
Image Classification



Curse of Dimensionality:

- ▶ There exist $2^{784} = 10^{236}$ possible binary images of resolution 28 x 28 pixels
- ▶ MNIST is gray-scale, thus 256^{784} combinations $\Rightarrow$ impossible to enumerate
- ▶ Why is image classification with just 60k labeled training images even possible?
- ▶ Answer: Images concentrated on low-dimensional manifold in high-dim. space

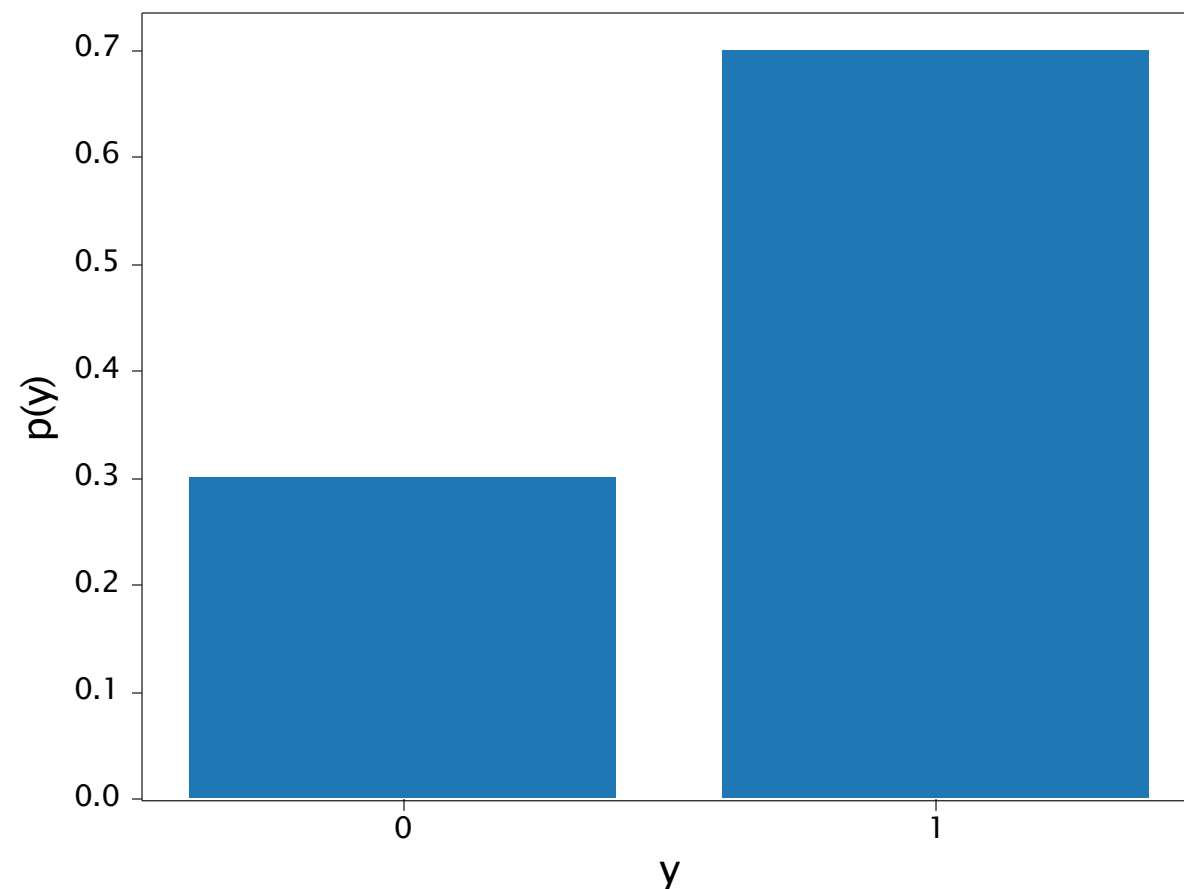
Image Classification



Bernoulli distribution:

$$p(y) = \mu^y (1 - \mu)^{(1-y)}$$

- ▶ μ : probability for $y = 1$
- ▶ Handles only two classes
e.g. (“cats” vs. “dogs”)



3 BCE Loss

Let $p_{model}(y|\mathbf{x}, \mathbf{w}) = f_{\mathbf{w}}(\mathbf{x})^y (1 - f_{\mathbf{w}}(\mathbf{x}))^{(1-y)}$ be a **Bernoulli distribution**. We obtain:

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log \left[f_{\mathbf{w}}(\mathbf{x}_i)^{y_i} (1 - f_{\mathbf{w}}(\mathbf{x}_i))^{(1-y_i)} \right] \\ &= \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^N \underbrace{-y_i \log f_{\mathbf{w}}(\mathbf{x}_i) - (1 - y_i) \log(1 - f_{\mathbf{w}}(\mathbf{x}_i))}_{\text{BCE Loss}}\end{aligned}$$

In other words, we minimize the **binary cross-entropy (BCE)** loss.

Remark: Last layer of $f_{\mathbf{w}}(\mathbf{x})$ can be a sigmoid function such that $f_{\mathbf{w}}(\mathbf{x})^y \in [0, 1]$.

How can we scale this to multiple classes?

Categorical Distribution

Categorical distribution:

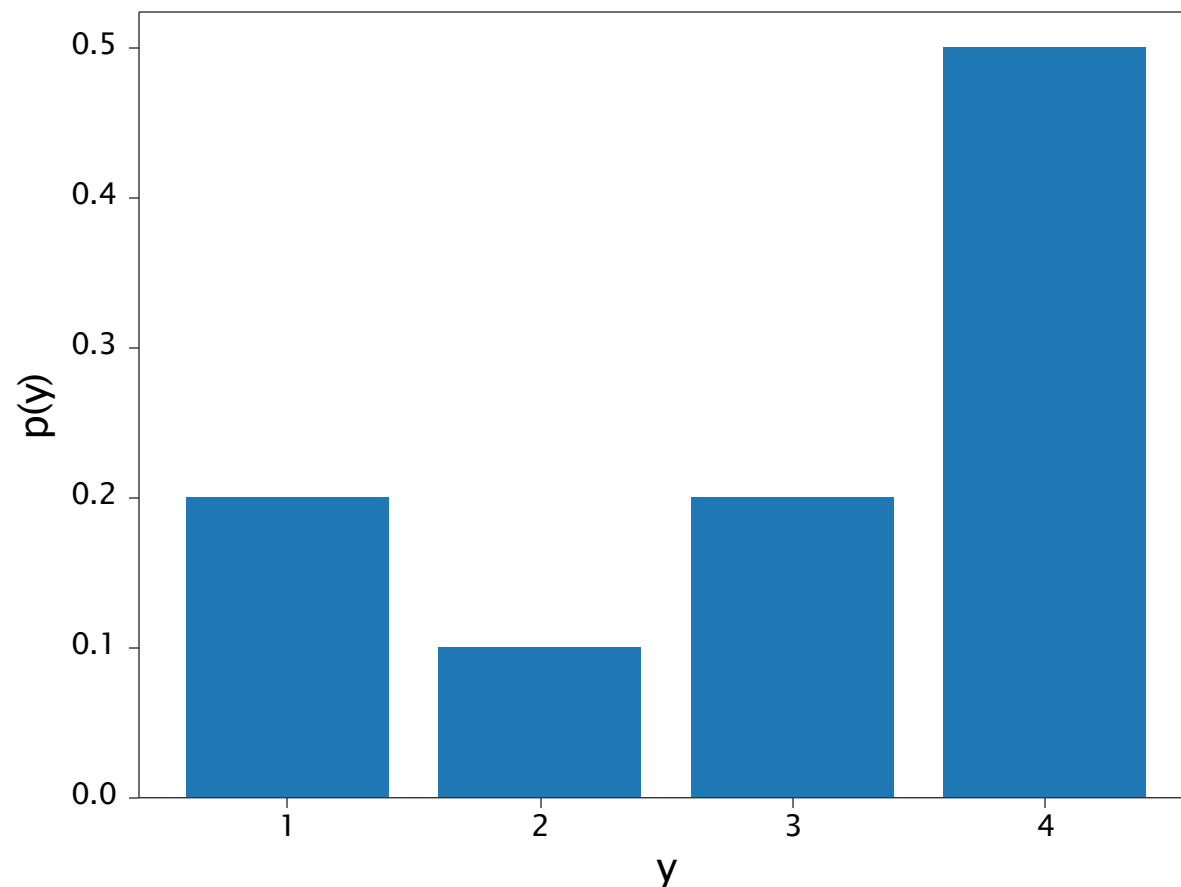
$$p(y = c) = \mu_c$$

- ▶ μ_c : probability for class c
- ▶ Multiple classes, multiple modes

Alternative notation:

$$p(\mathbf{y}) = \prod_{c=1}^C \mu_c^{y_c}$$

- ▶ $\mathbf{y}$: “one-hot” vector with $y_c \in \{0, 1\}$
- ▶ $\mathbf{y} = (0, \dots, 0, 1, 0, \dots, 0)^\top$ with all zeros except for one (the true class)



6 One-Hot Representation

class	y	$\mathbf{y}$
	1	$(1, 0, 0, 0)^T$
	2	$(0, 1, 0, 0)^T$
	3	$(0, 0, 1, 0)^T$
	4	$(0, 0, 0, 1)^T$

- ▶ One-hot vector $\mathbf{y}$ with binary elements $y_c \in \{0, 1\}$
- ▶ Index c with $y_c = 1$ determines the correct class, and $y_k = 0$ for $k \neq c$
- ▶ Interpretation as discrete distribution with all probability mass at the true class
- ▶ Often used in ML as it can make formalism more convenient

7 Cross Entropy Loss

Let $p_{model}(\mathbf{y}|\mathbf{x}, \mathbf{w}) = \prod_{c=1}^C f_{\mathbf{w}}^{(c)}(\mathbf{x})^{y_c}$ be a **Categorical distribution**. We obtain:

$$\begin{aligned}\hat{\mathbf{w}}_{ML} &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log p_{model}(\mathbf{y}_i | \mathbf{x}_i, \mathbf{w}) \\ &= \operatorname{argmax}_{\mathbf{w}} \sum_{i=1}^N \log \prod_{c=1}^C f_{\mathbf{w}}^{(c)}(\mathbf{x}_i)^{y_{i,c}} \\ &= \operatorname{argmin}_{\mathbf{w}} \sum_{i=1}^N \underbrace{\sum_{c=1}^C -y_{i,c} \log f_{\mathbf{w}}^{(c)}(\mathbf{x}_i)}_{\text{CE Loss}}\end{aligned}$$

In other words, we minimize the **cross-entropy (CE)** loss.

The target $\mathbf{y} = (0, \dots, 0, 1, 0, \dots, 0)^\top$ is a “one-hot” vector with y_c its c ’th element.

Softmax

How can we ensure that $f_{\mathbf{w}}^{(c)}(\mathbf{x})$ predicts a **valid Categorical (discrete) distribution**?

- ▶ We must guarantee (1) $f_{\mathbf{w}}^{(c)}(\mathbf{x}) \in [0, 1]$ and (2) $\sum_{c=1}^C f_{\mathbf{w}}^{(c)}(\mathbf{x}) = 1$
- ▶ An element-wise sigmoid as output function would ensure (1) but not (2)
- ▶ Solution: The **softmax function** guarantees both (1) and (2):

$$\text{softmax}(\mathbf{x}) = \left(\frac{\exp(x_1)}{\sum_{k=1}^C \exp(x_k)}, \dots, \frac{\exp(x_C)}{\sum_{k=1}^C \exp(x_k)} \right)$$

- ▶ Let $\mathbf{s}$ denote the network output after the last affine layer (=scores). Then:

$$f_{\mathbf{w}}^{(c)}(\mathbf{x}) = \frac{\exp(s_c)}{\sum_{k=1}^C \exp(s_k)} \quad \Rightarrow \quad \log f_{\mathbf{w}}^{(c)}(\mathbf{x}) = s_c - \log \sum_{k=1}^C \exp(s_k)$$

- ▶ Remark: s_c is a direct contribution to the loss function, i.e., it does not saturate

4 Cross Entropy Loss with Softmax Example

Putting it together: Cross Entropy Loss for a single training sample $(\mathbf{x}, \mathbf{y}) \in X$:

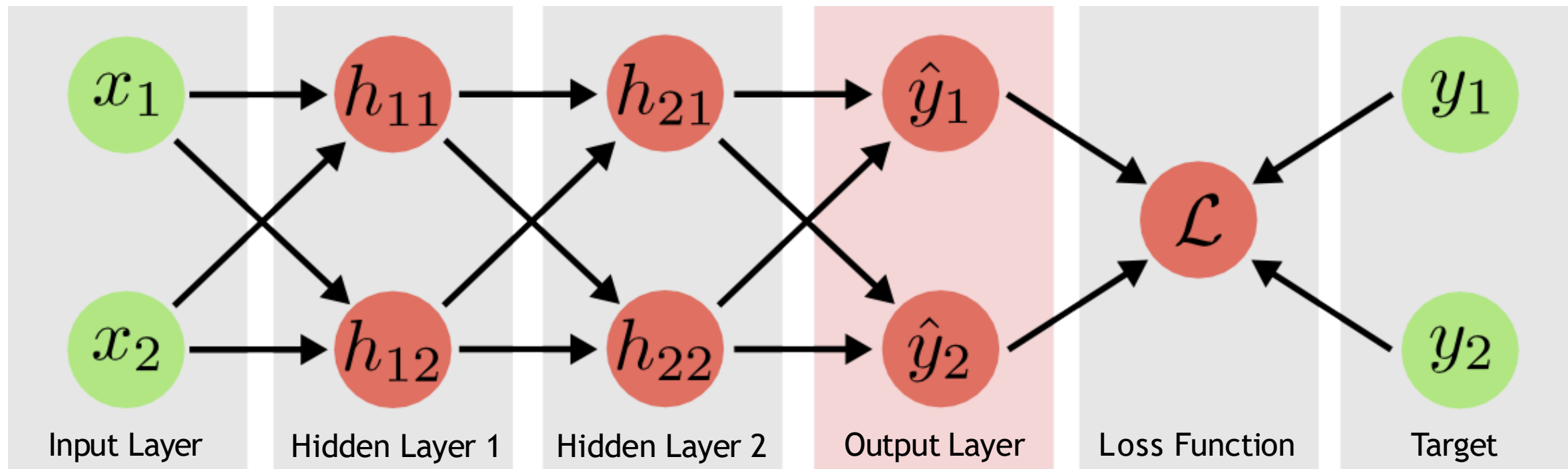
$$\text{CE Loss: } \sum_{c=1}^C -y_c \log f_{\mathbf{w}}^{(c)}(\mathbf{x})$$

Example: Suppose $C = 4$ and 4 samples $\mathbf{x}$ with labels $\mathbf{y}$ (log = natural logarithm)

Input $\mathbf{x}$	Label $\mathbf{y}$	Predicted scores $\mathbf{s}$	softmax($\mathbf{s}$) = $\mathbf{f}_{\mathbf{w}}(\mathbf{x})$	CE Loss
	$(\mathbf{1}, 0, 0, 0)^T$	$(+3, +1, -1, -1)^T$	$(\mathbf{0.85}, 0.12, 0.02, 0.02)^T$	0.16
	$(0, \mathbf{1}, 0, 0)^T$	$(+3, +3, +1, +0)^T$	$(0.46, \mathbf{0.46}, 0.06, 0.02)^T$	0.78
	$(0, 0, \mathbf{1}, 0)^T$	$(+1, +1, +1, +1)^T$	$(0.25, 0.25, \mathbf{0.25}, 0.25)^T$	1.38
	$(0, 0, 0, \mathbf{1})^T$	$(+3, +2, +3, -1)^T$	$(0.42, 0.16, 0.42, \mathbf{0.01})^T$	4.87

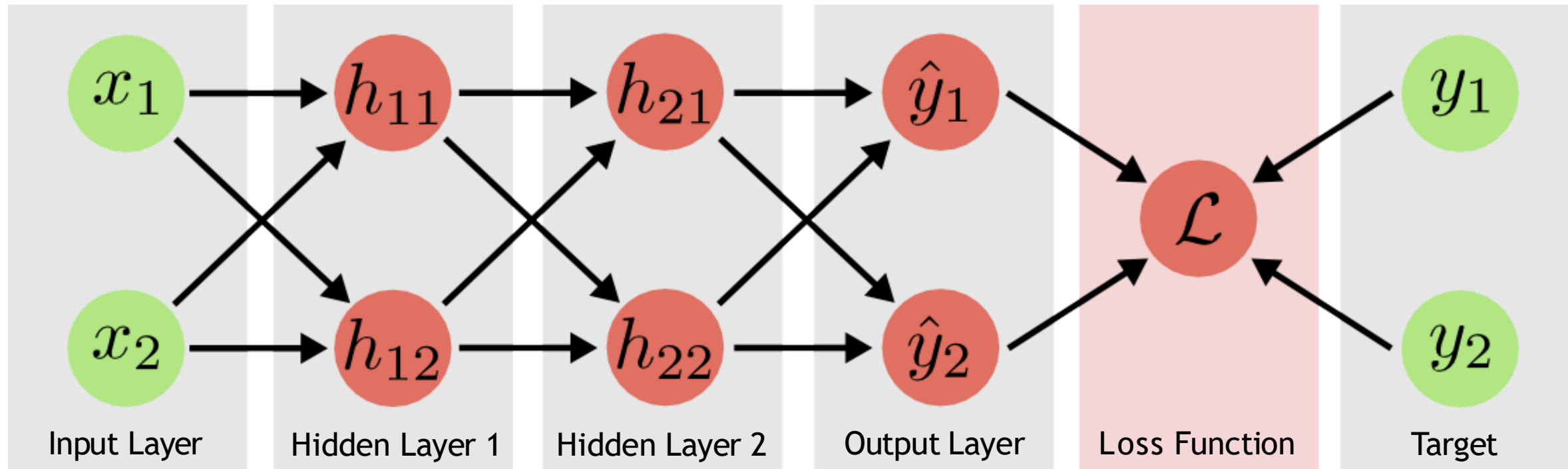
► Sample 4 contributes most strongly to the loss function! (elephant in the room)

5 Output Layer for Classification Problems



- For 2 classes, we can predict 1 value and use a sigmoid, or 2 values with softmax
- For $C > 2$ classes we typically predict C scores and use a softmax non-linearity

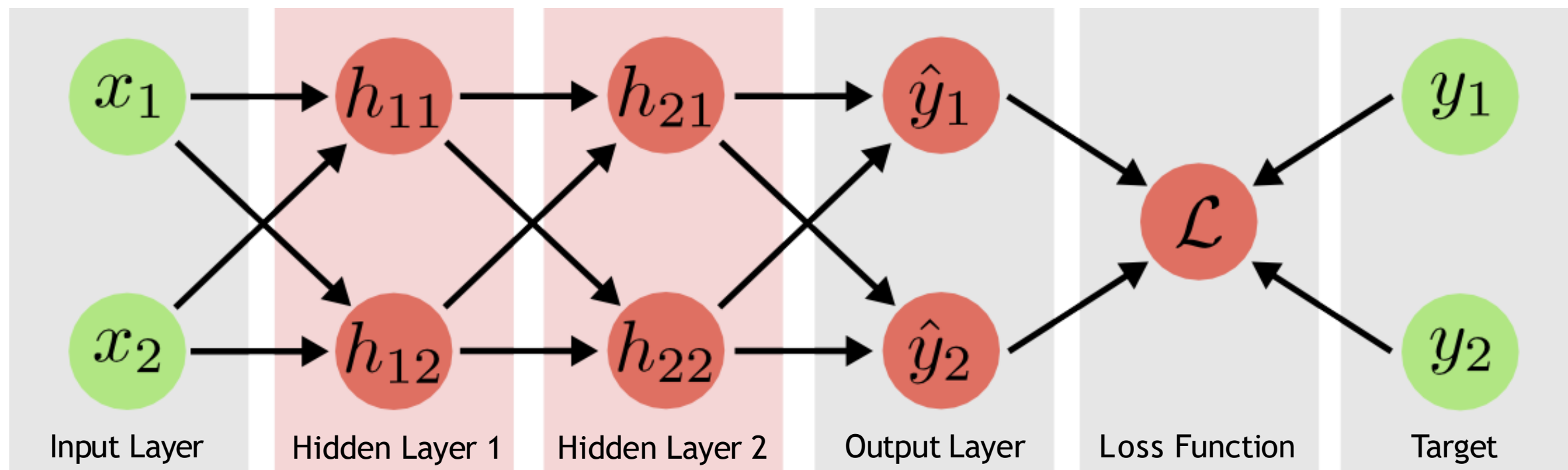
6 Loss Function for Classification Problems



- For 2 classes, we use the binary cross-entropy loss (BCE)
- For $C > 2$ classes, we use the cross-entropy loss (CE)

Activation Functions

8 Activation Functions



- ▶ Hidden layer $\mathbf{h}_i = g(\mathbf{A}_i \mathbf{h}_{i-1} + \mathbf{b}_i)$ with **activation function** $g(\cdot)$ and weights $\mathbf{A}_i$, $\mathbf{b}_i$
- ▶ The activation function is frequently applied **element-wise** to its input
- ▶ Activation functions must be **non-linear** to learn non-linear mappings
- ▶ Some of them are not differentiable everywhere (but still ok for training)

9 Activation Functions

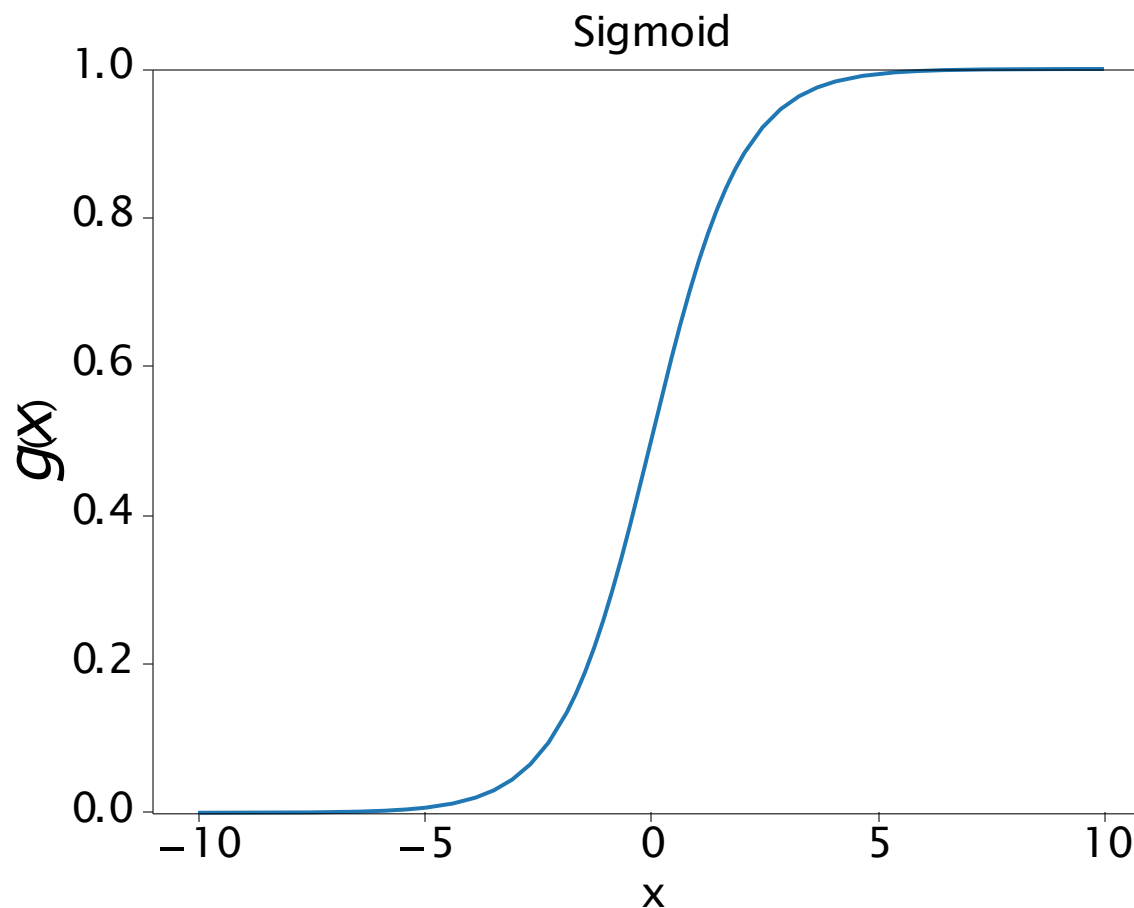
Sigmoid:

$$g(x) = \frac{1}{1 + \exp(-x)}$$

- ▶ Maps input to range $[0, 1]$
- ▶ Neuroscience interpretation as saturating “firing rate” of neurons

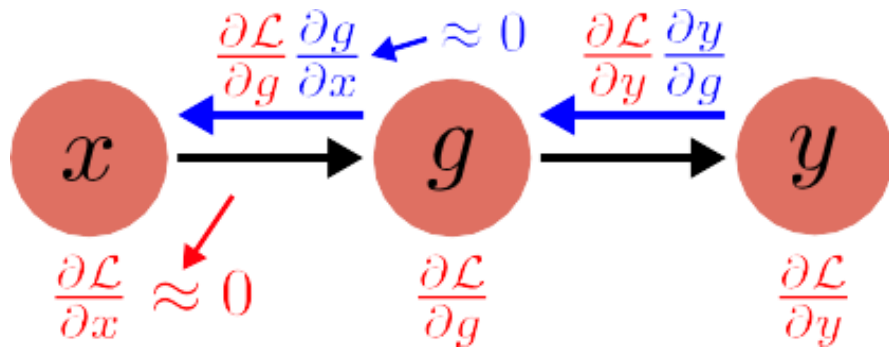
Problems:

- ▶ Saturation “kills” gradients
- ▶ Outputs are not zero-centered
- ▶ Introduces bias after first layer

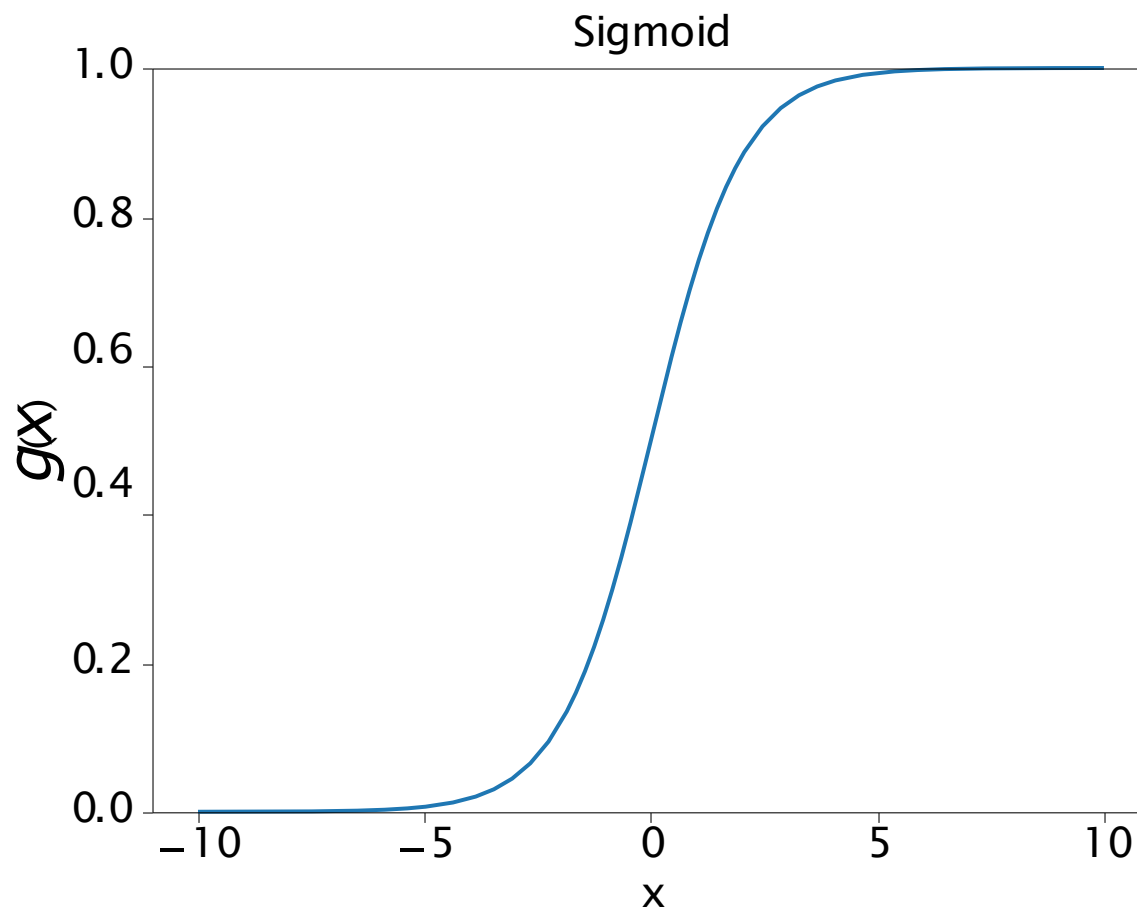


0 Activation Functions

Sigmoid Problem #1:



- Downstream gradient becomes zero when input x is saturated: $g^r(x) \approx 0$
- No learning if x is very small (< -10)
- No learning if x is very large (> 10)



1 Activation Functions

Sigmoid Problem #2:

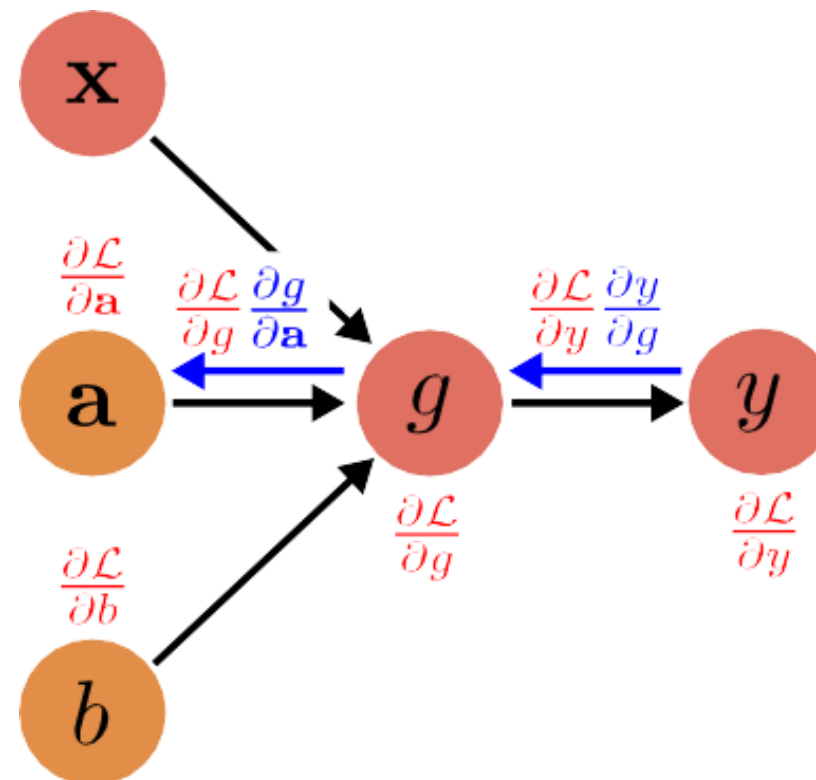
$$g(x) = \frac{1}{1 + \exp(-x)} \quad x = \sum_i a_i x_i + b$$

- Sigmoid is always positive $\Rightarrow x_i$ also
- Gradient of sigmoid is always positive

The gradient wrt. parameter a_i is given by:

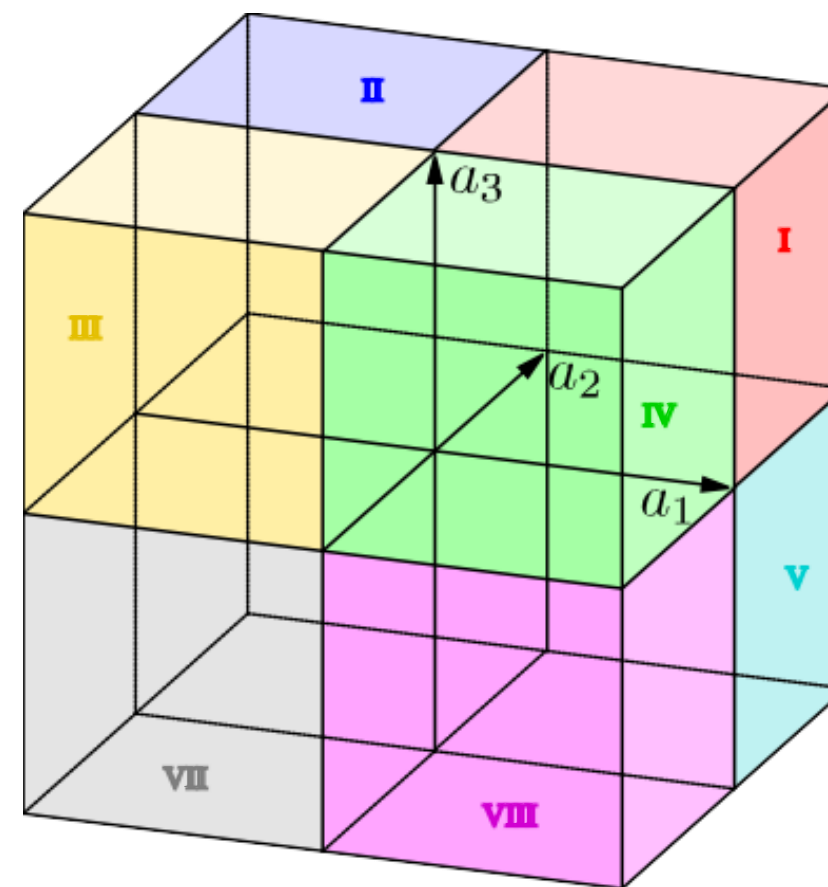
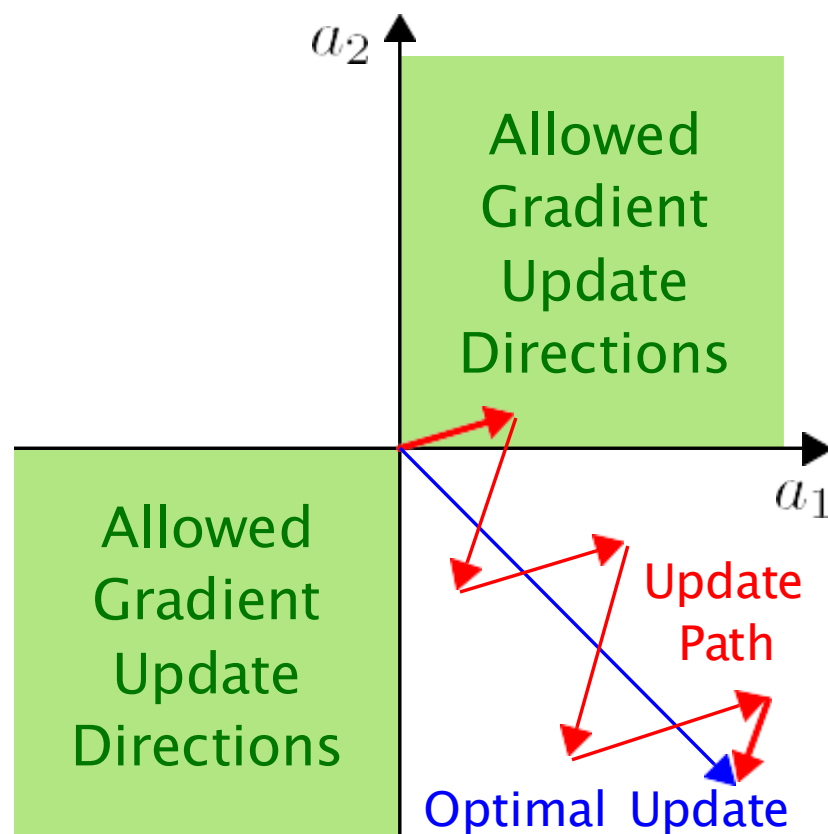
$$\frac{\partial \mathcal{L}}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial x} \frac{\partial x}{\partial a_i} = \frac{\partial \mathcal{L}}{\partial g} \frac{\partial g}{\partial x} x_i$$

- Therefore: $\text{sgn}(\partial \mathcal{L} / \partial a_i) = \text{sgn}(\partial \mathcal{L} / \partial g)$
- All gradients have the same sign (+ or -)



2 Activation Functions

Sigmoid Problem #2:



- Restricts gradient updates and leads to inefficient optimization (minibatches help)

3 Activation Functions

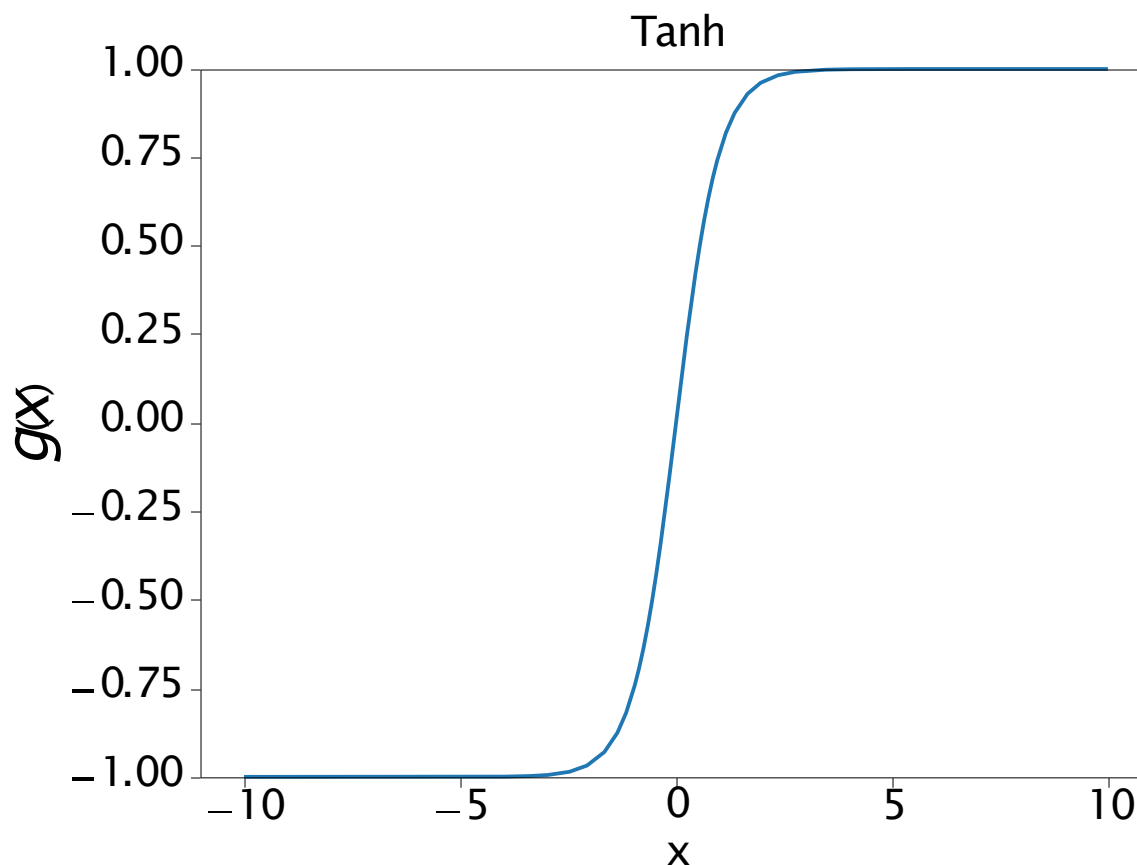
Tanh:

$$g(x) = \frac{2}{1 + \exp(-2x)} - 1$$

- ▶ Maps input to range $[-1, 1]$
- ▶ Anti-symmetric
- ▶ Zero-centered

Problems:

- ▶ Again, saturation “kills” gradients



4 Activation Functions

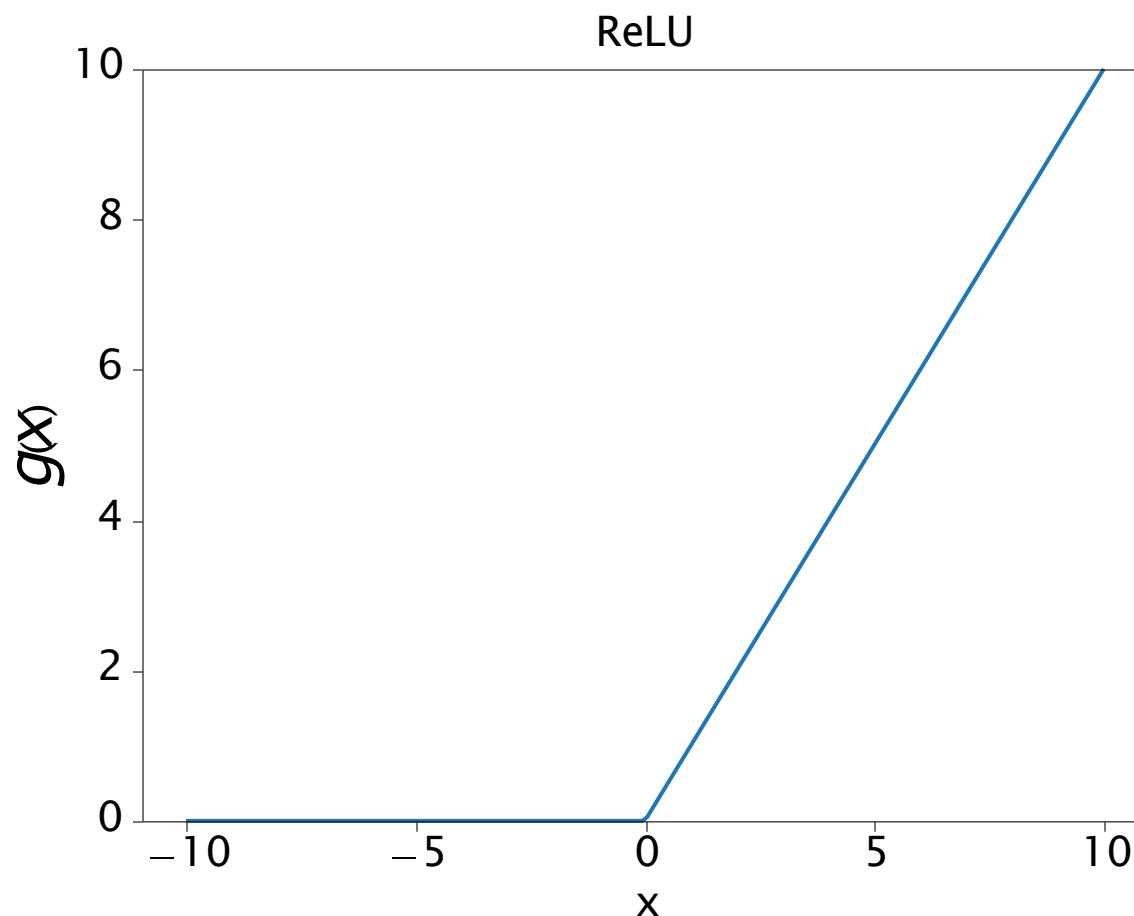
Rectified Linear Unit (ReLU):

$$g(x) = \max(0, x)$$

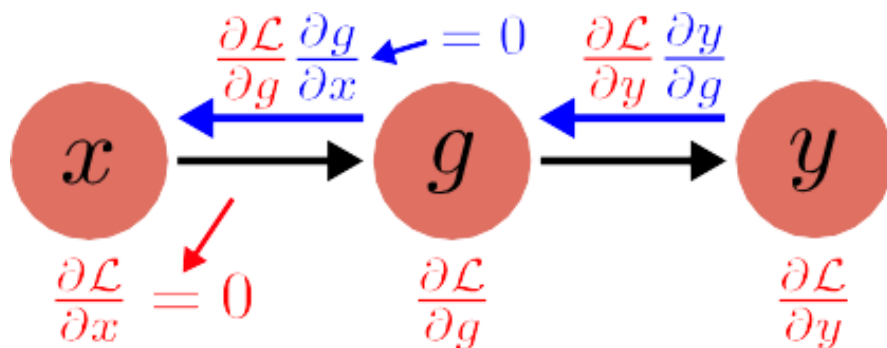
- ▶ Does not saturate (for $x > 0$)
- ▶ Leads to fast convergence
- ▶ Computationally efficient

Problems:

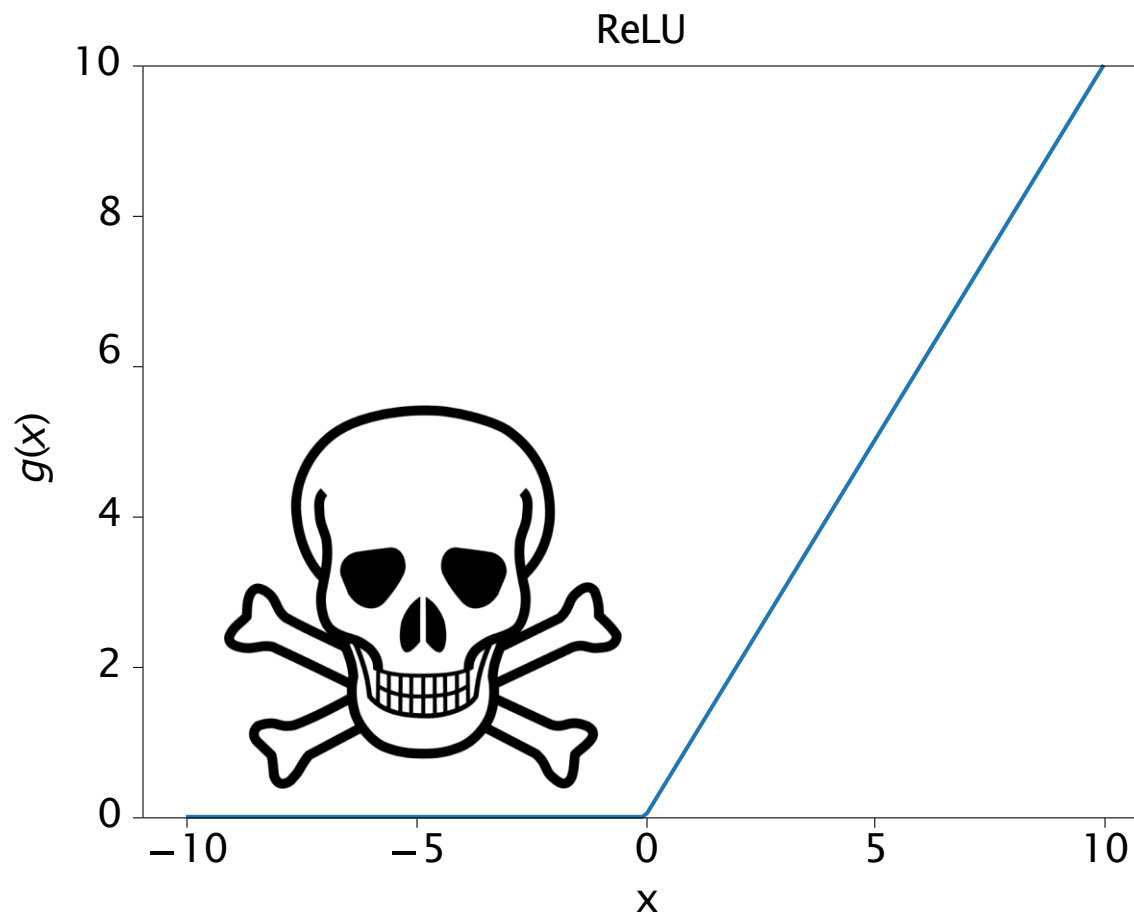
- ▶ Not zero-centered
- ▶ No learning for $x < 0 \Rightarrow$ dead ReLUs



ReLU Problem:



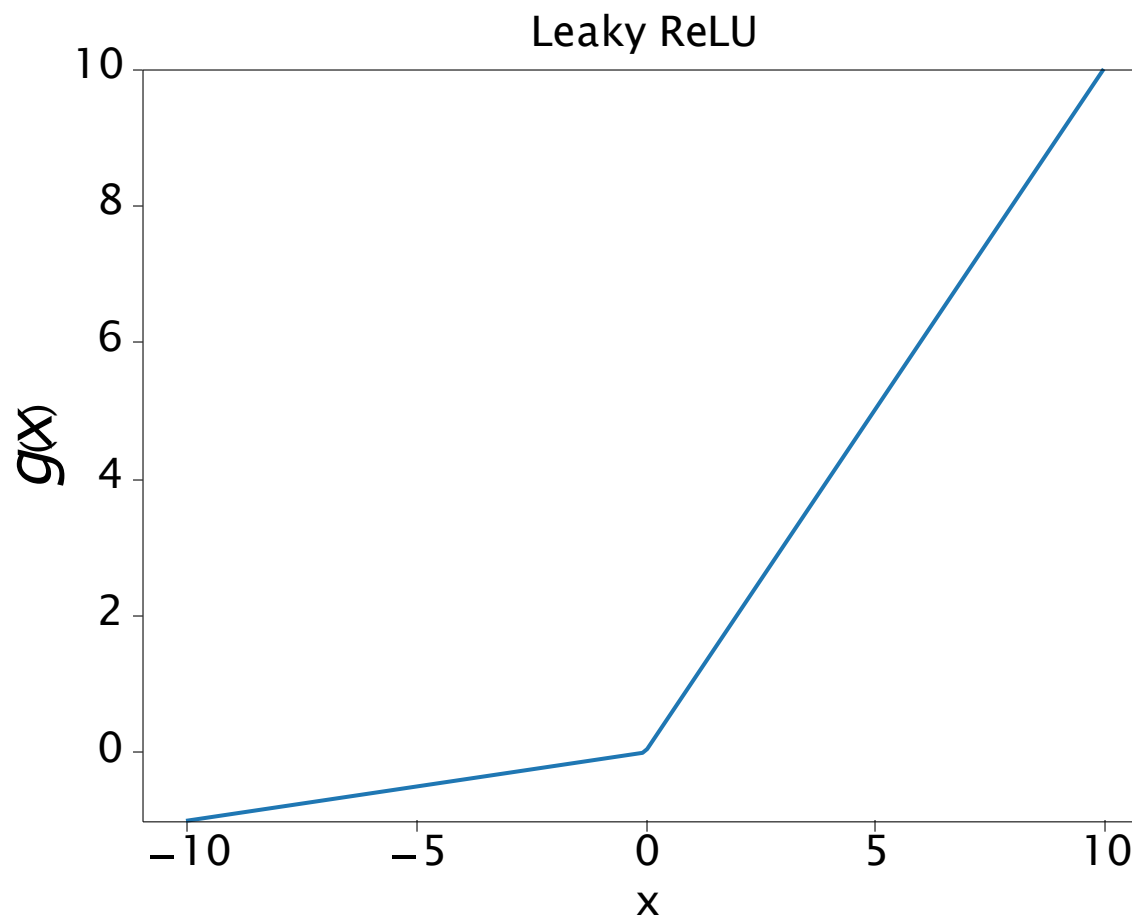
- ▶ Downstream gradient becomes zero when input $x < 0$
- ▶ Results in so-called “dead ReLUs” that never participate in learning
- ▶ Often initialize with pos. bias ($b > 0$)



Leaky ReLU:

$$g(x) = \max(0.01x, x)$$

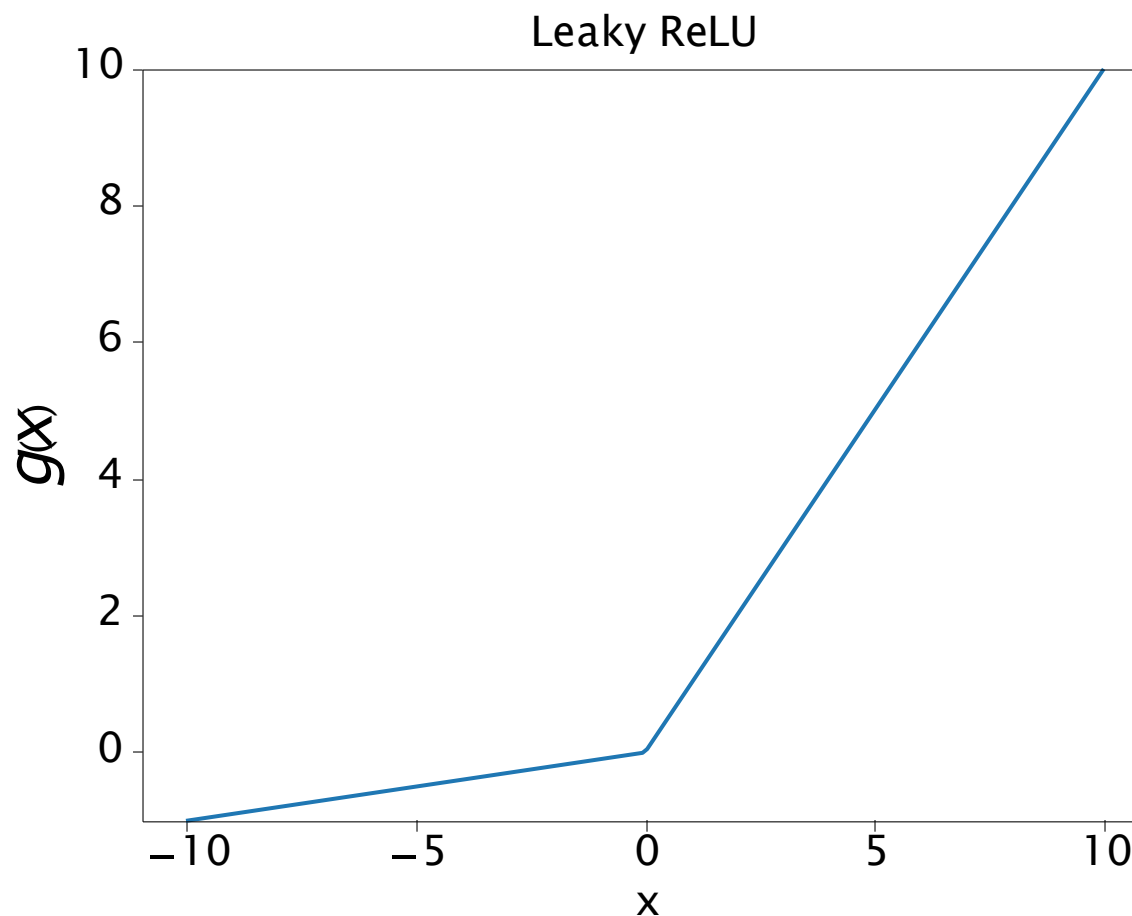
- ▶ Does not saturate (i.e., will not die)
- ▶ Closer to zero-centered outputs
- ▶ Leads to fast convergence
- ▶ Computationally efficient



Parametric ReLU:

$$g(x) = \max(\alpha x, x)$$

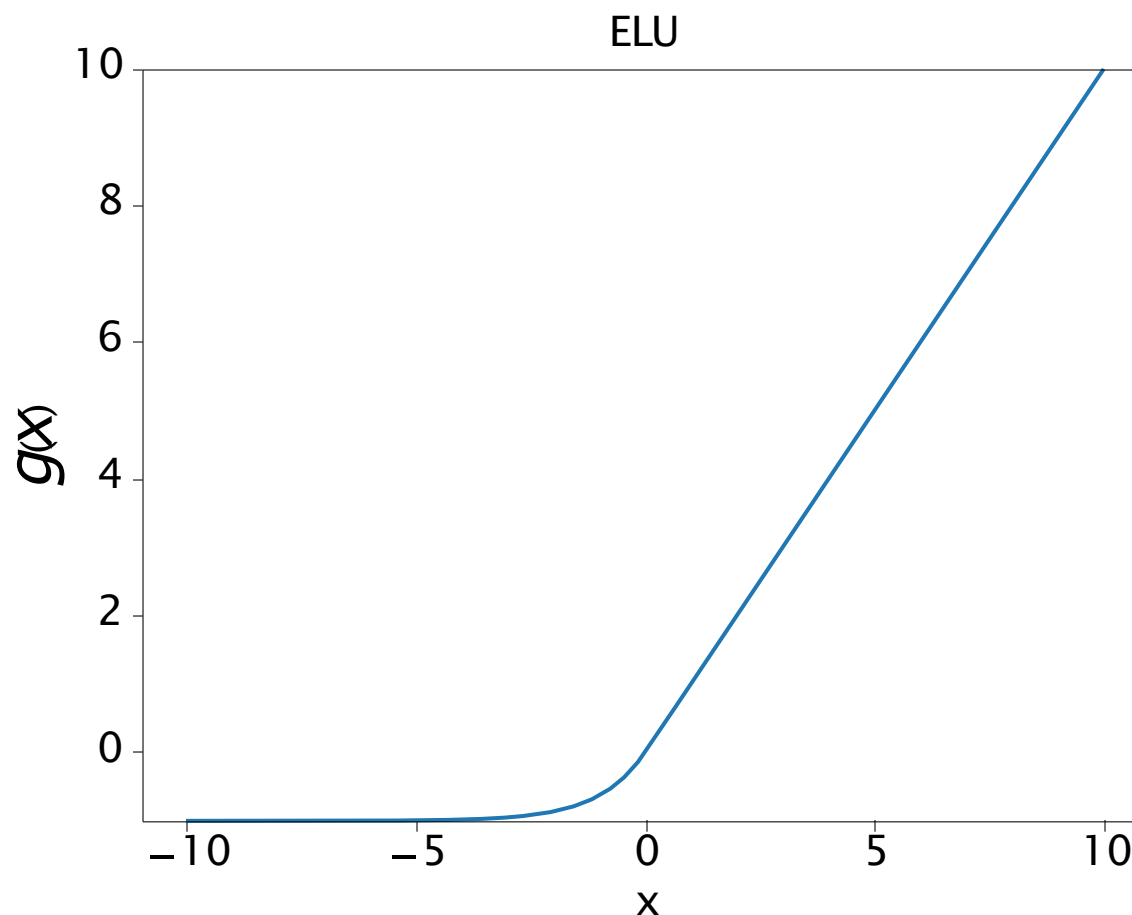
- ▶ Does not saturate (i.e., will not die)
- ▶ Leads to fast convergence
- ▶ Computationally efficient
- ▶ Parameter α learned from data



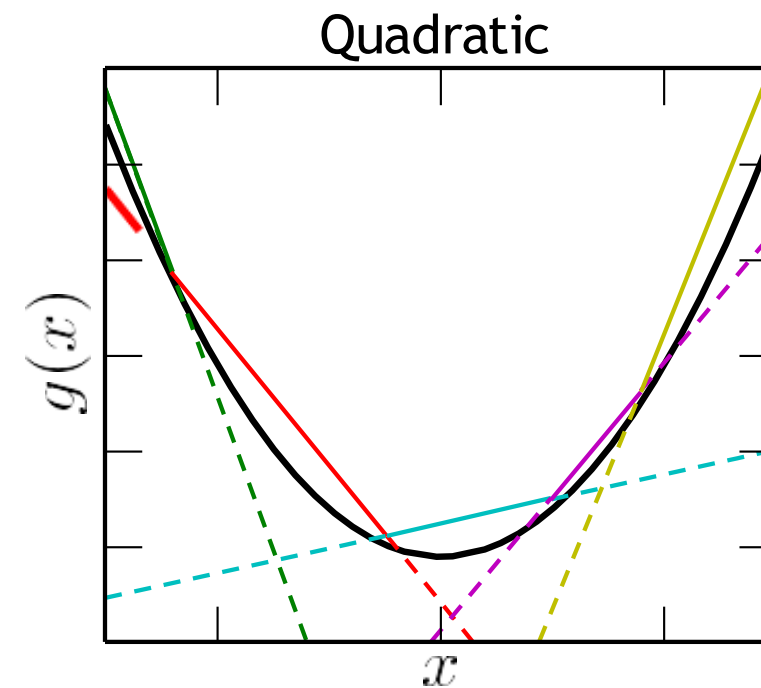
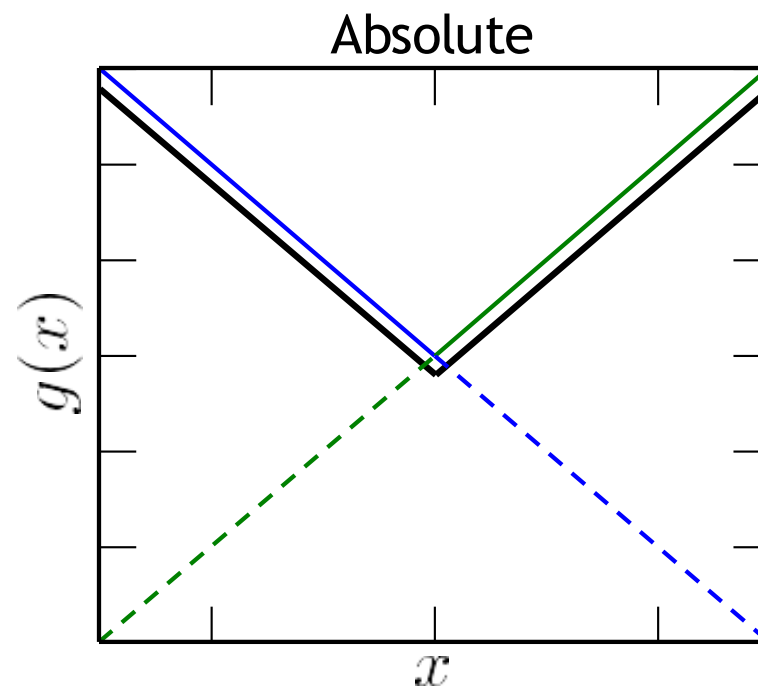
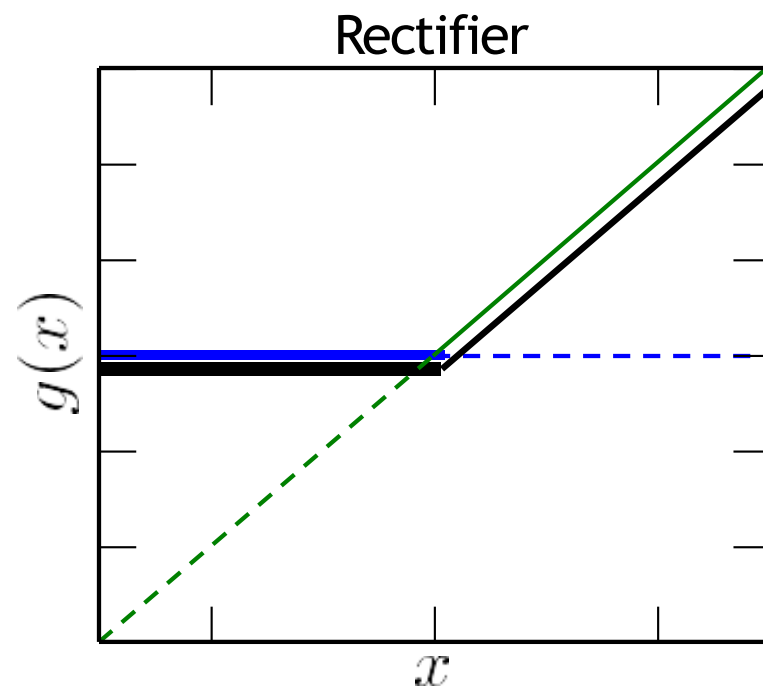
Exponential Linear Units (ELU):

$$g(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

- ▶ All benefits of Leaky ReLU
- ▶ Adds some robustness to noise
- ▶ Default $\alpha = 1$



9 Activation Functions



Maxout: $g(x) = \max(\mathbf{a}_1^T \mathbf{x} + b_1, \mathbf{a}_2^T \mathbf{x} + b_2)$

► Generalizes ReLU and Leaky ReLU

► Increases the number of parameters per neuron

0 Activation Functions

Summary:

- ▶ No one-size-fits-all: Choice of activation function depends on problem
- ▶ We only showed the most common ones, there exist many more
- ▶ Best activation function/model is often found using trial-and-error in practice
- ▶ It is important to ensure a good “gradient flow” during optimization

Rule of Thumb:

- ▶ Use ReLU by default (with small enough learning rate)
- ▶ Try Leaky ReLU, Maxout, ELU for some small additional gain
- ▶ Prefer Tanh over Sigmoid (Tanh often used in recurrent models)

Preprocessing and Initialization

Data Preprocessing

7 Data Preprocessing

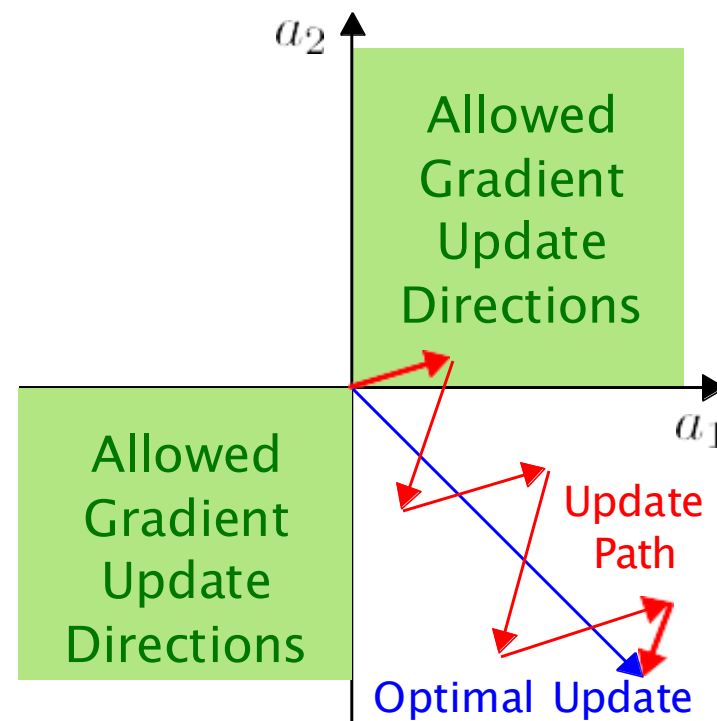
Remember what happens for positive inputs:

$$g(x) = g \left(\sum_i a_i x_i + b \right)$$

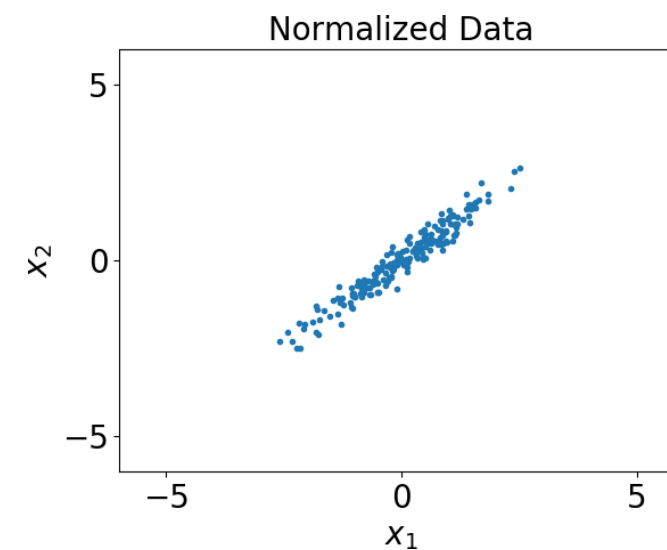
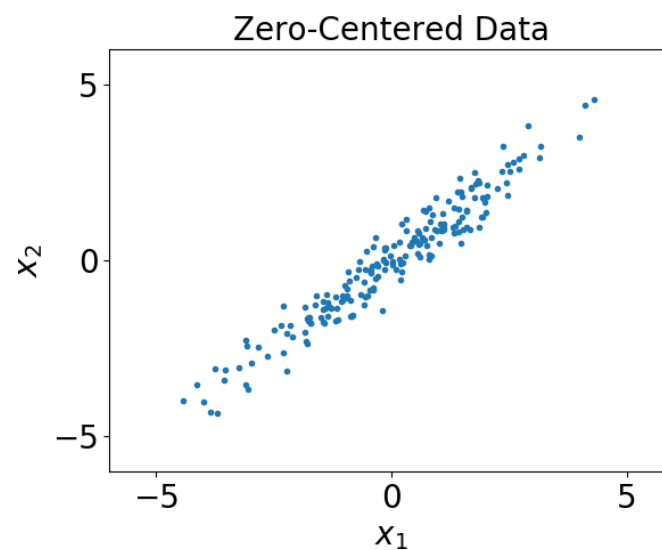
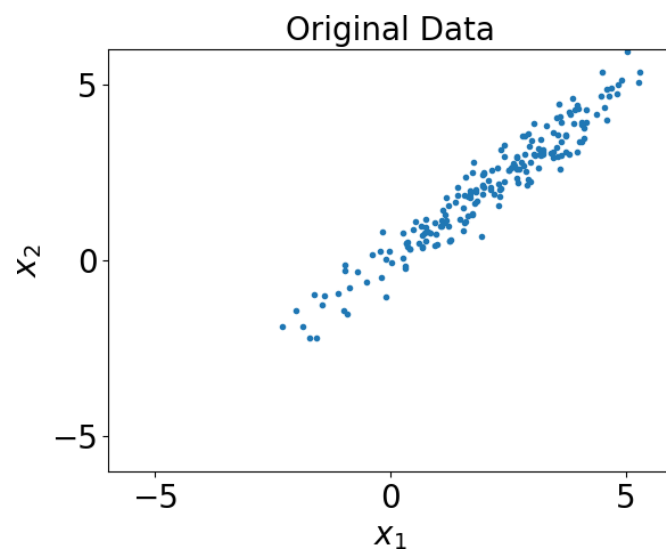
The gradient wrt. parameter a_i is given by:

$$\frac{\partial L}{\partial a_i} = \frac{\partial L}{\partial g} \frac{\partial g}{\partial x} x_i$$

- ▶ Both **terms in blue** are positive
- ▶ All gradients have the same sign (+ or -)
- ▶ We should pre-process the input data such that it is “well distributed”

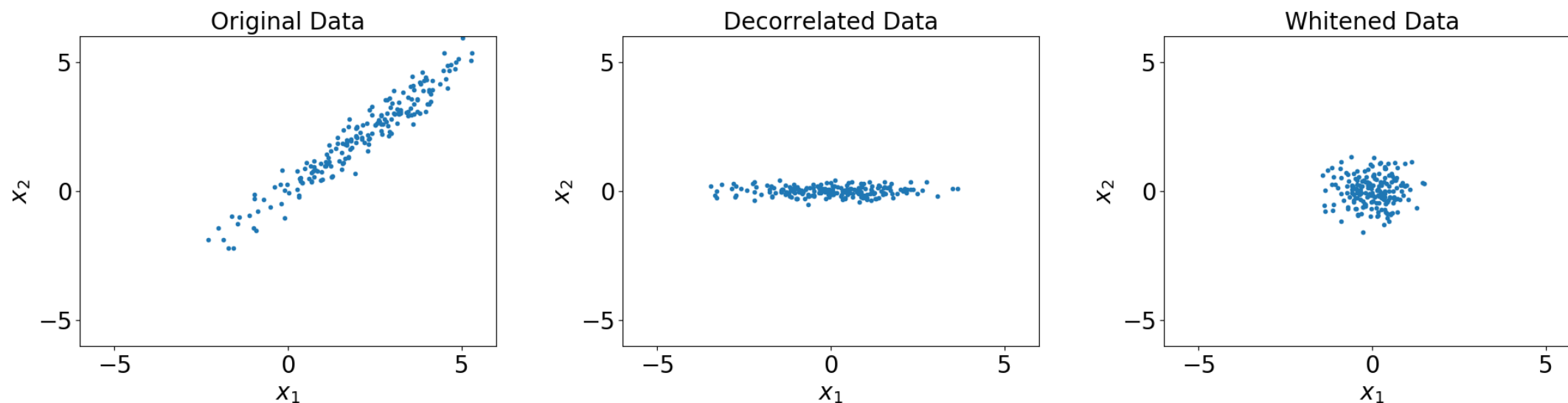


8 Data Preprocessing



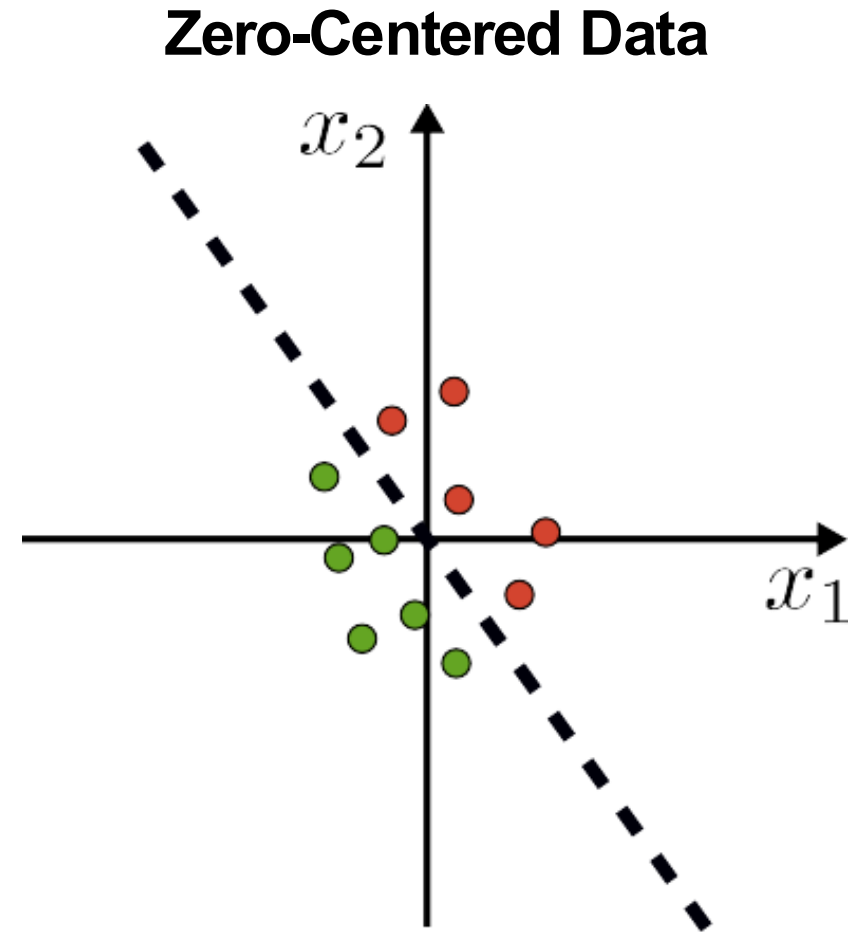
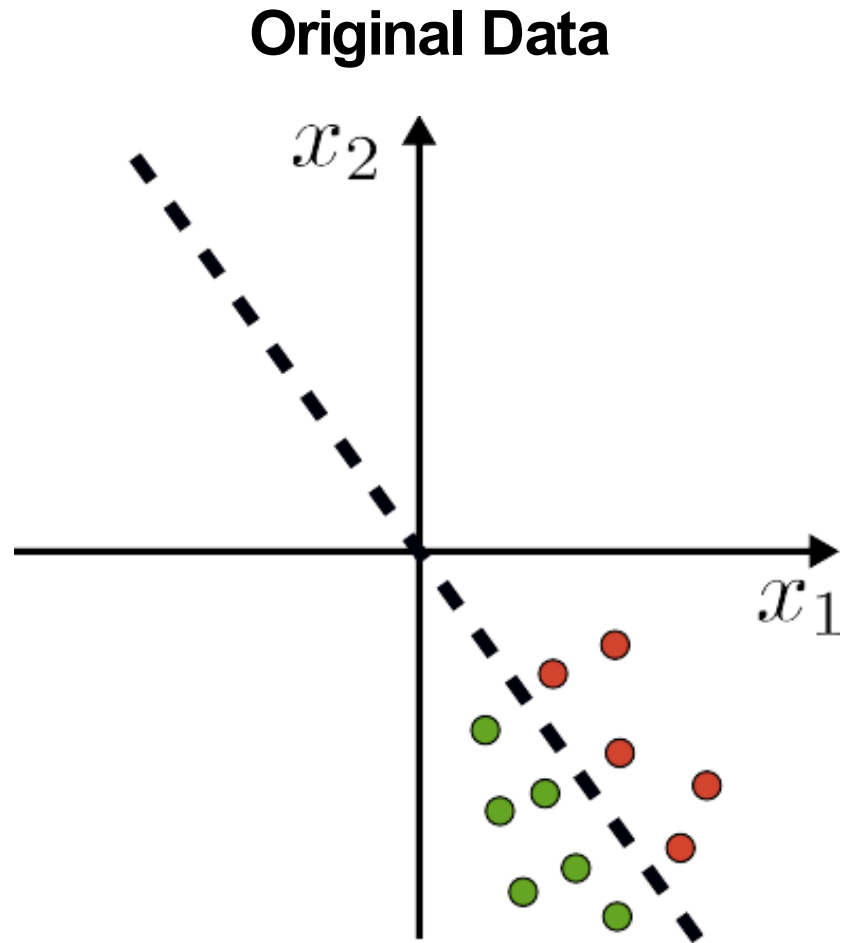
- **Zero-center:** $x_{i,j} \leftarrow x_{i,j} - \mu_j$ with $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$
- **Normalization:** $x_{i,j} \leftarrow x_{i,j} / \sigma_j$ with $\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$

9 Data Preprocessing



- **Decorrelate:** Multiply with eigenvectors of covariance matrix
- **Whiten:** Divide by square root of eigenvalues of covariance matrix

Data Preprocessing



- Classification loss becomes less sensitive to changes in the weight matrix

1 Data Preprocessing

Common Practices for Images:

- ▶ AlexNet: Subtract mean image
(mean image: $W \times H \times 3$ numbers)
- ▶ VGGNet: Subtract per-channel mean
(mean along each channel: 3 numbers)
- ▶ ResNet: Subtract per-channel mean and divide by per-channel std. dev.
(mean along each channel: 3 numbers)
- ▶ Whitening is less common

Weight Initialization

3 Recap: Stochastic Gradient Descent

Algorithm for training an MLP using (stochastic) gradient descent:

1. Initialize weights $\mathbf{w}$, pick learning rate η and minibatch size $|X_{\text{batch}}|$
2. Draw (random) minibatch $X_{\text{batch}} \subseteq X$
3. For all elements $(\mathbf{x}, \mathbf{y}) \in X_{\text{batch}}$ of minibatch (in parallel) do:
 1. Forward propagate $\mathbf{x}$ through network to calculate $\mathbf{h}_1, \mathbf{h}_2, \dots, \hat{\mathbf{y}}$
 2. Backpropagate gradients through network to obtain $\nabla_{\mathbf{w}} L(\hat{\mathbf{y}}, \mathbf{y})$
4. Update gradients: $\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \frac{1}{|X_{\text{batch}}|} \sum_{(\mathbf{x}, \mathbf{y}) \in X_{\text{batch}}} \nabla_{\mathbf{w}} L(\hat{\mathbf{y}}, \mathbf{y})$
5. If validation error decreases, go to step 2, otherwise stop

Question:

- How to best initialize the weights $\mathbf{w}$?

3 Vanishing Gradients

Back-propagation works by computing the gradients and going "down-hill"

Starting from the output layer towards the input layers, the gradients get smaller and smaller:

$$\frac{\partial E}{\partial w_{i,j}} := o_i \cdot \delta_j,$$

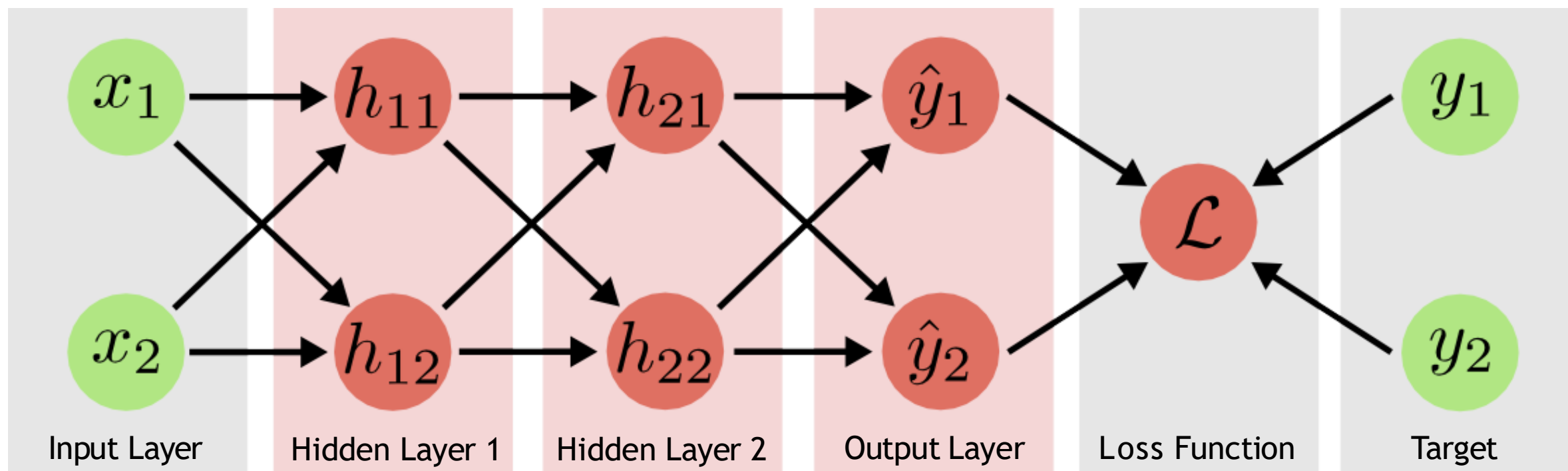
with δ being the error back-propagated to unit j . If $o_i < 1$, the error terms get smaller and smaller

This is called *vanishing gradient problem*

3 Vanishing Gradients

- Prevented the usage of deep networks
- In 2010, Xavier Glorot and Joshua Bengio investigated the problem in depth
 - The combination of sigmoidal activation and the random initialisation scheme with mean 0 and variance 1 (as usually used then) creates the problem
 - During forward pass, the variance of the inputs to a unit is smaller than the variance of the output
 - Activation function saturate in later layers
 - Derivative is close to 0, i.e., no errors get back propagated to previous layers
- Solutions: Change activation function, change initialization scheme, normalization

4 Constant Initialization



- How to initialize the parameters $\mathbf{w}$ of all network layers?
- Simple solution: set all network parameters to a constant (i.e., $\mathbf{w} = 0$)
- Learning not be possible (all units of each layer are learning the same)

Random Initialization

- Initialize all weights and biases by sampling from mean 0 and variance 1
- Most common scheme until at least 2010

3 Xavier Glorot Initialization Scheme

- Information should flow through the network properly in forward and backward pass
- Variance of the input should be similar to variance of the output
- Compute fan-in and fan-out for each layer, and average
- Initialize the weights, either
 - With normal distribution with mean=0 and variance = $1/\text{average weight}$
 - Uniform between $-r$ and r with $r = \sqrt{3/\text{average}}$
- Helps to balance variance between input and output of each layer

Summary

Data Preprocessing:

- ▶ Zero-centering the network inputs is important for efficient learning
- ▶ Decorrelation and whitening used less frequently

Weight Initialization:

- ▶ Proper initialization important for ensuring a good “gradient flow”
- ▶ For zero-centered activation functions, use Xavier initialization
- ▶ For ReLU activation functions, use He initialization
- ▶ Initialization is a research topic, much more literature on this topic
- ▶ However, not so relevant anymore these days: Today, we usually train models with normalization layers, like batch norm in convolutional networks and layer norm in transformers, which take care of exploding/vanishing problems

Advanced Neural Networks

Convolutional Neural Networks

Jun.-Prof. Dr.-Ing. Stefan Lüdtke

Institute for Visual & Analytic Computing

University of Rostock

Agenda

1. Convolution Layers

2. Downsampling

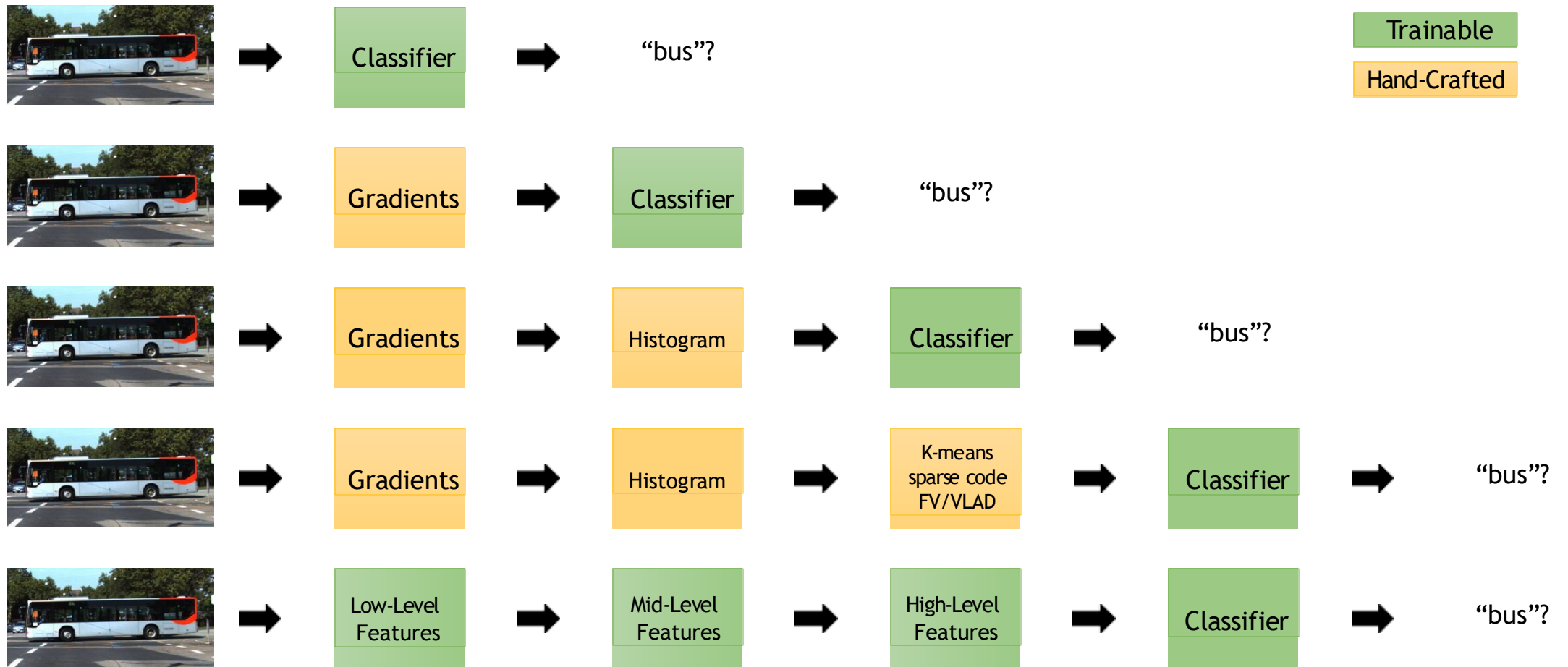
3. Upsampling

4. Architectures

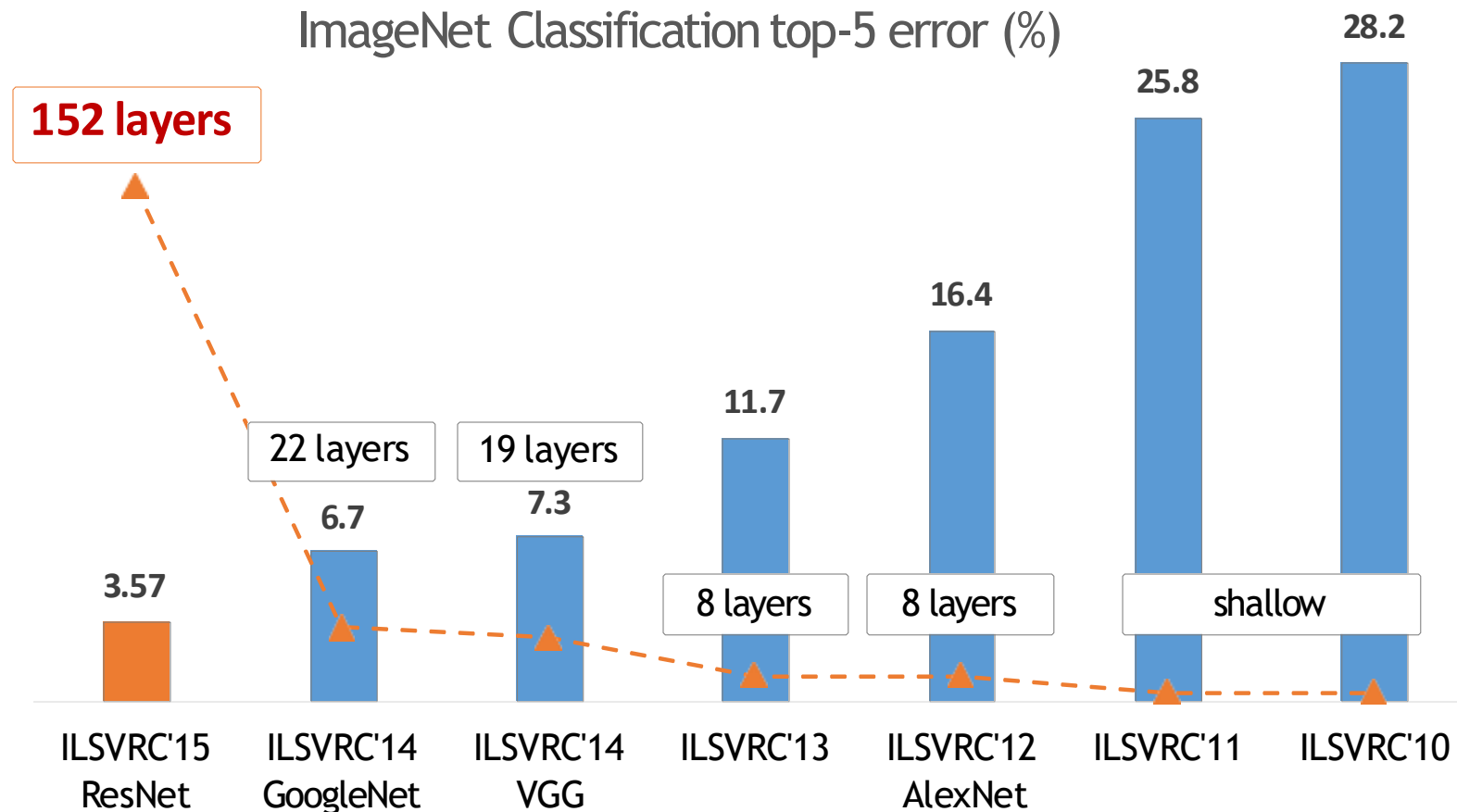
5. Transfer Learning

Convolution Layers

Representation Learning

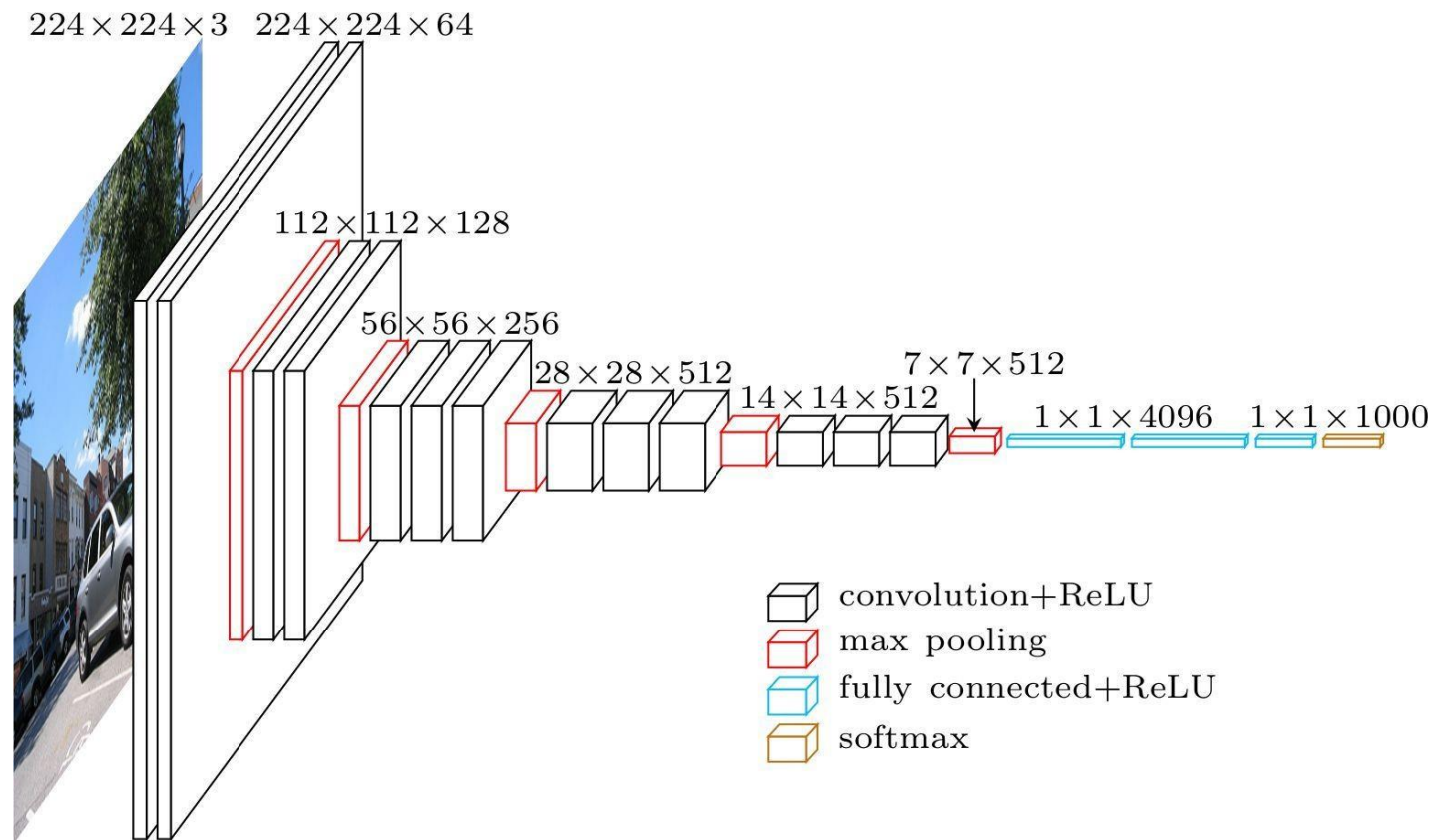


ImageNet Large Scale Visual Recognition Challenge



► Classification into 1000 object categories. Current state-of-the-art: 1.3 %

6 VGG Network



3 Types of Layers:

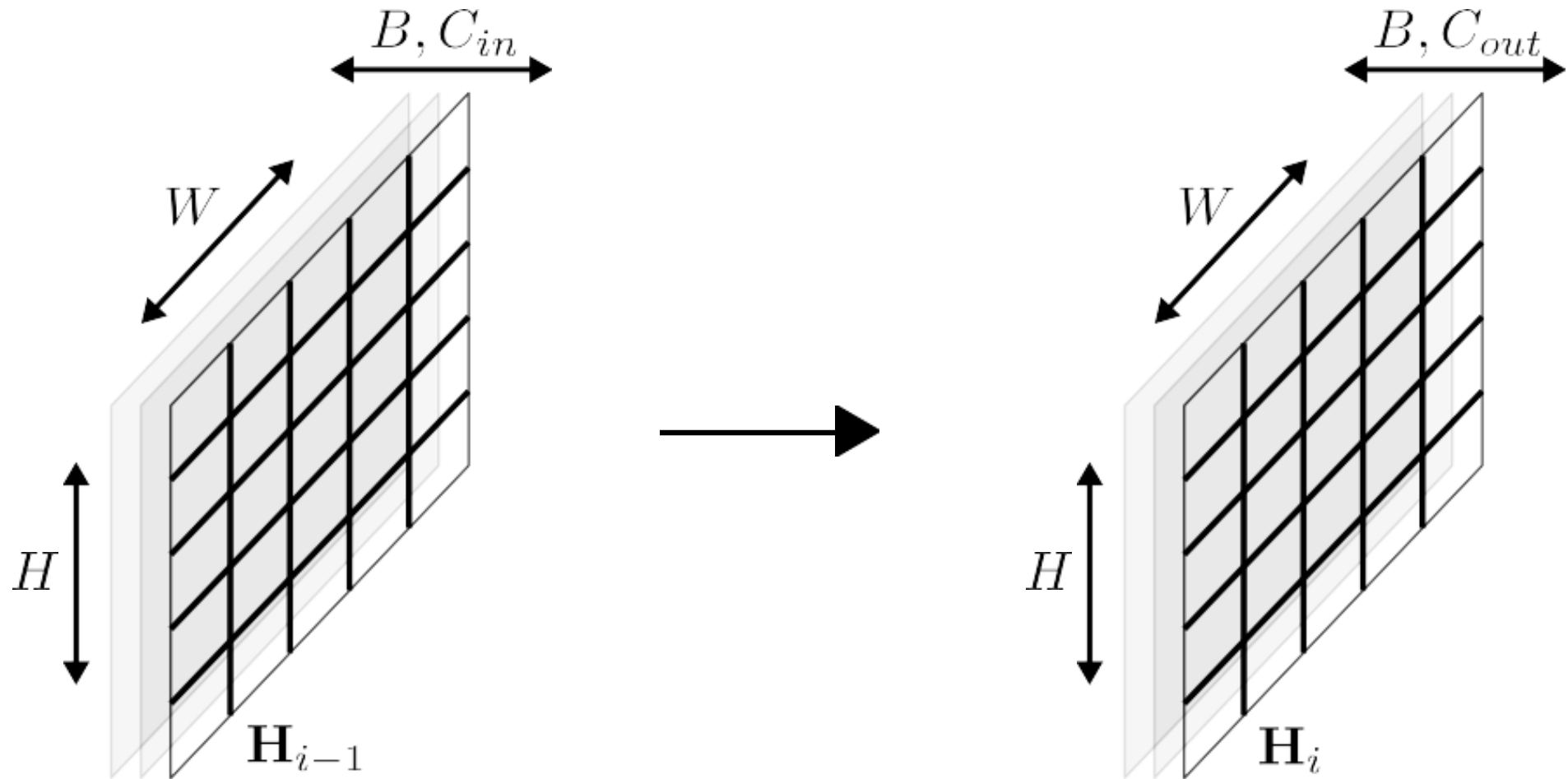
- Convolution layers, pooling layers and fully connected layers

Notation

We will use the following **notation**:

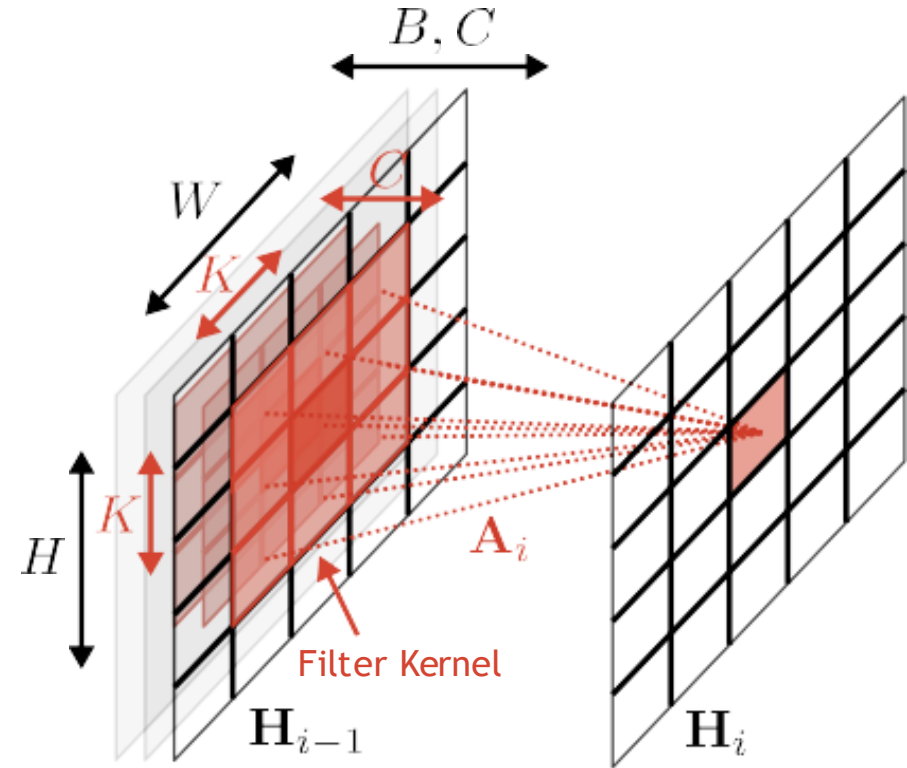
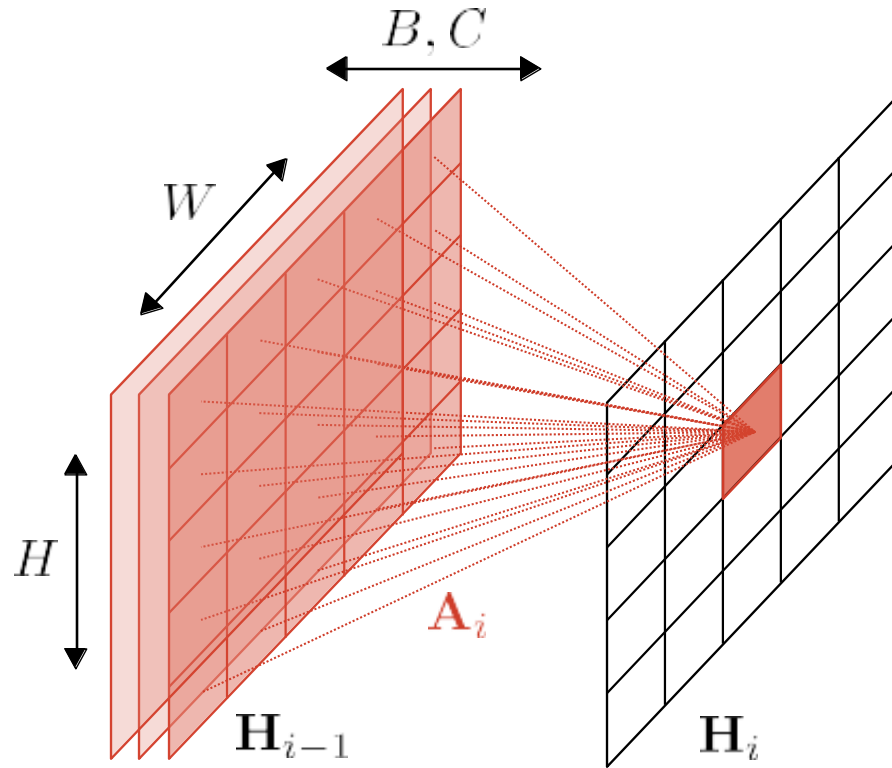
- ▶ B : **Batch size**
- ▶ W : **Width** of image or feature map
- ▶ H : **Height** of image or feature map
- ▶ C : Number of feature **channels** (=neurons per pixel)
 - ▶ C_{in} : Number of feature channels in current layer
 - ▶ C_{out} : Number of feature channels in next layer
 - ▶ $C_{in} = 1$ for first layer if input is a grayscale image
 - ▶ $C_{in} = 3$ for first layer if input is a color image
- ▶ $K \times K$: Convolutional filter **kernel size**

Network Layers with Spatial Dimensions



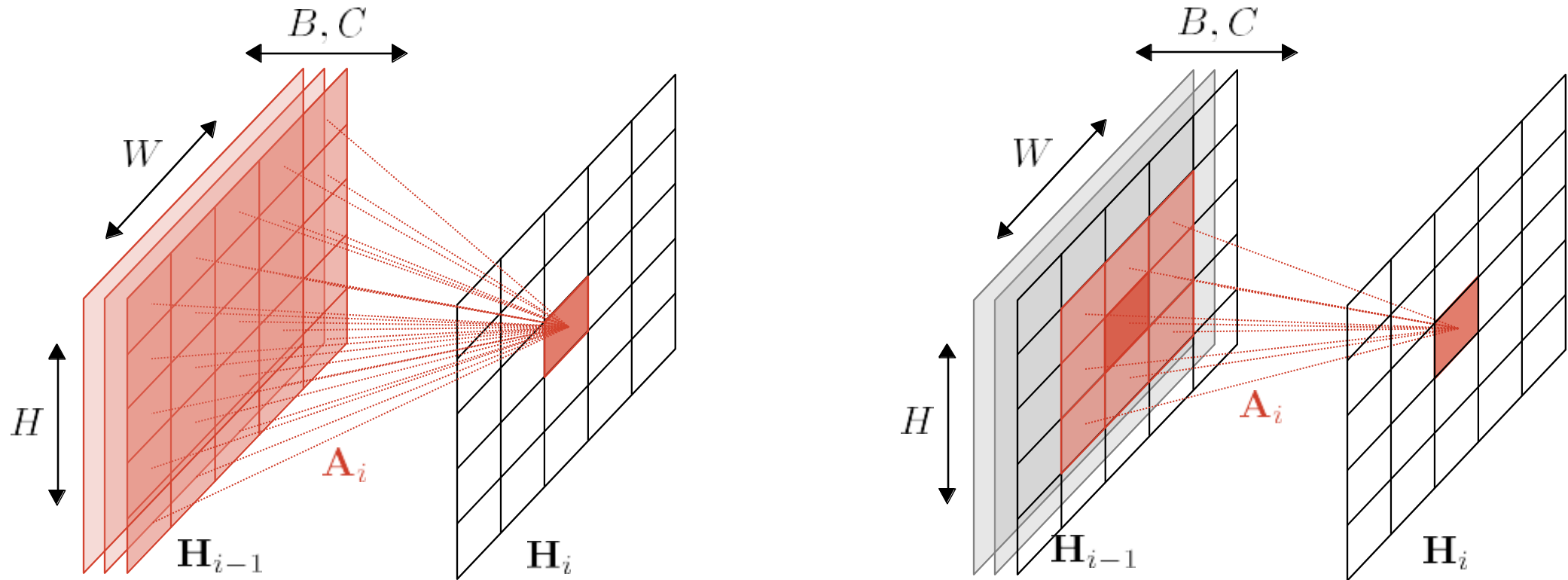
- In contrast to MLPs, ConvNet layers have a **spatial extent**

Fully Connected vs. Convolutional Layers



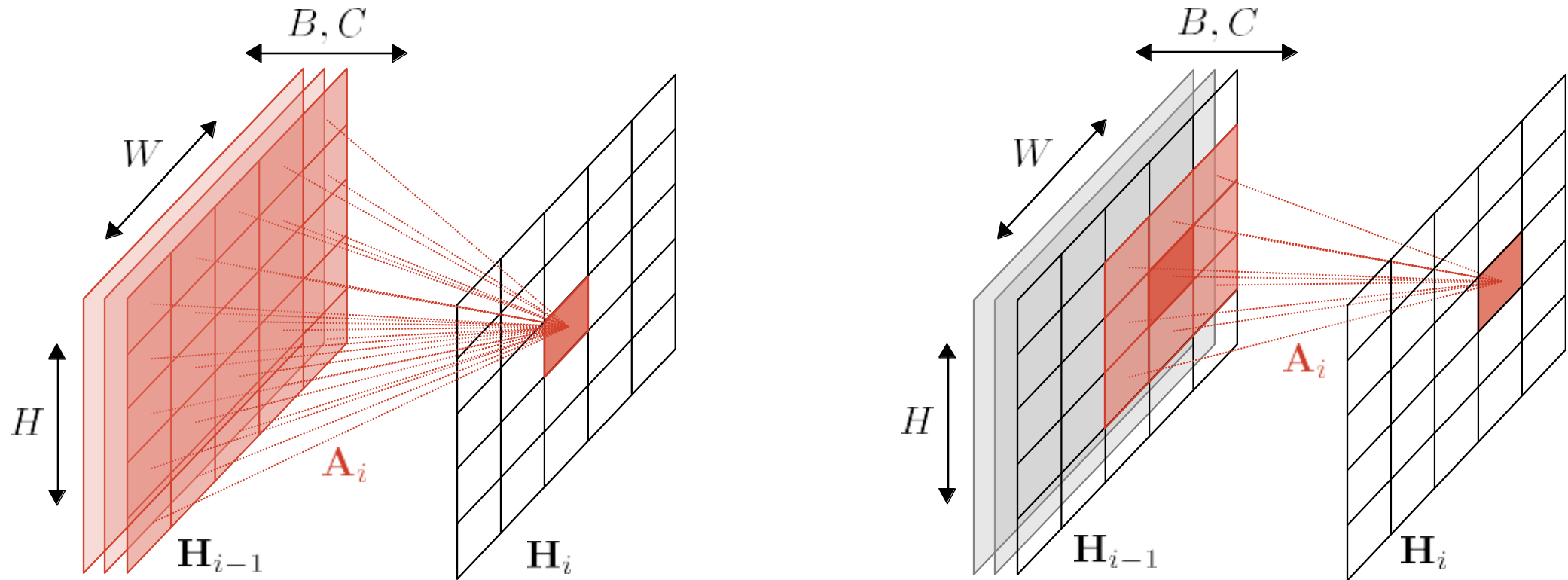
- **Fully connected layer:** $\#Weights = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\#Weights = C_{out} \times (K \times K \times C_{in} + 1)$ ("weight sharing")
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Fully Connected vs. Convolutional Layers



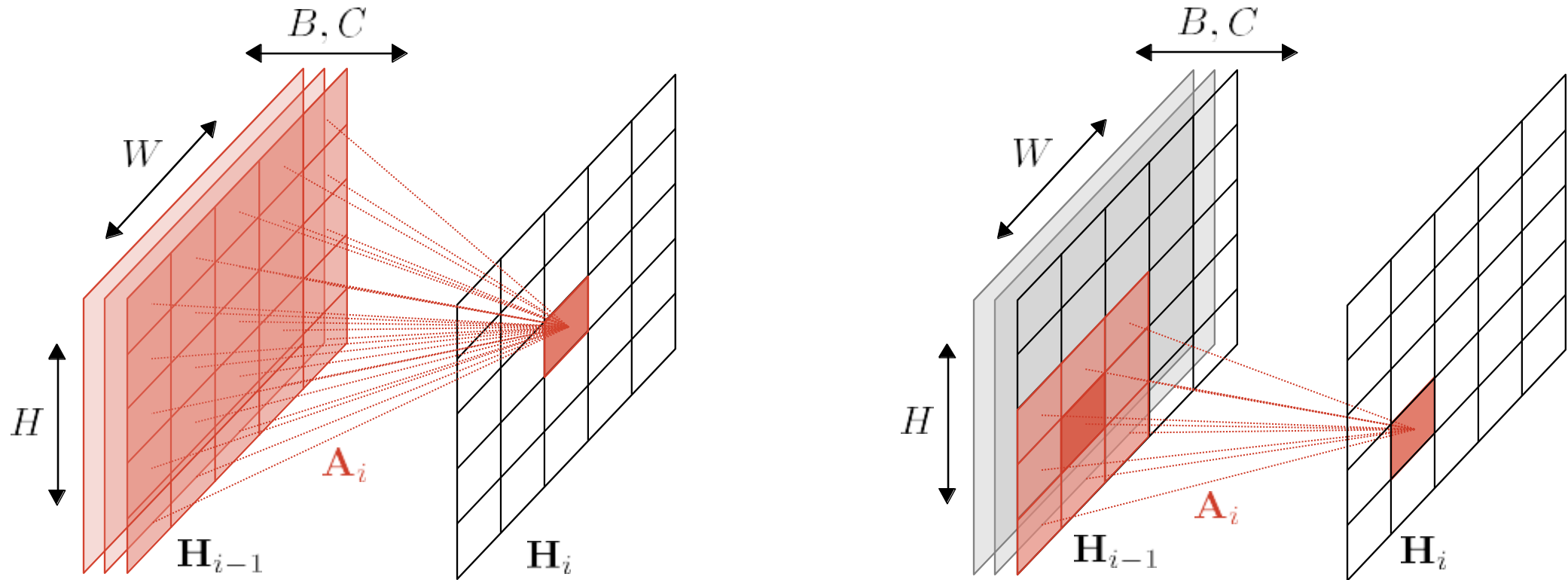
- **Fully connected layer:** $\#Weights = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\#Weights = C_{out} \times (K \times K \times C_{in} + 1)$ (“weight sharing”)
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Fully Connected vs. Convolutional Layers



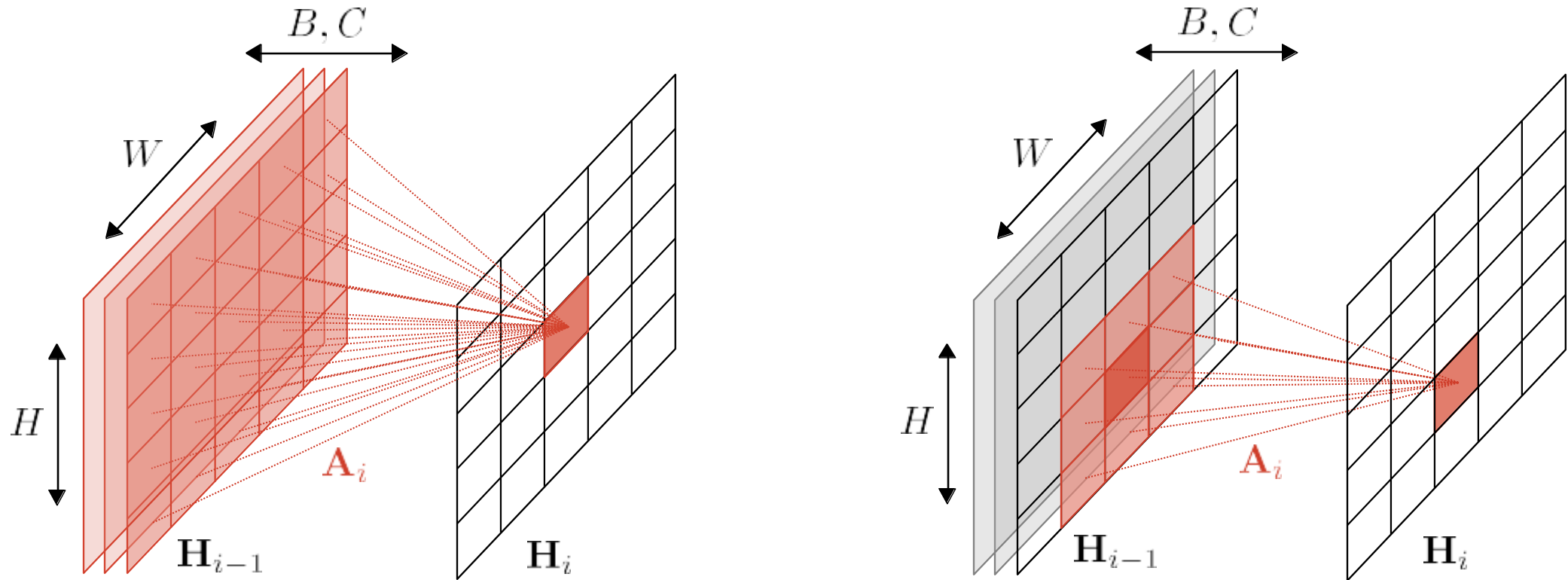
- **Fully connected layer:** $\text{\#Weights} = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\text{\#Weights} = C_{out} \times (K \times K \times C_{in} + 1)$ (“weight sharing”)
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Fully Connected vs. Convolutional Layers



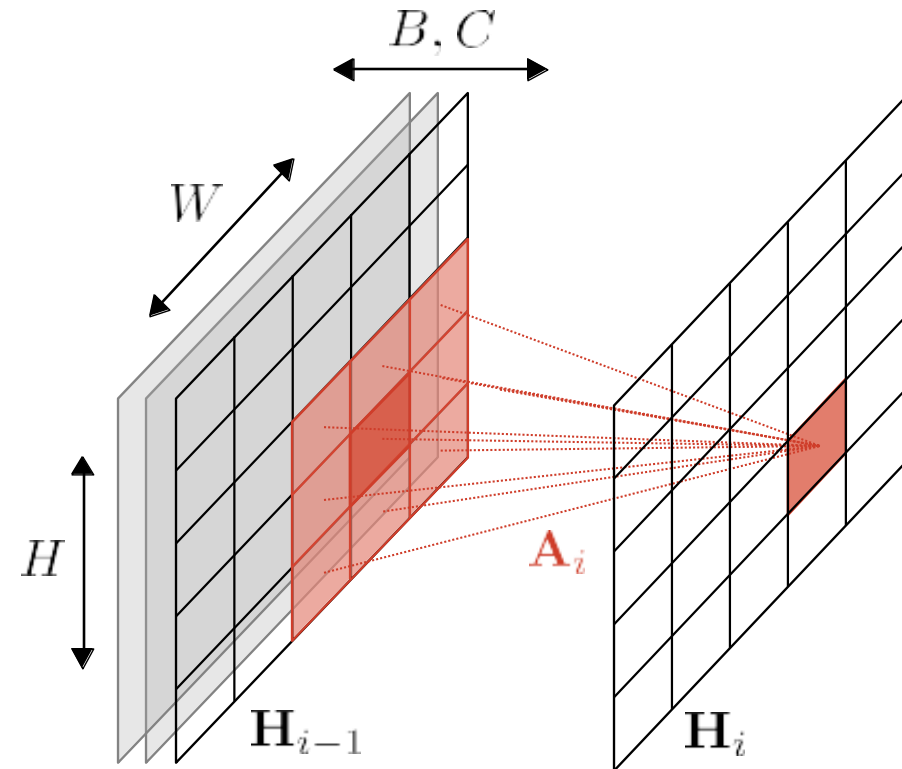
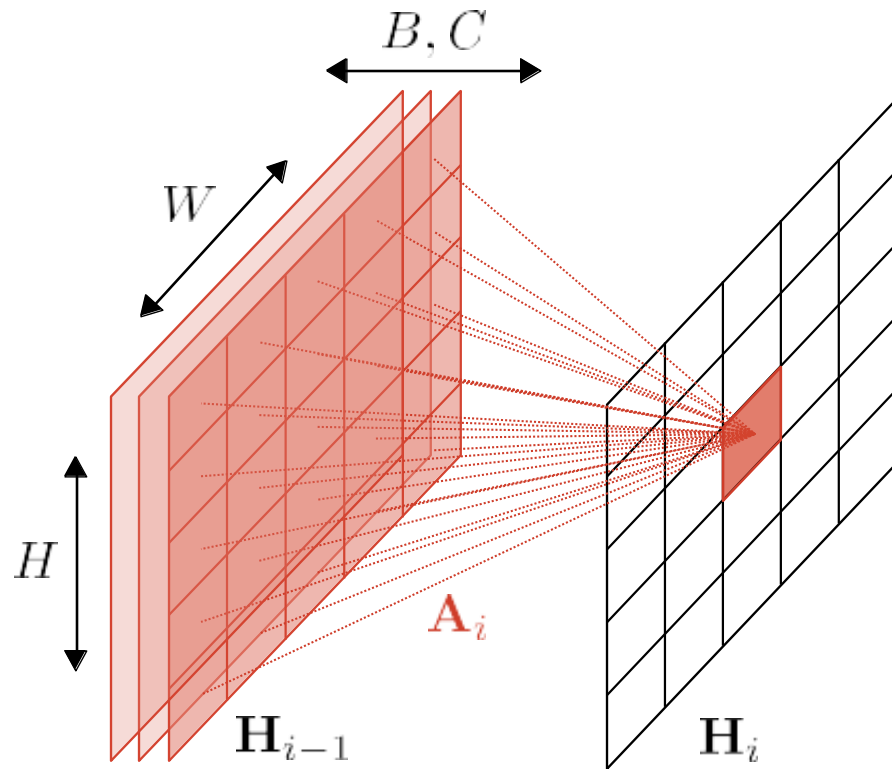
- **Fully connected layer:** $\#Weights = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\#Weights = C_{out} \times (K \times K \times C_{in} + 1)$ (“weight sharing”)
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Fully Connected vs. Convolutional Layers



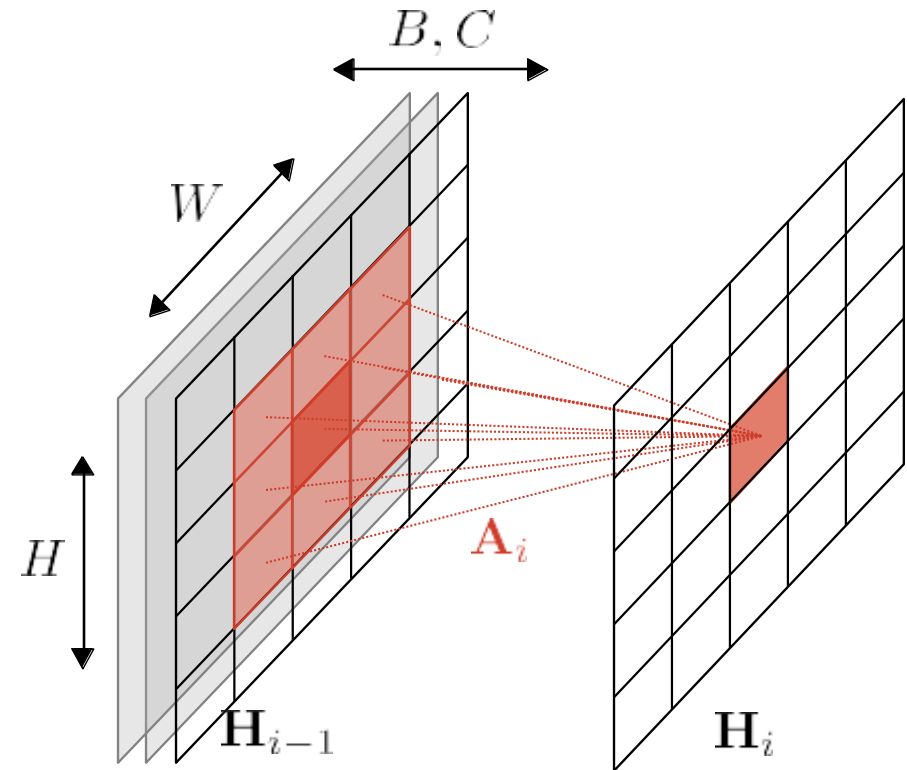
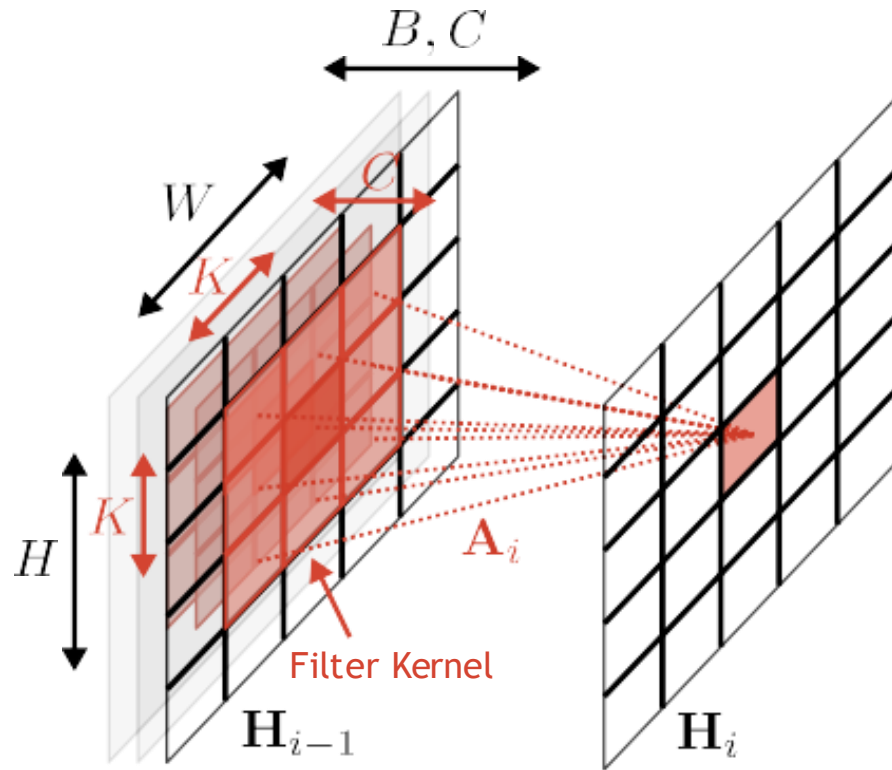
- **Fully connected layer:** $\text{\#Weights} = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\text{\#Weights} = C_{out} \times (K \times K \times C_{in} + 1)$ (“weight sharing”)
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Fully Connected vs. Convolutional Layers



- **Fully connected layer:** $\#Weights = W \times H \times C_{out} \times (W \times H \times C_{in} + 1)$
- **Convolutional layer:** $\#Weights = C_{out} \times (K \times K \times C_{in} + 1)$ (“weight sharing”)
- With C_{in} input and C_{out} output channels, layer size $W \times H$ and kernel size $K \times K$
- Convolution kernel is three-dimensional. Only 1 output channel shown for clarity.

Illustration Conventions



- Convolution kernels are 3D, but in illustrations we drop the C dimension for clarity.
- Usually, multiple convolution kernels are convolved with the input, each producing a different output channel. In illustrations, we show a single convolution for clarity.
- We will use Einstein notation to make all operations formally clear and concise.

Einstein Notation

Einstein Notation

We use a modified form of **Einstein Notation**
where **capital letters** are used to denote **slices of a tensor**:

- ▶ $A[i, j]$ denotes **one element** of the matrix A
- ▶ $A[i, J]$ denotes the **i'th row** of matrix A
- ▶ $A[I, j]$ denotes the **j'th column** of matrix A
- ▶ $A[I, J]$ denotes the **full matrix** A

- ▶ $H[i, j, k]$ denotes **one element** of the tensor H
- ▶ ...

Einstein Notation

Repeated capital letters in a product denote **summation** over those letters:

$$\begin{aligned} \mathbf{y} = \mathbf{A}\mathbf{x} &\equiv y[i] = \sum_j A[i, j]x[j] \\ &\equiv y[i] = A[i, J]x[J] \end{aligned}$$

$$\begin{aligned} \mathbf{y} = \mathbf{x}^\top \mathbf{A} &\equiv y[j] = \sum_i A[i, j]x[i] \\ &\equiv y[j] = A[I, j]x[I] \end{aligned}$$

Multiple capital letters denote summation over the **combination** of indices:

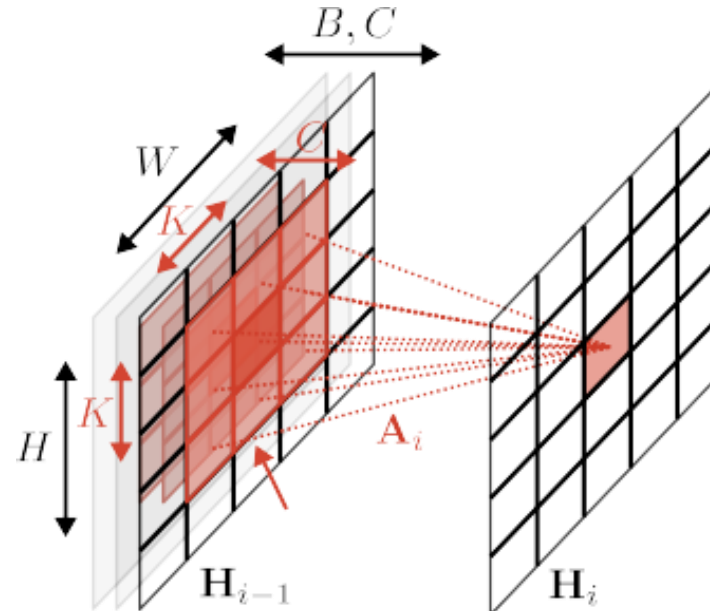
$$c = \sum_{i,j} A[i, j]B[i, j] = A[I, J]B[I, J]$$

Einstein Notation

- ▶ The indices of tensors typically have **types** such as “batch index”, “x coordinate”, “y coordinate” or “channel index”
- ▶ Writing a tensor as $A[B, X, Y, C]$ with batch B , coordinates X, Y and feature channels C makes the type of the tensor elements and the order of the indices **explicit**. It therefore also disambiguates $\mathbf{A}$ from $\mathbf{A}^T$.
- ▶ We will therefore use Einstein notation to describe the (hidden) layers and filter weights in a CNN
- ▶ Einstein notation can also be used in **NumPy** (`numpy.einsum`)

Convolution Layer

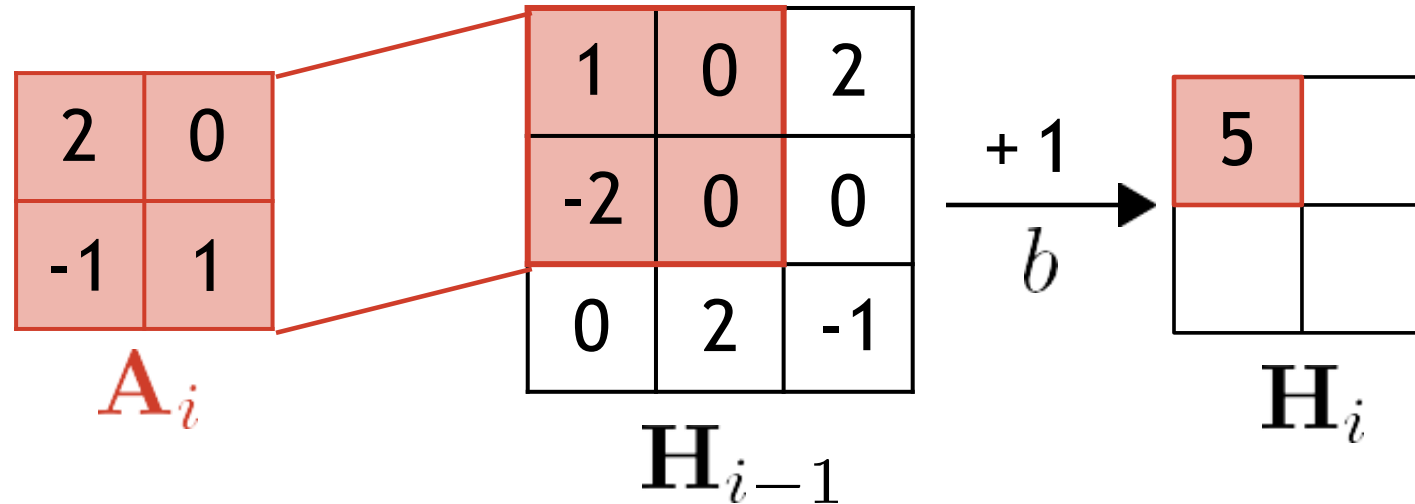
Convolution Layer



$$\underbrace{H_i[b, x, y, c_{out}]}_{\text{Current Layer}} = g \left(\underbrace{A_i[\Delta X, \Delta Y, C_{in}, c_{out}]}_{\text{Weights}} \underbrace{H_{i-1}[b, x + \Delta X, y + \Delta Y, C_{in}]}_{\text{Prev. Layer}} + \underbrace{b_i[c_{out}]}_{\text{Bias}} \right)$$

- b : Batch index x, y : Spatial locations C_{in}, C_{out} : Feature channels g : Act. func.
- $H_i[b, x, y, d]$ denotes feature d at image position x, y in layer i of batch b
- A_i and b_i depend on i as every layer has its own parameters
- Einstein summation considers all combinations over a range of indices $(\Delta X, \Delta Y)$

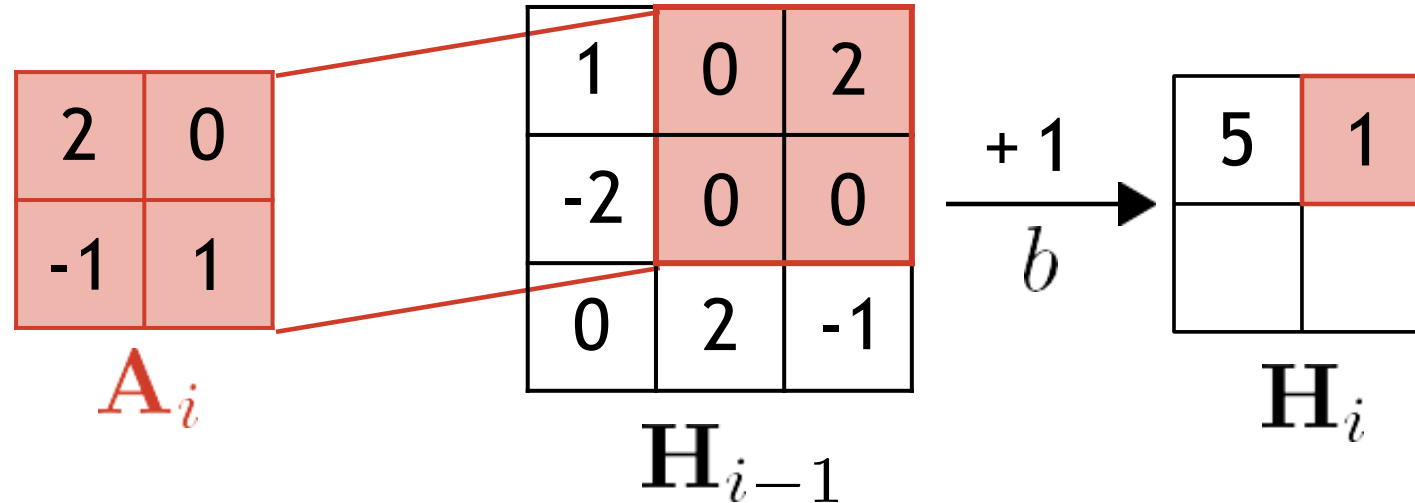
Convolution Example



Remarks:

- Technically, ConvNets implement **correlation** (not convolution) operations
- Low-rank kernels can be implemented by efficient **separable convolutions**

Convolution Example

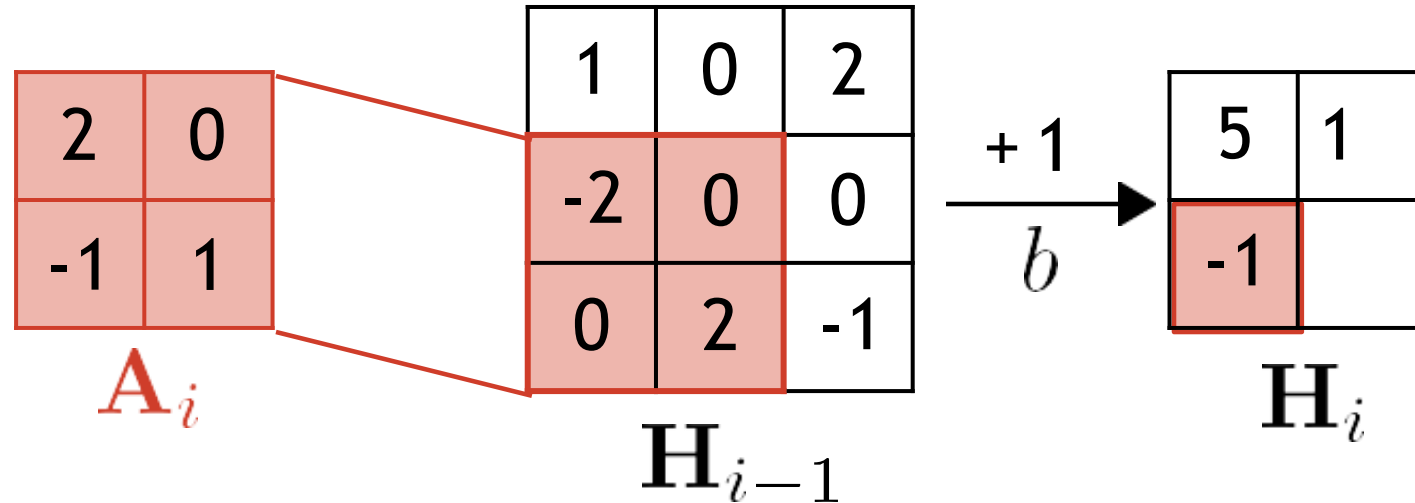


$$0 + 1 = 1$$

Remarks:

- Technically, ConvNets implement **correlation** (not convolution) operations
- Low-rank kernels can be implemented by efficient **separable convolutions**

Convolution Example

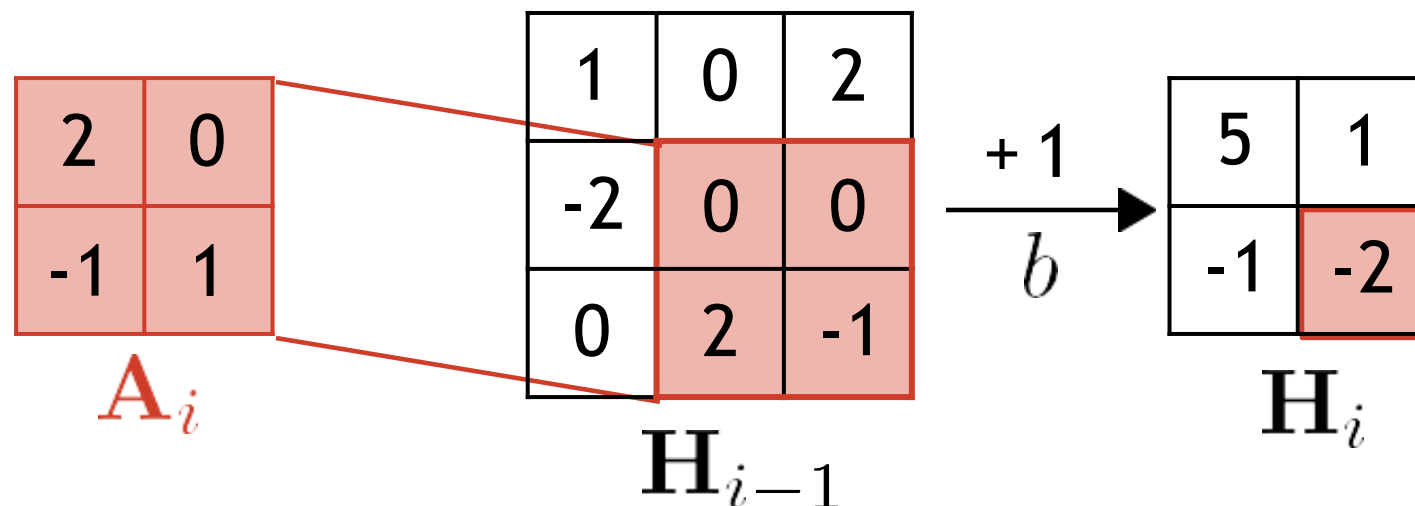


$$2 \cdot (-2) + 1 \cdot 2 + 1 = -1$$

Remarks:

- Technically, ConvNets implement **correlation** (not convolution) operations
- Low-rank kernels can be implemented by efficient **separable convolutions**

Convolution Example



Remarks:

- Technically, ConvNets implement **correlation** (not convolution) operations
- Low-rank kernels can be implemented by efficient **separable convolutions**

Convolution Example



*

0	1	0
1	4	1
0	1	0



- Common filters. In deep learning our goal is to learn the filter kernels.

Convolution Example



*

1	0	-1
2	0	-2
1	0	-1



- Common filters. In deep learning our goal is to learn the filter kernels.

Invariance & Equivariance

Invariance: $f(\mathbf{H}) = f(\mathbf{T}_\theta[\mathbf{H}])$

Equivariance: $\mathbf{T}_\theta[f](\mathbf{H}) = f(\mathbf{T}_\theta[\mathbf{H}])$

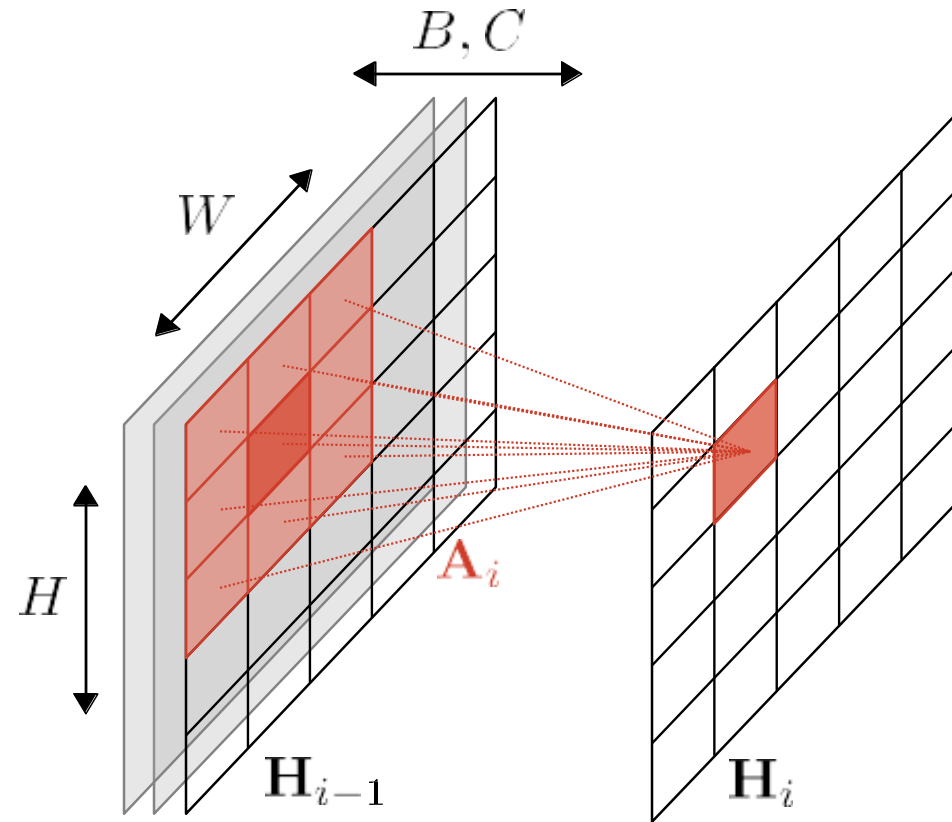
- ▶ With feature map $\mathbf{H}$, operation $f(\cdot)$ and transformation $\mathbf{T}_\theta$
- ▶ An operation is **invariant** if it returns the same result for any transformed input
- ▶ An operation is **equivariant** if its output transforms as its input for some specific transformation type (in the case of ConvNets: translation)

Translation Equivariance of Convolution

- ▶ Convolutions are **translation equivariant** but not translation invariant
- ▶ Shifting the input, results in a **shifted output** (exception: image boundaries)
- ▶ Interpretation as **inductive bias** built into ConvNet architecture

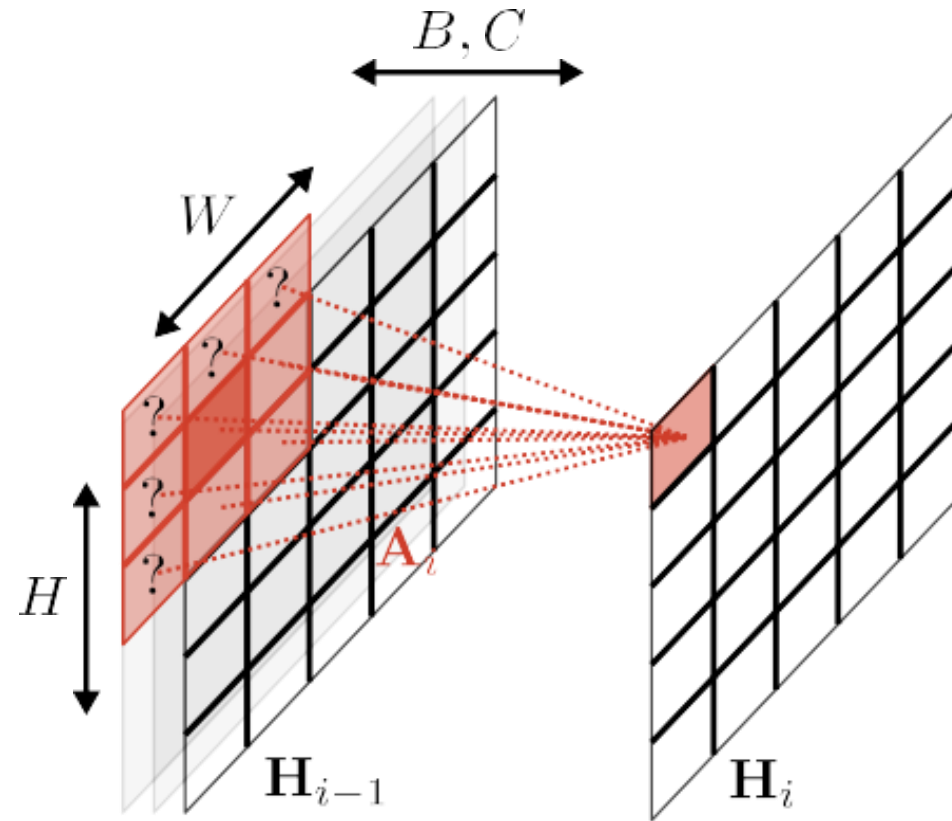
Padding

Padding



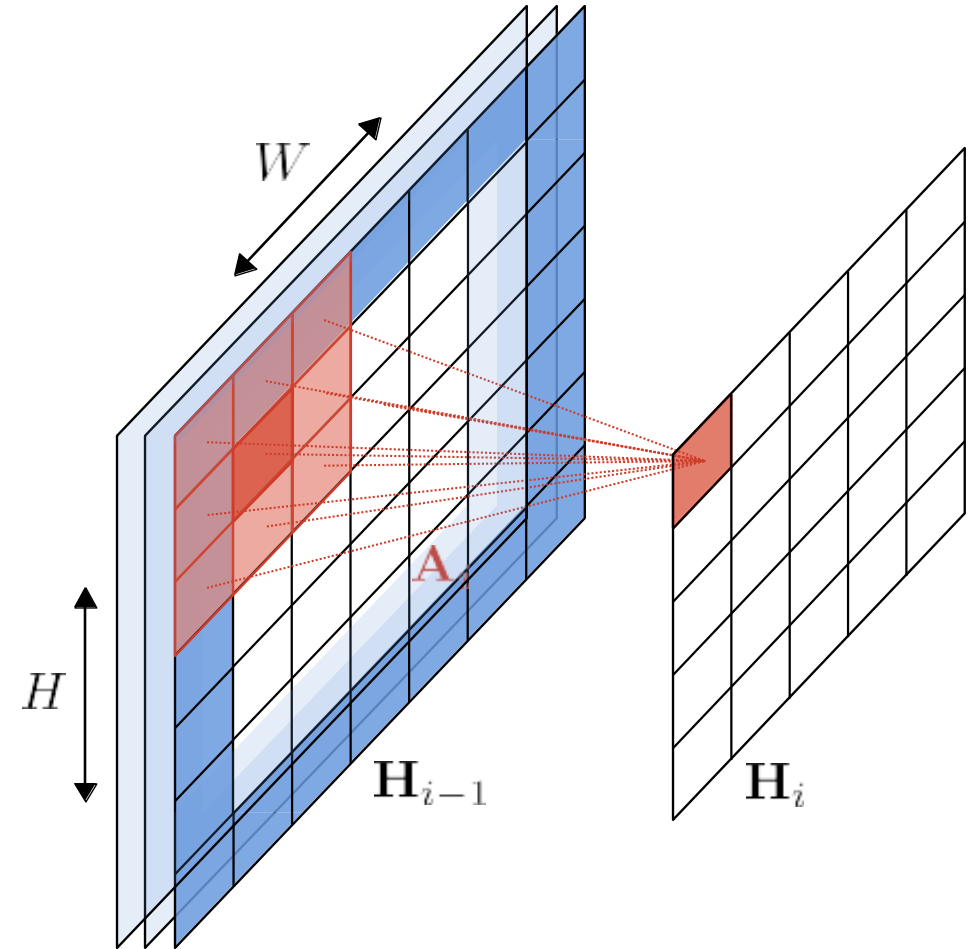
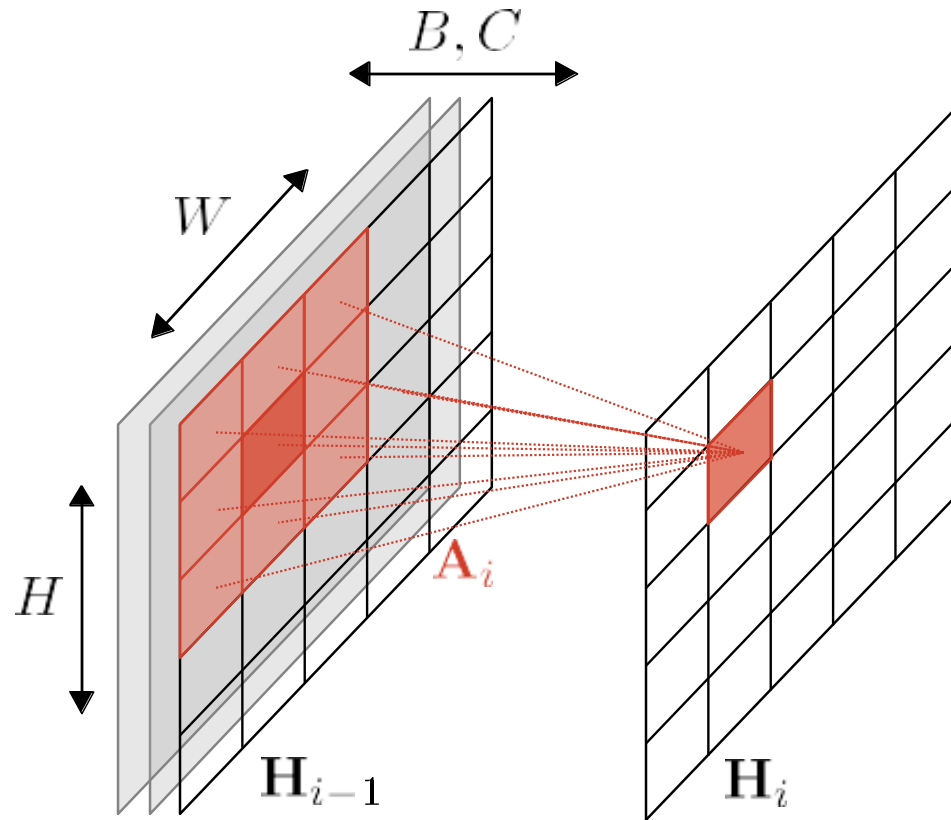
- Convolutions can only be executed if kernel lies entirely **within input domain**
- Often undesirable and **inconvenient** as it couples architecture and input size

Padding



- Convolutions can only be executed if kernel lies entirely **within input domain**
- Often undesirable and **inconvenient** as it couples architecture and input size

Padding



Idea of Padding:

- Add boundary of appropriate size with zeros (blue) around input tensor

Padding

- ▶ Let $H[\cdot]$ denote a padded version of the tensor $H[\cdot]$
- ▶ The output of the **padded convolution** is given by:

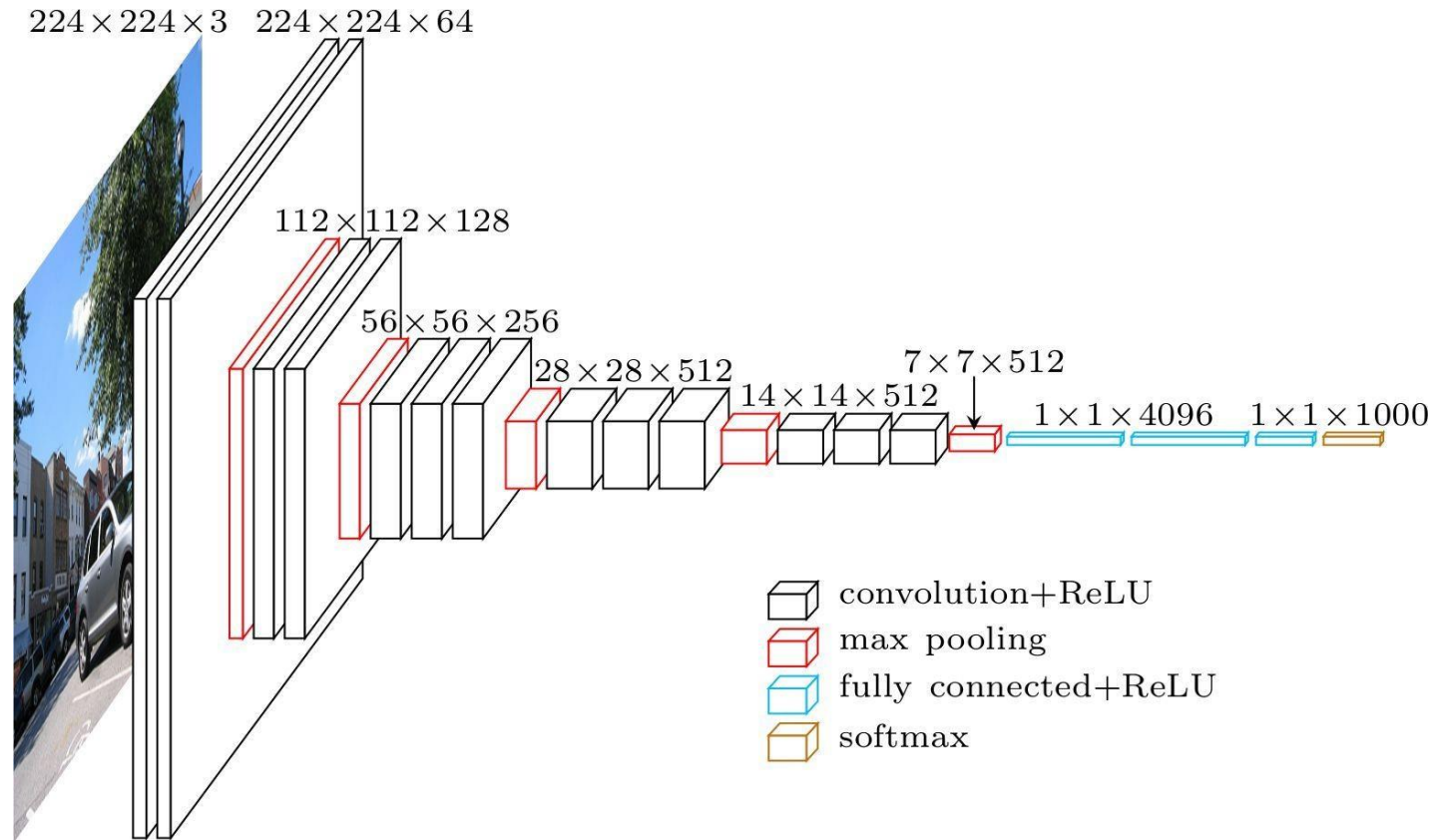
–
–

$$\underbrace{H_i[b, x, y, c_{out}]}_{\text{Current Layer}} = g \left(\underbrace{A_i[\Delta X, \Delta Y, C_{in}, c_{out}]}_{\text{Weights}} \underbrace{H'_{i-1}[b, x + \Delta X, y + \Delta Y, C_{in}]}_{\text{Prev. Layer}} + \underbrace{b_i[c_{out}]}_{\text{Bias}} \right)$$

- ▶ Note: if the input is padded, then Δx and Δy are non-negative

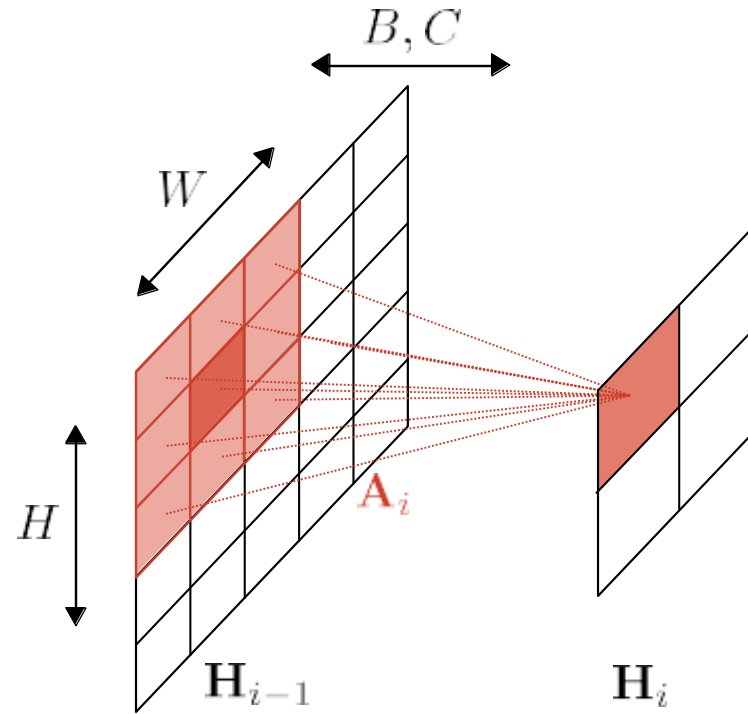
Downsampling

Downsampling



- Downsampling **reduces the spatial resolution** (e.g., for image level predictions)
- Downsampling **increases the receptive field** (which pixels influence a neuron)

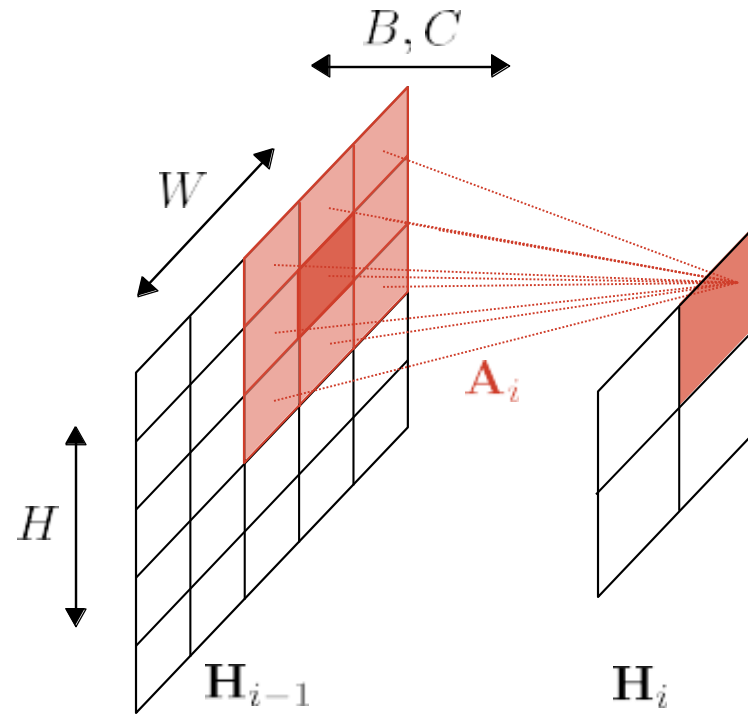
Pooling



$$\underbrace{H_i[b, x, y, c]}_{\text{Current Layer}} = \max_{\Delta x, \Delta y} \underbrace{H_{i-1}[b, s \cdot x + \Delta x, s \cdot y + \Delta y, c]}_{\text{Prev. Layer}}$$

- ▶ Typically, stride $s = 2$ and kernel size $2 \times 2 \Rightarrow$ **reduces spatial dimensions** by 2
- ▶ Pooling has **no parameters** (typical pooling operations: max, min, mean)
- ▶ Pooling is **applied to each channel separately** $\Rightarrow$ keeps number of channels

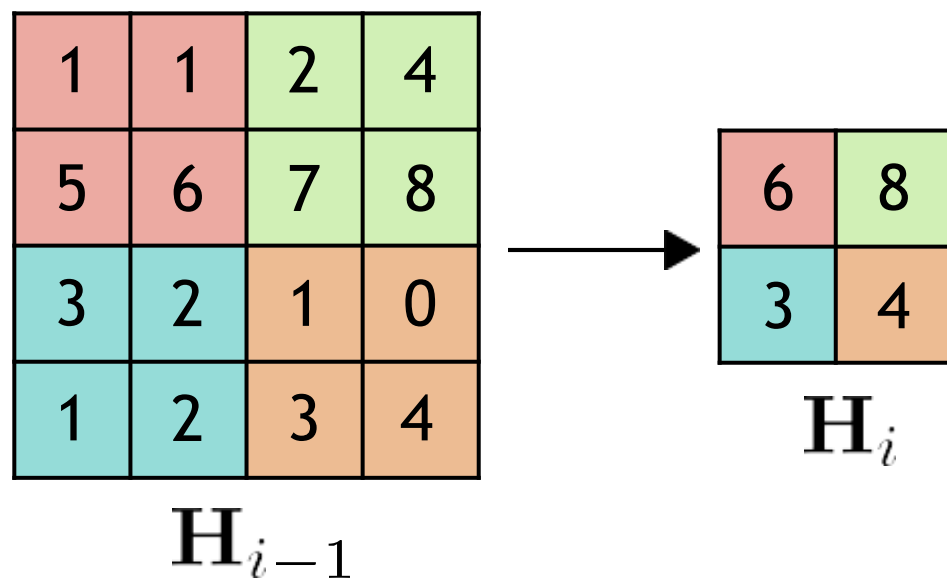
Pooling



$$\underbrace{H_i[b, x, y, c]}_{\text{Current Layer}} = \max_{\Delta x, \Delta y} \underbrace{H_{i-1}[b, s \cdot x + \Delta x, s \cdot y + \Delta y, c]}_{\text{Prev. Layer}}$$

- Typically, stride $s = 2$ and kernel size $2 \times 2 \Rightarrow$ **reduces spatial dimensions** by 2
- Pooling has **no parameters** (typical pooling operations: max, min, mean)
- Pooling is **applied to each channel separately** $\Rightarrow$ keeps number of channels

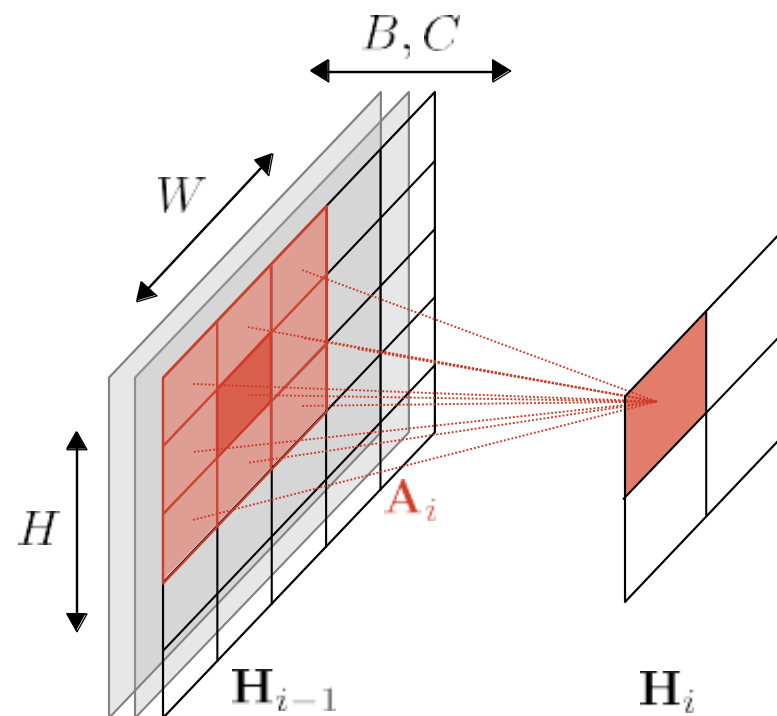
3 Pooling Example



$$\underbrace{H_i[b, x, y, c]}_{\text{Current Layer}} = \max_{\Delta x, \Delta y} \underbrace{H_{i-1}[b, s \cdot x + \Delta x, s \cdot y + \Delta y, c]}_{\text{Prev. Layer}}$$

- ▶ Max pooling with stride $s = 2$ and kernel size 2×2 reduces spatial dimension by 2
- ▶ Remark: Max pooling provides **invariance** to small translations of the input

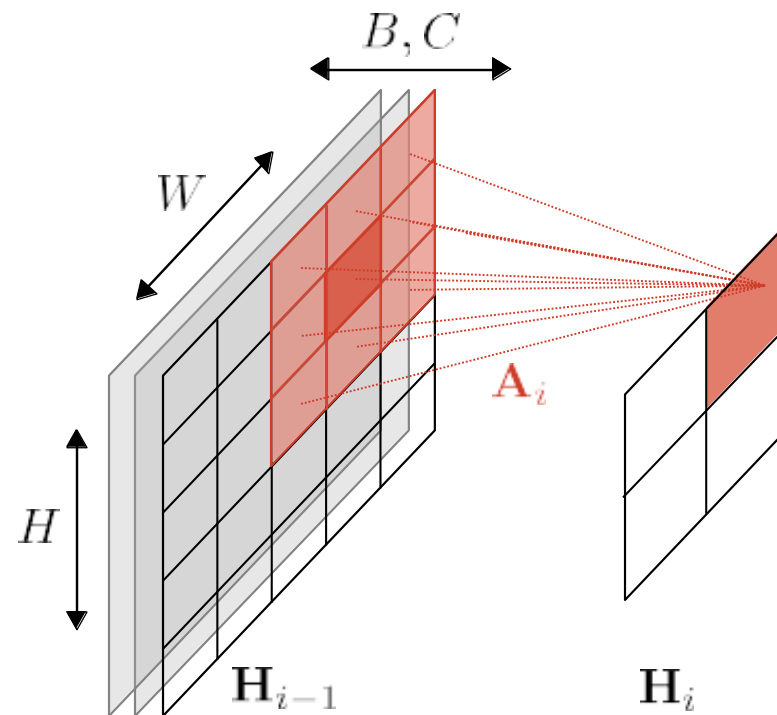
4 Strided Convolution



$$\underbrace{H_i[b, \mathbf{x}, \mathbf{y}, c_{out}]}_{\text{Current Layer}} = g \left(\underbrace{A_i[\Delta X, \Delta Y, C_{in}, c_{out}]}_{\text{Weights}} \underbrace{H_{i-1}[b, \mathbf{s} \cdot \mathbf{x} + \Delta X, \mathbf{s} \cdot \mathbf{y} + \Delta Y, C_{in}]}_{\text{Prev. Layer}} + \underbrace{b_i[c_{out}]}_{\text{Bias}} \right)$$

- **Move convolution filter** (with parameters $\mathbf{A}$ and $\mathbf{b}$) by stride s
- **Learned downsampling.** Often replaces max pooling today (e.g., in ResNet)

4 Strided Convolution

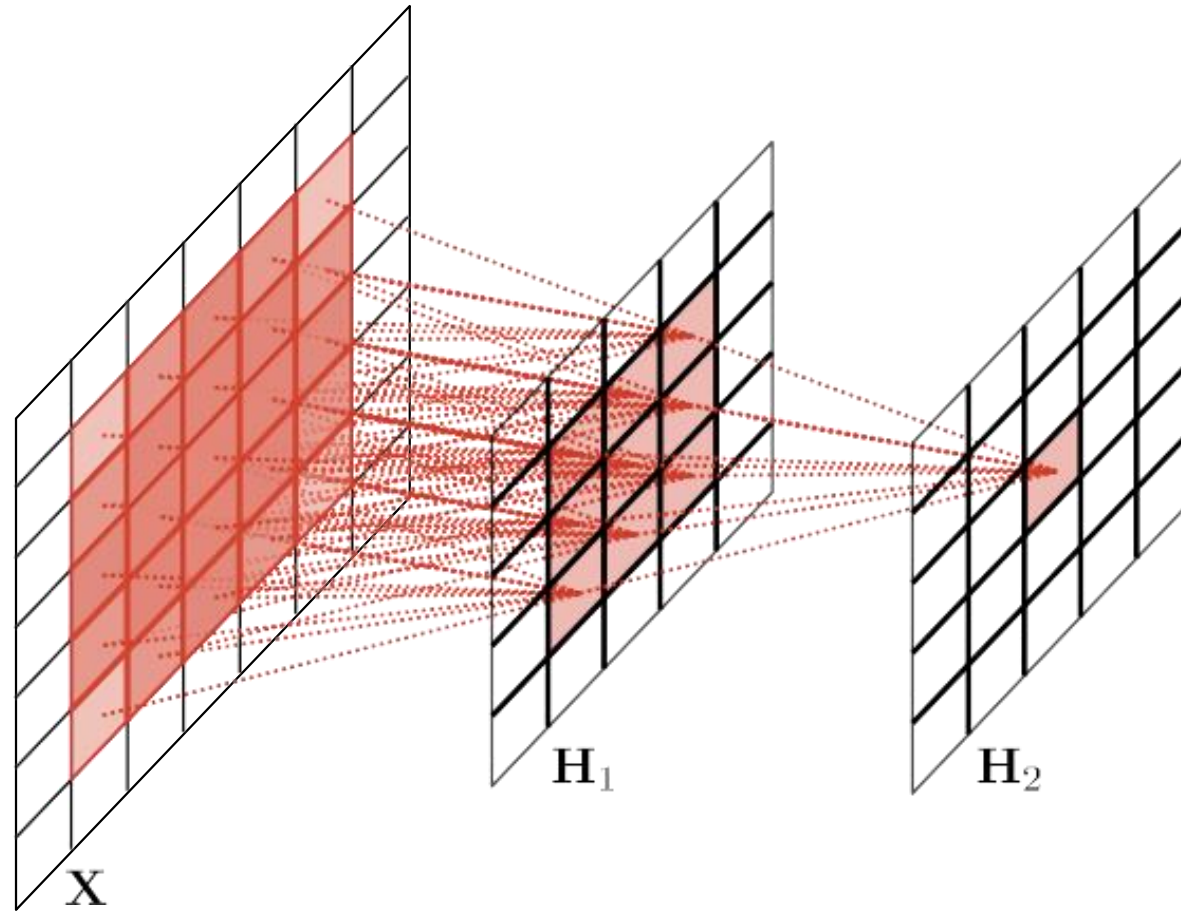


$$\underbrace{H_i[b, \mathbf{x}, \mathbf{y}, c_{out}]}_{\text{Current Layer}} = g \left(\underbrace{A_i[\Delta X, \Delta Y, C_{in}, c_{out}]}_{\text{Weights}} \underbrace{H_{i-1}[b, \mathbf{s} \cdot \mathbf{x} + \Delta X, \mathbf{s} \cdot \mathbf{y} + \Delta Y, C_{in}]}_{\text{Prev. Layer}} + \underbrace{b_i[c_{out}]}_{\text{Bias}} \right)$$

- **Move convolution filter** (with parameters $\mathbf{A}$ and $\mathbf{b}$) by stride s
- **Learned downsampling.** Often replaces max pooling today (e.g., in ResNet)

Receptive Field and Arithmetics

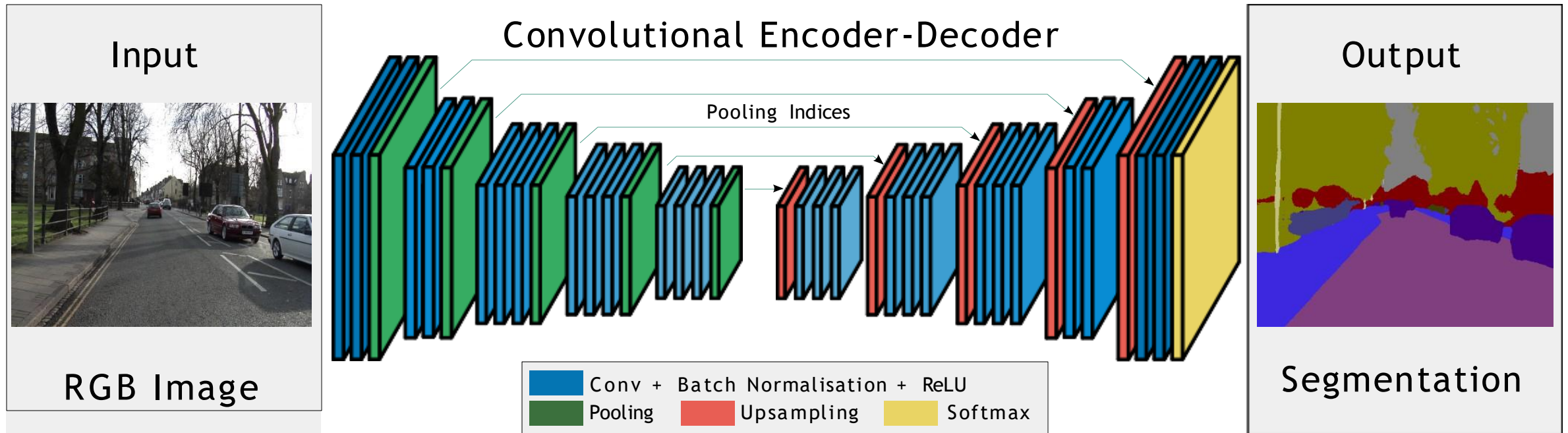
Receptive Field



- **Receptive field** of a neuron is defined as all pixels in the input X which influence it

Upsampling

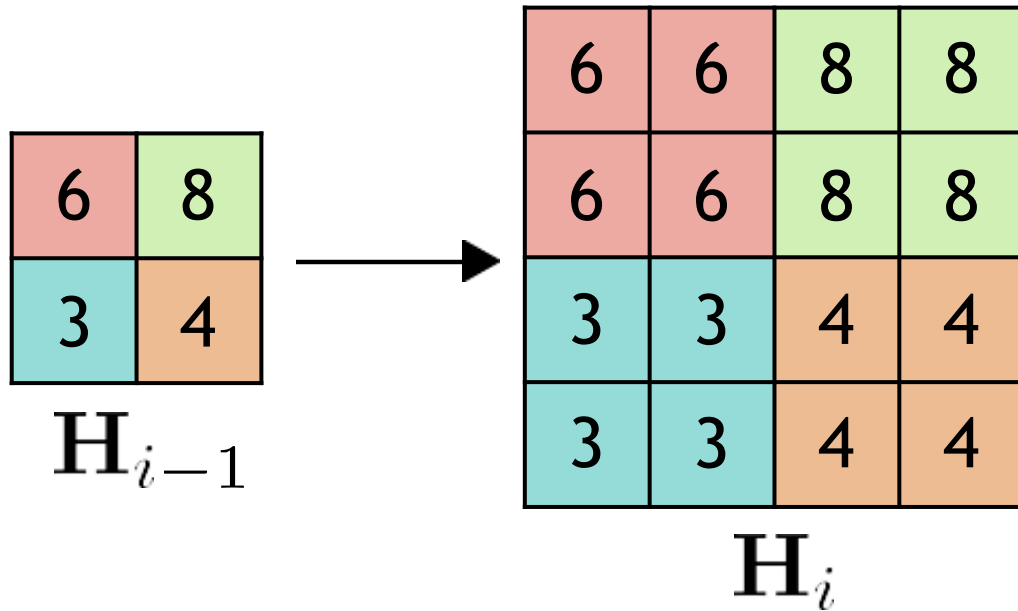
Upsampling



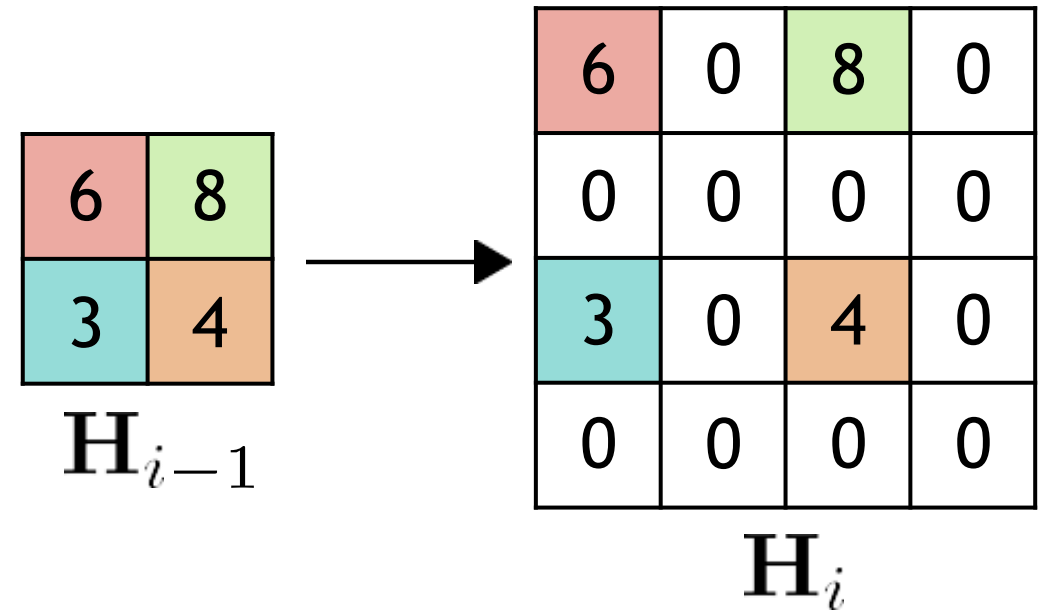
- ▶ If **pixel-level outputs** are desired, we have to **upsample** the features again
- ▶ Downsampling provides strong features with large receptive field
- ▶ Upsampling yields output at the same resolution as input

4 Upsampling

Nearest Neighbor Upsampling



Bed of Nails Upsampling



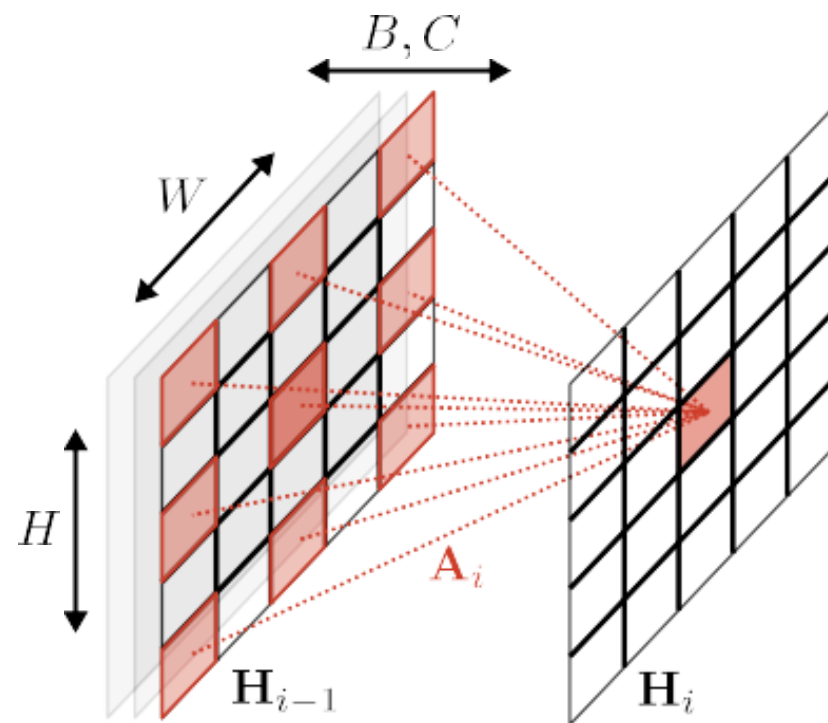
- **Nearest neighbor:** scale each channel using nearest neighbor interpolation
- **Bilinear:** scale each channel using bilinear neighbor interpolation
- **Bed of nails:** insert elements at sparse location (followed by convolution)

Dilated Convolution

7 Dilated Convolution

- ▶ Dilated convolutions are an alternative to combining downsampling and upsampling operations in order to reach a large receptive field size quickly
- ▶ Dilated convolutions increase the receptive field of standard convolutions without increasing the number of parameters
- ▶ Thus, a network with dilated convolutions is able to perform image-level predictions (e.g., semantic segmentation) without upsampling and downsampling
- ▶ Without dilated convolutions, a very large number of layers of standard convolutions would be required to reach the same receptive field size without using upsampling and downsampling (as in a U-Net)
- ▶ Remark: Dilated convolutions are not upsampling operations

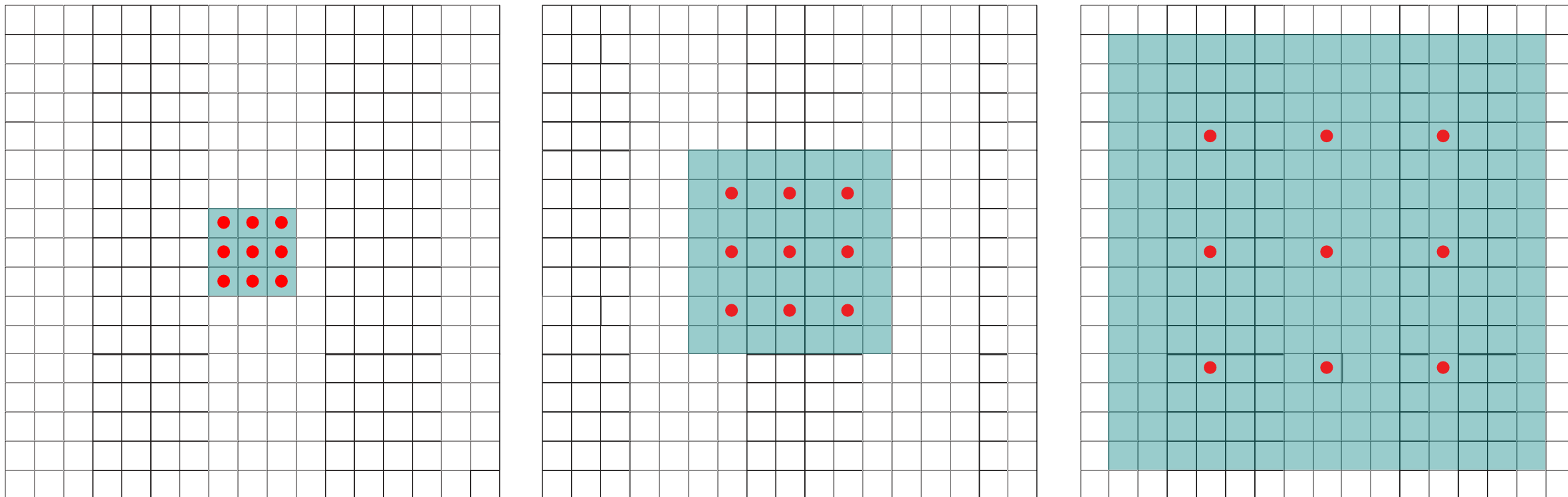
8 Dilated Convolution



$$\underbrace{H_i[b, x, y, c_{out}]}_{\text{Current Layer}} = g \left(\underbrace{A_i[\Delta X, \Delta Y, C_{in}, c_{out}]}_{\text{Weights}} \underbrace{H_{i-1}[b, x + d \cdot \Delta X, y + d \cdot \Delta Y, C_{in}]}_{\text{Prev. Layer}} + \underbrace{b_i[c_{out}]}_{\text{Bias}} \right)$$

- Dilation factor d (here $d=2$) leads to more “distributed” sampling of the input
- **Increases receptive field** without increasing parameters or reducing spatial dim.

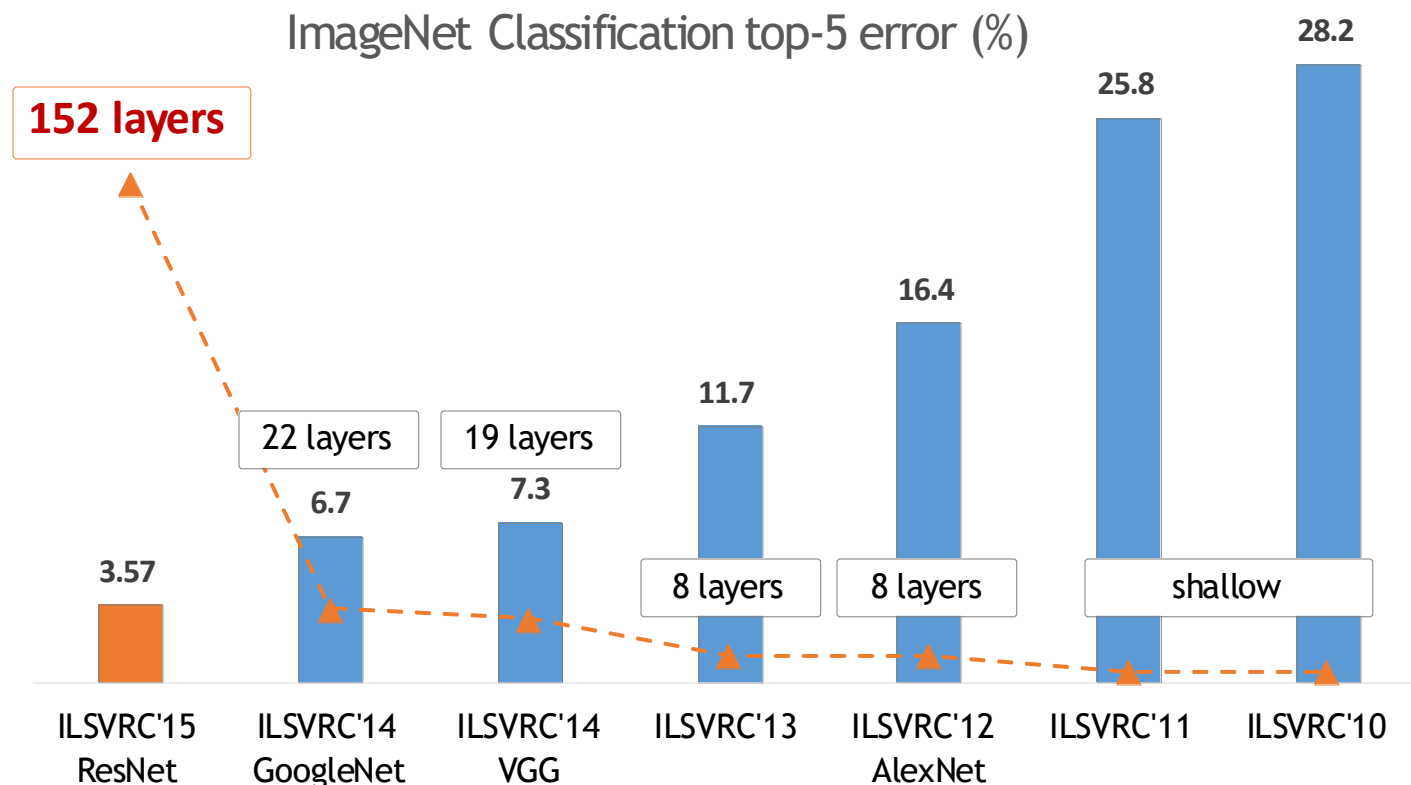
9 Dilated Convolution



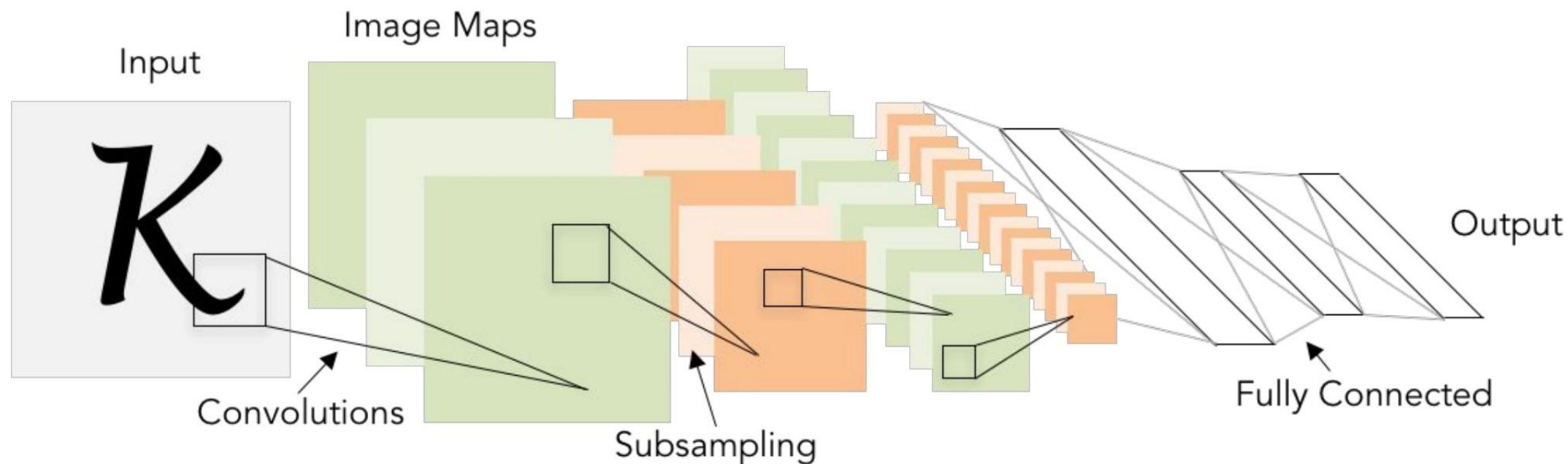
- ▶ **No pooling** required to increase the receptive field
- ▶ Can produce predictions at the **same resolution** as the inputs
- ▶ Examples: semantic segmentation, depth, optical flow, ...

Architectures

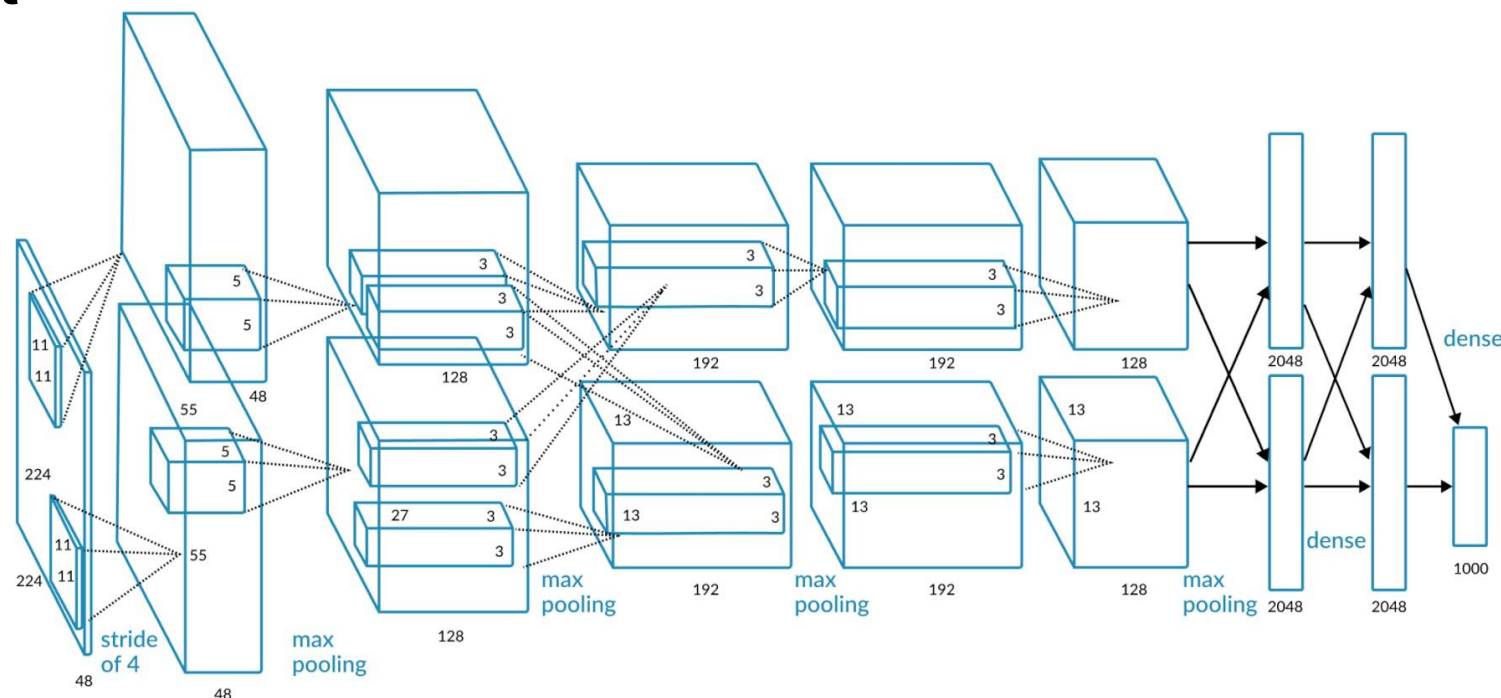
2 ImageNet Large Scale Visual Recognition Challenge



- Deeper networks are better (provided enough training data and compute)
- Nearly all state-of-the-art networks for image applications use convolution layers



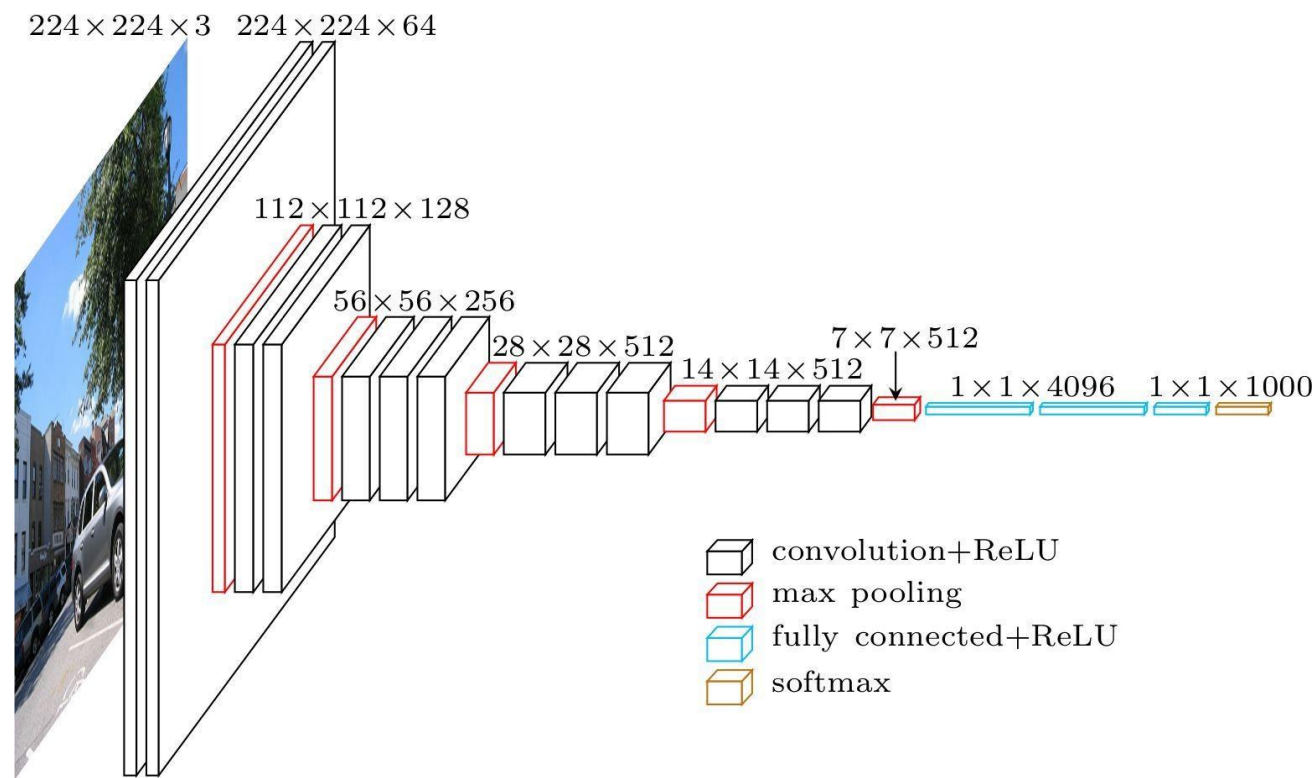
- ▶ 2 convolution layers (5×5), 2 pooling layers (2×2), 2 fully connected layers
- ▶ Achieved state-of-the-art accuracy on MNIST (prior to ImageNet)



AlexNet:

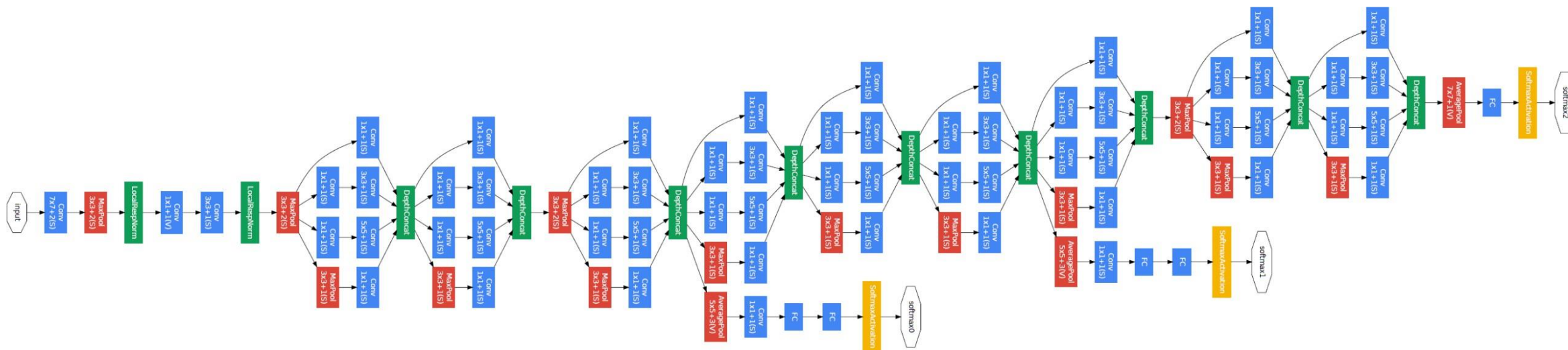
- ▶ 8 layers, ReLUs, dropout, data augmentation, trained on 2 GTX 580 GPUs
- ▶ Number of feature channels increase with depth, spatial resolution decreases
- ▶ Triggered deep learning revolution, showed that CNNs work well in practice

2015: VGG



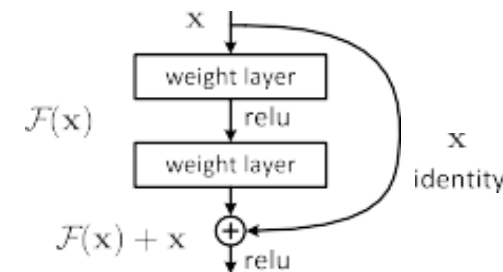
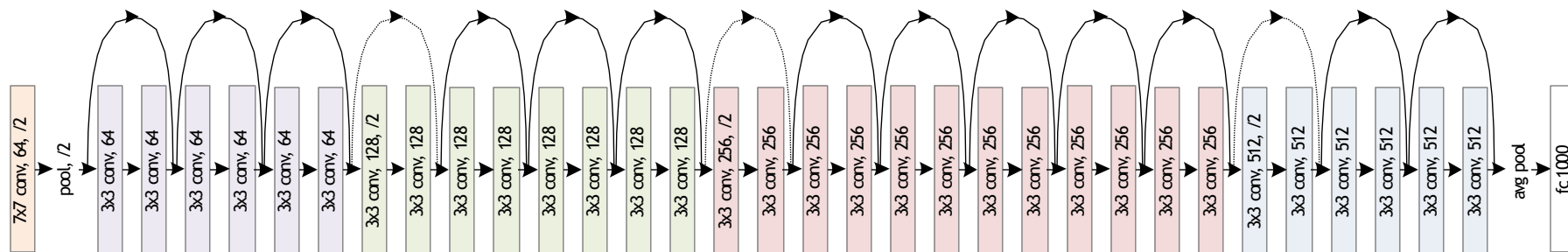
VGG:

- Uses 3×3 convolutions everywhere (same expressiveness, fewer parameters)
- Three 3×3 layers: same receptive field as one 7×7 layer, but less parameters
- 2 Variants: 16 and 19 Layers. Showed that deeper networks are better.



Inception:

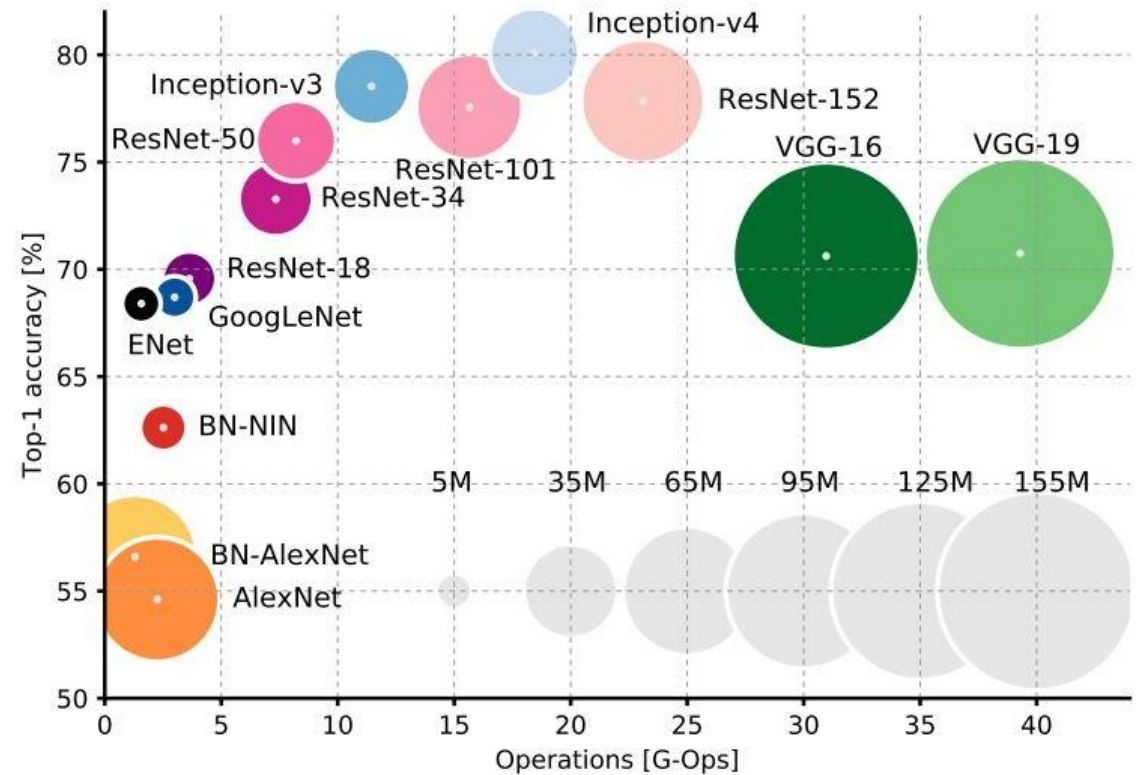
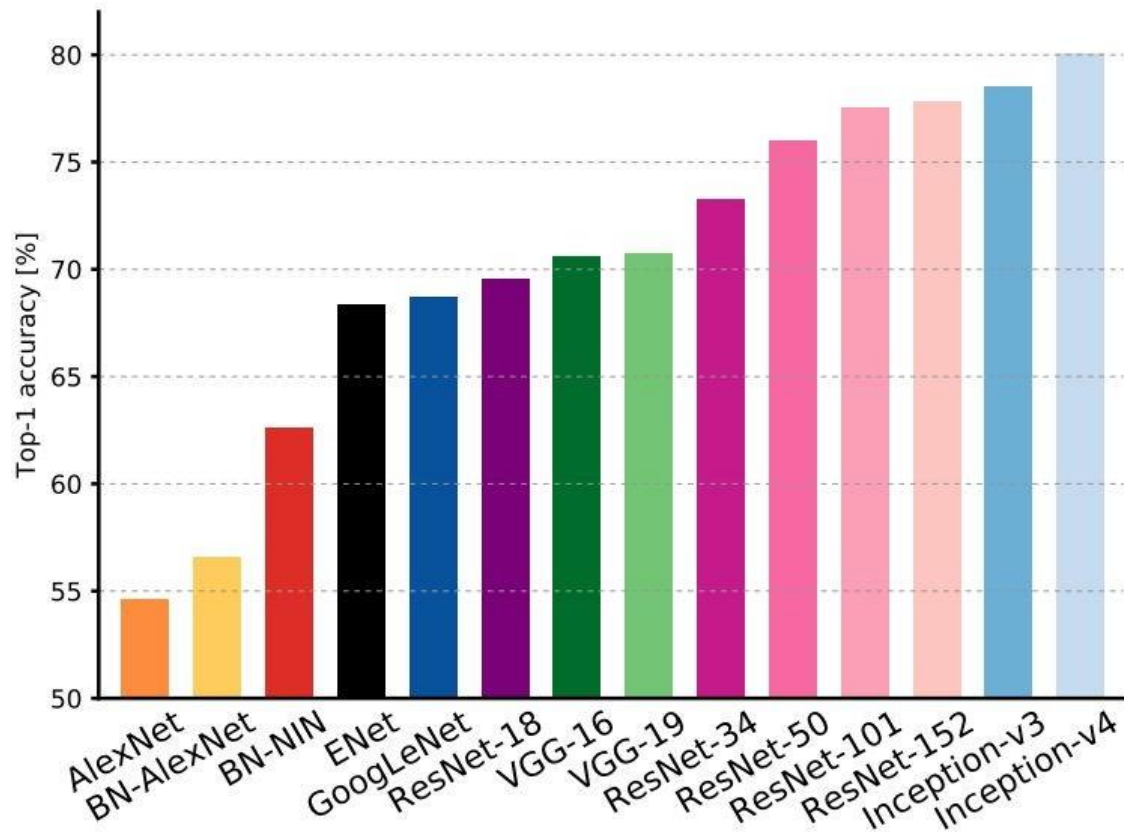
- 22 layers. Inception modules utilize conv/pool operations with varying filter size
- Multiple intermediate classification heads to improve gradient flow
- Global average pooling (no FC layers), 27x less parameters (5 million) than VGG-16
- Uses 1×1 convolutions to reduce number of features $\Rightarrow$ higher efficiency



ResNet:

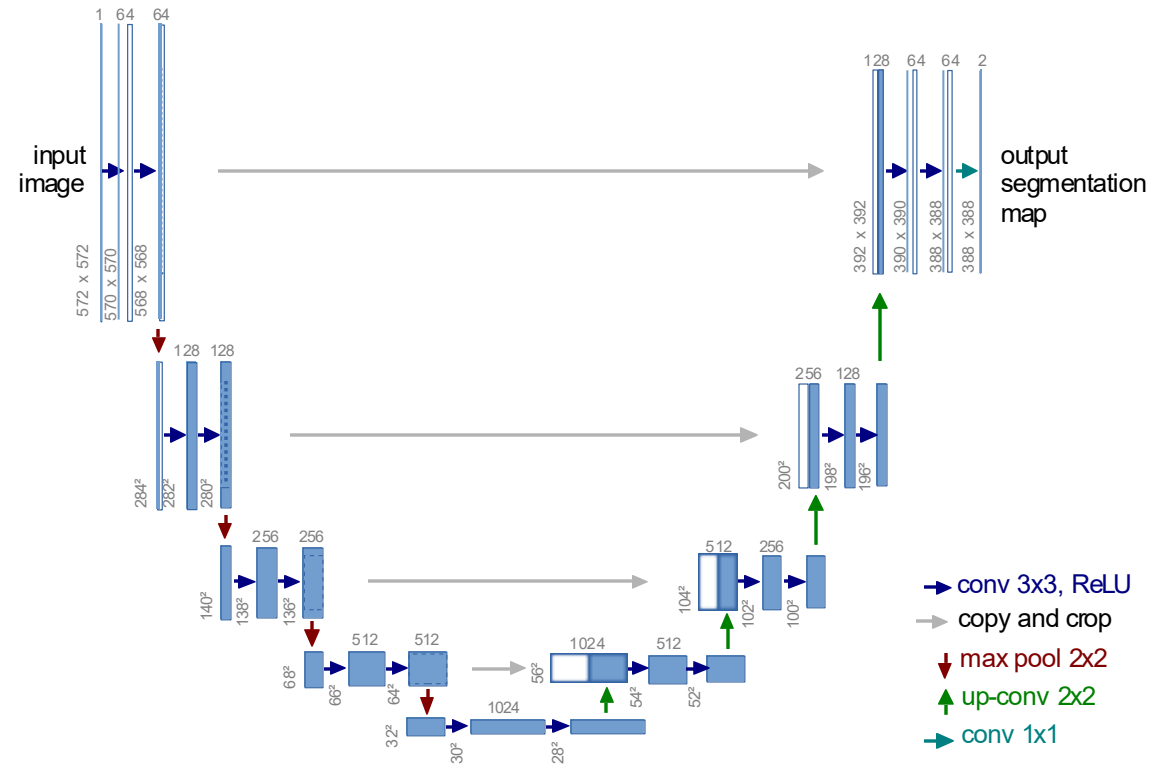
- Residual connections allow for training deeper networks (up to 152 layers)
- Very simple and regular network structure with 3×3 convolutions
- Uses strided convolutions for downsampling
- ResNet and ResNet-like architectures are dominating today

Complexity



- VGG has most parameters and is slowest
- ResNet/Inception/GoogLeNet are faster and have fewer parameters

U-Net

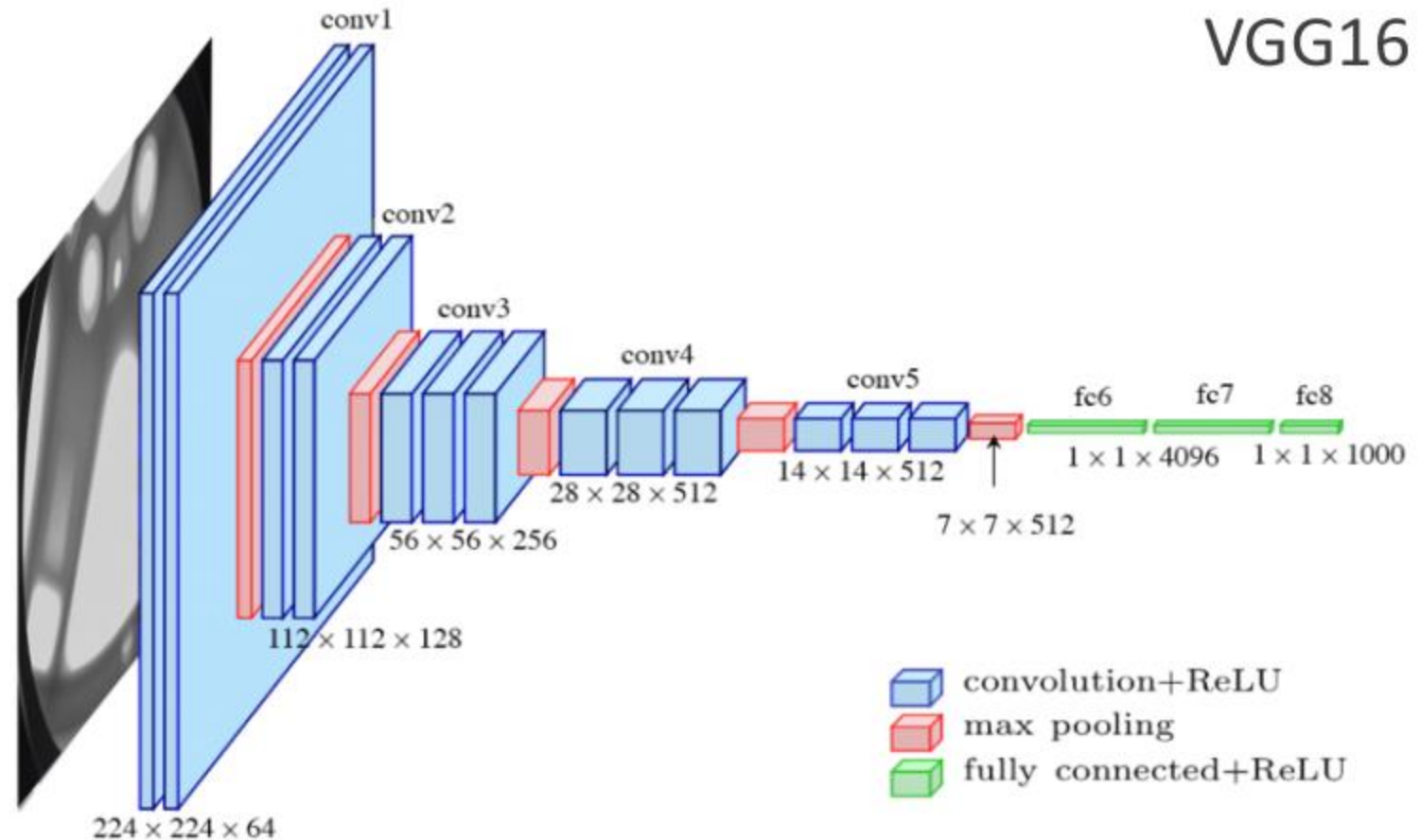
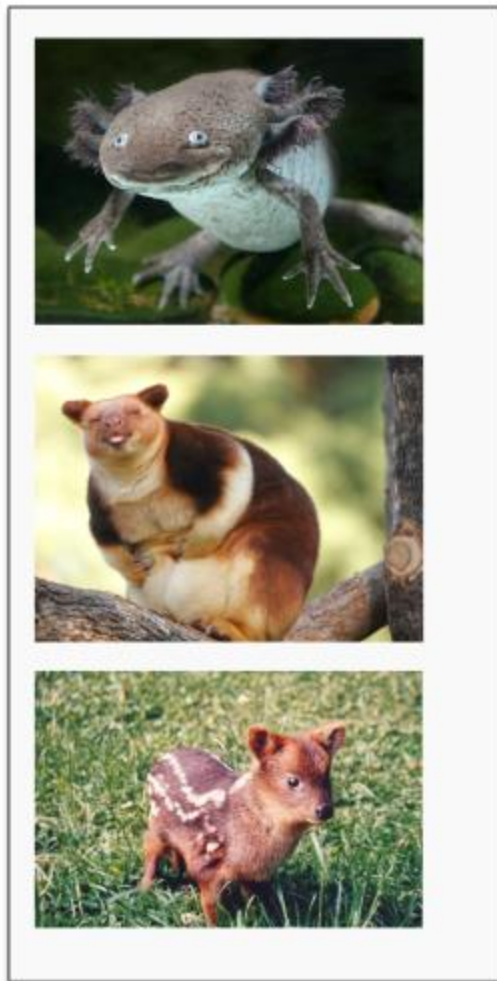


U-Net:

- Max-pooling, up-convolutions and skip-connections
- Defacto standard for many tasks with image output (e.g., depth, segmentation)

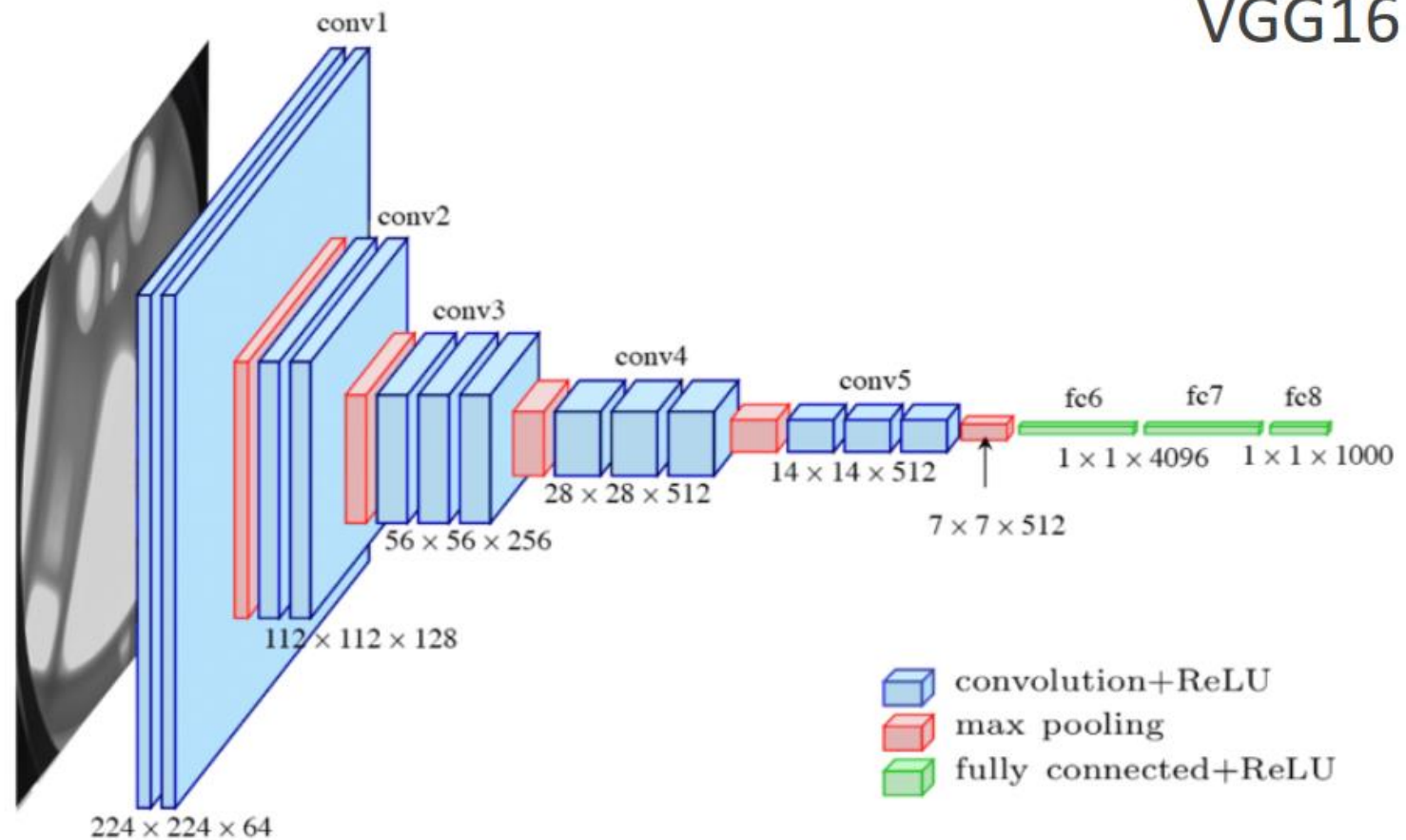
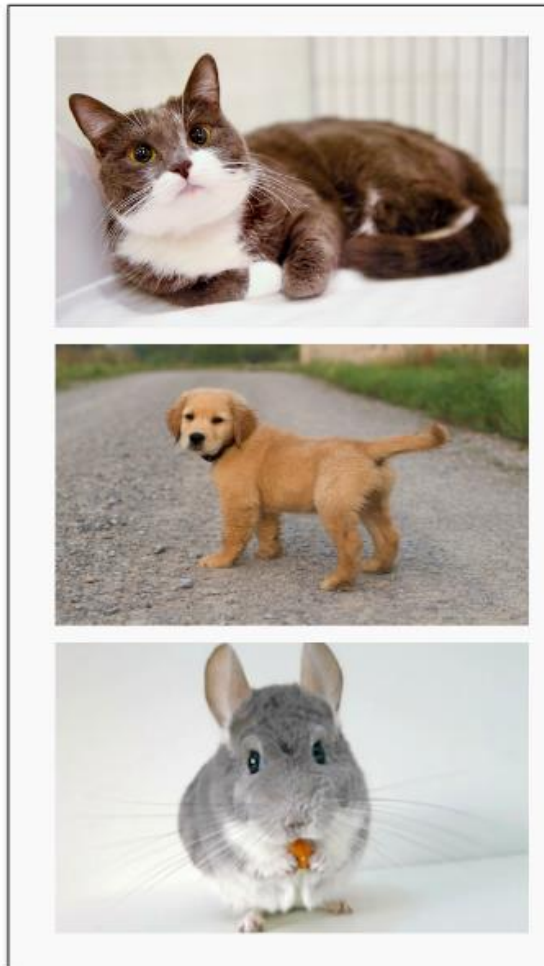
Transfer Learning

Motivation: Classify rare animals



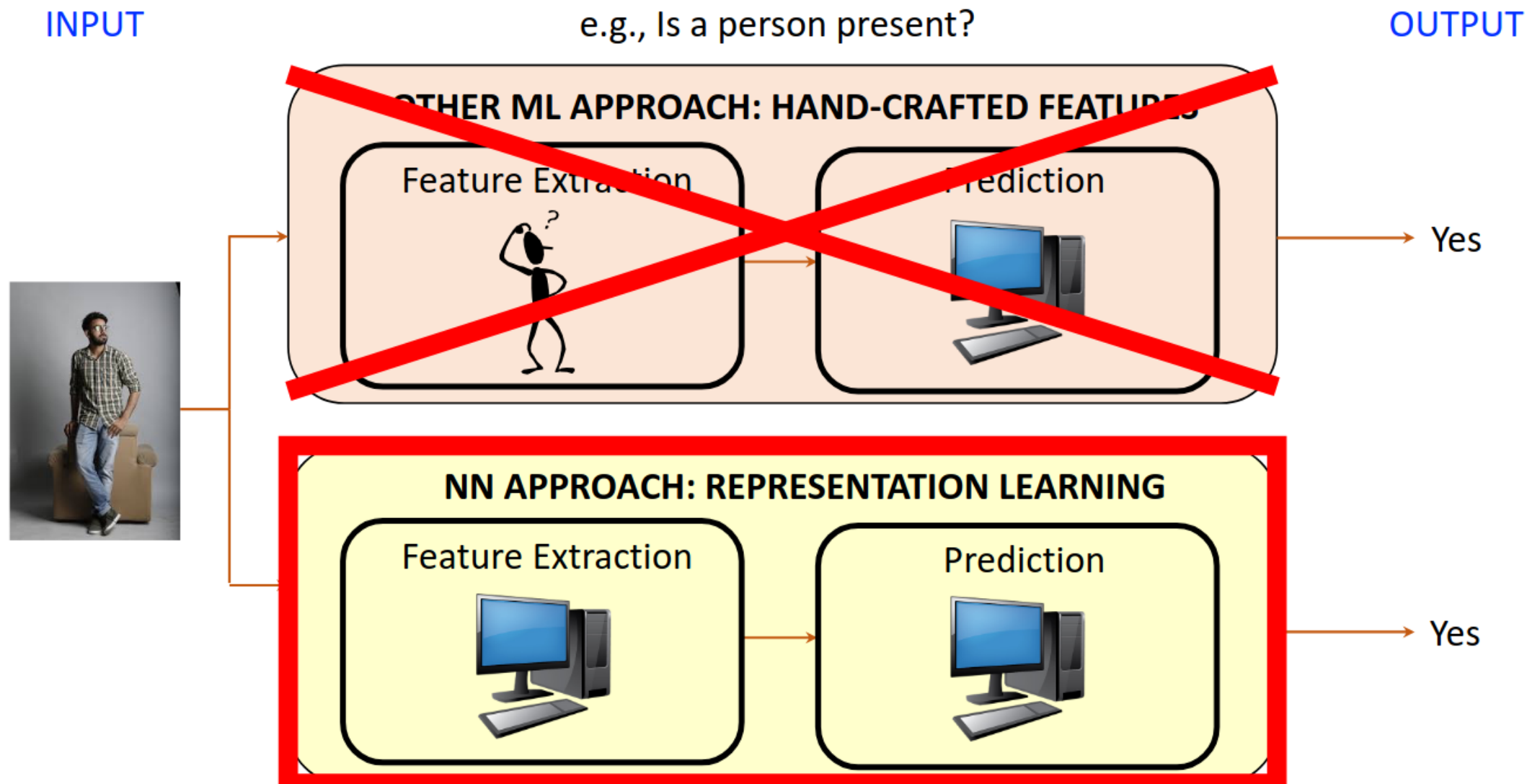
134M parameters, few hundred images -> not going to work :(

Motivation: Classify common animals

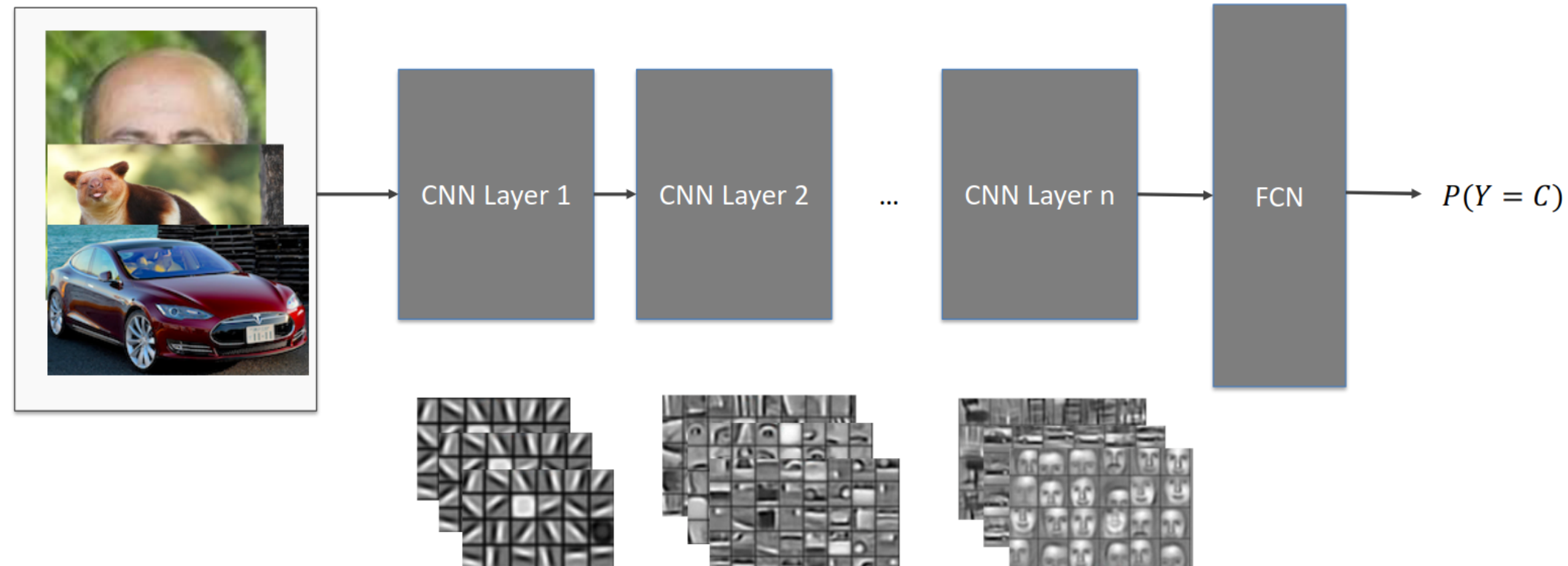


Enough training data :)

9 Reminder: Neural Networks are meant to learn features



Representation Learning in CNNs



9 Transfer Learning: Basic Idea

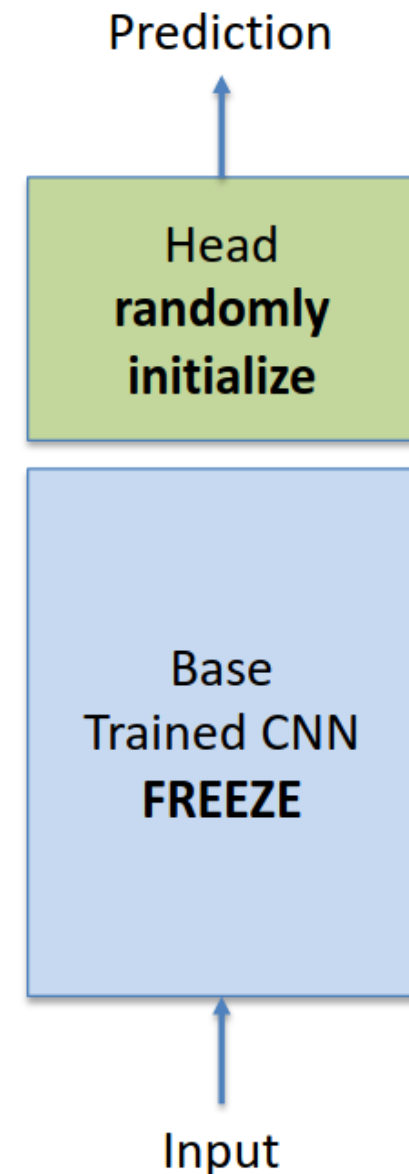
- **Transfer Learning** is a research problem in machine learning (ML) that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem (Wikipedia)
- In our context, this often means: Training a model on some dataset, and using (part of) that model for a different dataset
- Common approach in Deep Learning:
 - *Pre-Train* a network on a large, labeled dataset (e.g., ImageNet)
 - Keep first few layers, re-initialize last few layers
- Train only last few layers with your own, small dataset (possibly different labels or training distribution) or for new task on old images
- Why this might work:
 - First few layers of a network act as feature extractors: Learn to extract features that are generally meaningful, beyond pre-training dataset

9 Transfer Learning

- Is standard approach nowadays in domains where large, high-quality pre-training datasets are available
- E.g. Computer Vision: Rarely a good idea to start training a model from scratch
- Instead, start with model pre-trained on large, existing datasets like ImageNet
- These pre-trained models are publicly available
- Advantages:
 - Potentially higher accuracy, even with small dataset (already have a good feature extractor, only need to learn mapping from features to specific target)
 - Potentially computationally efficient: Do not need to train all parameters, but can *freeze* large part of the network, only train last few layers

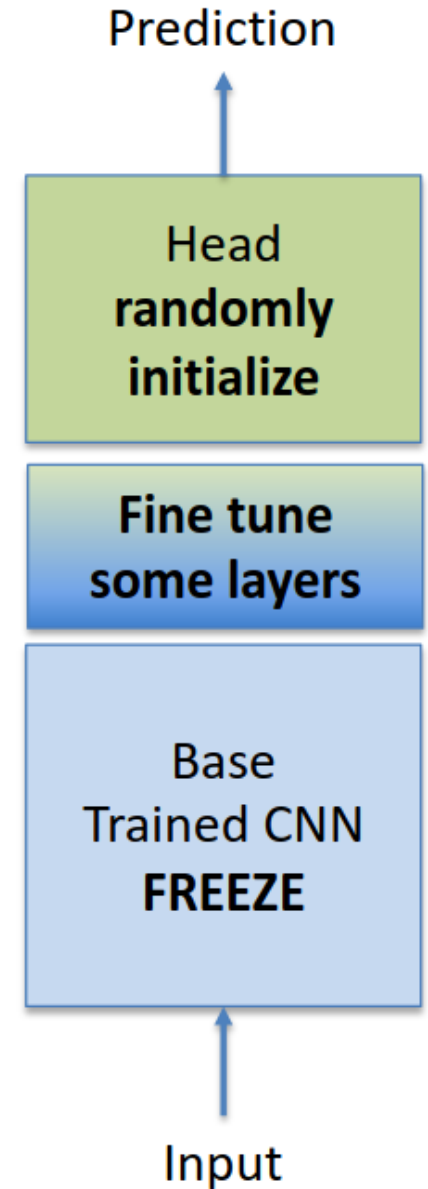
9 Representation Extraction

- Use representations learned by big net to extract features from new samples, which are then fed to a new classifier:
- Keep (frozen) convolutional base from big model.
- Generally throw away head FC layers since these have no notion of space, and convolutional base is more generic.
- Since there are both dogs and cats in ImageNet you could get away with using the head FC layers as well. But by throwing it away you can learn more from other dog/cat images.



9 Fine-Tuning

- Up to now we have frozen the entire convolutional base
- Remember that earlier layers learn highly generic feature maps (edges, colors, textures).
- Later layers learn abstract concepts (dog's ear).
- To particularize the model to our task, its often wort tuning the later layers as well.
- But we must be very careful not to have big gradient updates



BIG DATA PROCESSING

Part II: Time Series Analysis

2

Introduction

Introduction – Big Data

3

□ Characteristics of Big Data

▣ Velocity

- Speed of collection, management, ...
- Esp. relevant for streaming data

▣ Varacity

- Data quality
- Errors, missing values need to be considered

▣ Volume

- Size of data sets
- How do we store and process large data sets?

▣ Variety

- Heterogenous data sets
- Dealing with different formats, encodings, schemas is necessary

4

Cloud Data Management

Cloud Data Management – Overview

5

□ Cloud Computing

- ▣ On demand, reliable services provided over the internet (the „cloud“) with easy access to virtually infinite computing, storage and networking resources
- ▣ Users can outsource complexe tasks through simple web interfaces
 - Data management
 - System administration
 - Application deployment
- ▣ Complexity of managing infrastructure is shiftet from the user's organization to the cloud provider
- ▣ Built upon known computing modules like web services, cluster computing, virtualisation, grid computing, ...

Cloud Data Management – Taxonomy

6

- Infrastructure-as-a-Service (IaaS)
 - ▣ Computing, networking and storage resources as a service
 - ▣ Provides elasticity: ability to scale up (add more resources) or scale down (release resources) as needed
 - ▣ E.g. Amazon Web Services, Microsoft Azure
- Software-as-a-Service (SaaS)
 - ▣ Application software as a service
 - ▣ Generalizes the earlier ASP model with tools to integrate other applications, e.g. developed by the customer (using a cloud platform)
 - ▣ Hosted applications can range from simple (email, calendar) to complex (data analysis, social network, ...)
 - ▣ E.g. Salesforce CRM system, Adobe Creative Cloud

Cloud Data Management - Taxonomy

7

- Platform-as-a-Service (PaaS)
 - ▣ Computing platform with development tools and APIs as a service
 - ▣ Enables developers to create and deploy custom applications directly on the cloud infrastructure and integrate them with applications provided as SaaS
 - ▣ E.g. Google Apps, IBM Cloud Platform (formerly Bluemix)

Cloud Data Management – Benefits

8

- Reduced cost
 - ▣ Customers no longer need to own and manage IT infrastructure and are billed by resource consumption
 - ▣ Cloud providers share costs for multiple customers reducing the cost of ownership and operation
- Ease of access/use
 - ▣ Customers can access to IT services anytime, from anywhere with an internet connection
- Quality of Service
 - ▣ Operation of infrastructure by a specialized, experienced provider
- Elasticity
 - ▣ dealing with sudden increases in load is very easy, e.g. by creating more VMs

Cloud Data Management – Downsides

9

- ❑ Main Issues: Security and Privacy
- ❑ Current Solutions
 - ▣ Internal/private cloud: use of cloud technologies in a private network behind a firewall
 - Much tighter security
 - Reduced cost advantage because the infrastructure is not shared with other customers
 - Compromise: hybrid cloud (internal cloud for OLTP, public Cloud for OLAP)
 - ▣ Virtual private cloud (VPC): VPN within a public cloud with security services
 - Promise of a similar level of security as an internal cloud and tighter integration with internal cloud security
 - But: security integration is complex and requires talented security administrators

Cloud Data Management - When to use cloud solutions?

10

- Common requirements
 - ▣ Data characteristics
 - Very large sets of unstructured or semi-structured data
 - Typically append-only (few actual changes to existing data sets)
 - ▣ Users and application developers
 - Diverse expertise (likely no database specialists)
 - ▣ Availability
 - High availability and fault tolerance
 - High scalability

Cloud Data Management – Why no RDBMS?

11

- Distributed/Parallel versions exists
- Support for XML, multimedia, structured types, ...
- Counter Arguments
 - ▣ Complexity
 - SQL integration of other concepts can be unintuitive or make queries hard to understand
 - ▣ Rigidity
 - Relational structure clashes with unstructured or semi-structured data
 - ACID transactions are unnecessarily strict if updates are rare

Cloud Data Management – OLTP and OLAP

12

Online Transaction Processing

- Operational databases
- Average to large sizes (TB)
- Write-intensive
- ACID transactional properties
- Strong data protection
- Response time guarantees

Not very suitable for cloud

- ▣ Requires shared-disk multiprocessors
- ▣ Corporate data stored at untrusted host

Online Analytical Processing

- Historical databases
- Very large sizes (PB)
- Read-intensive
- Can accept relaxed ACID properties

Suitable for cloud

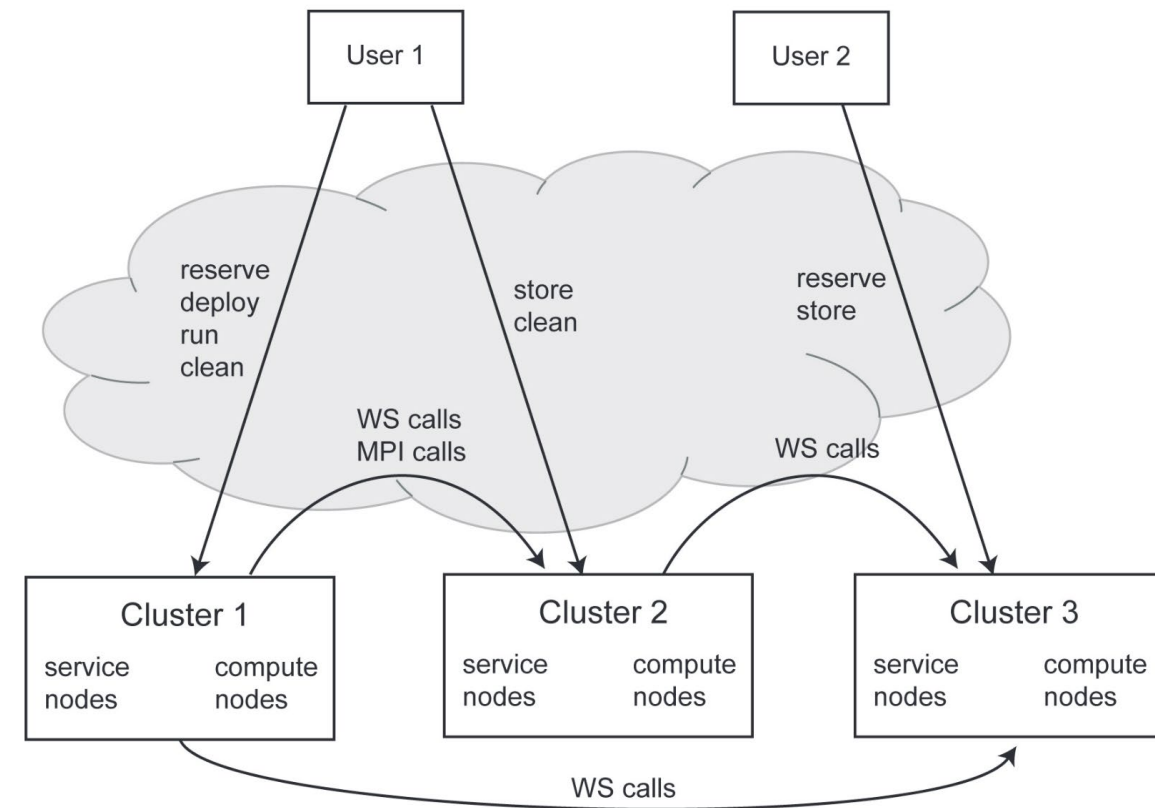
- ▣ Shared-nothing clusters of commodity servers are cost-effective
- ▣ Sensitive data can be anonymized in the cloud

Cloud Data Management – Architecture

13

□ Grid Architecture

- Access through web services to distributed heterogeneous resources
 - Supercomputers, clusters, databases, ...
- For Virtual Organizations
 - Which share the same resources, with common rules and access rights
- Grid middleware
 - Security, database, provisioning, job scheduling, workflow management, ...

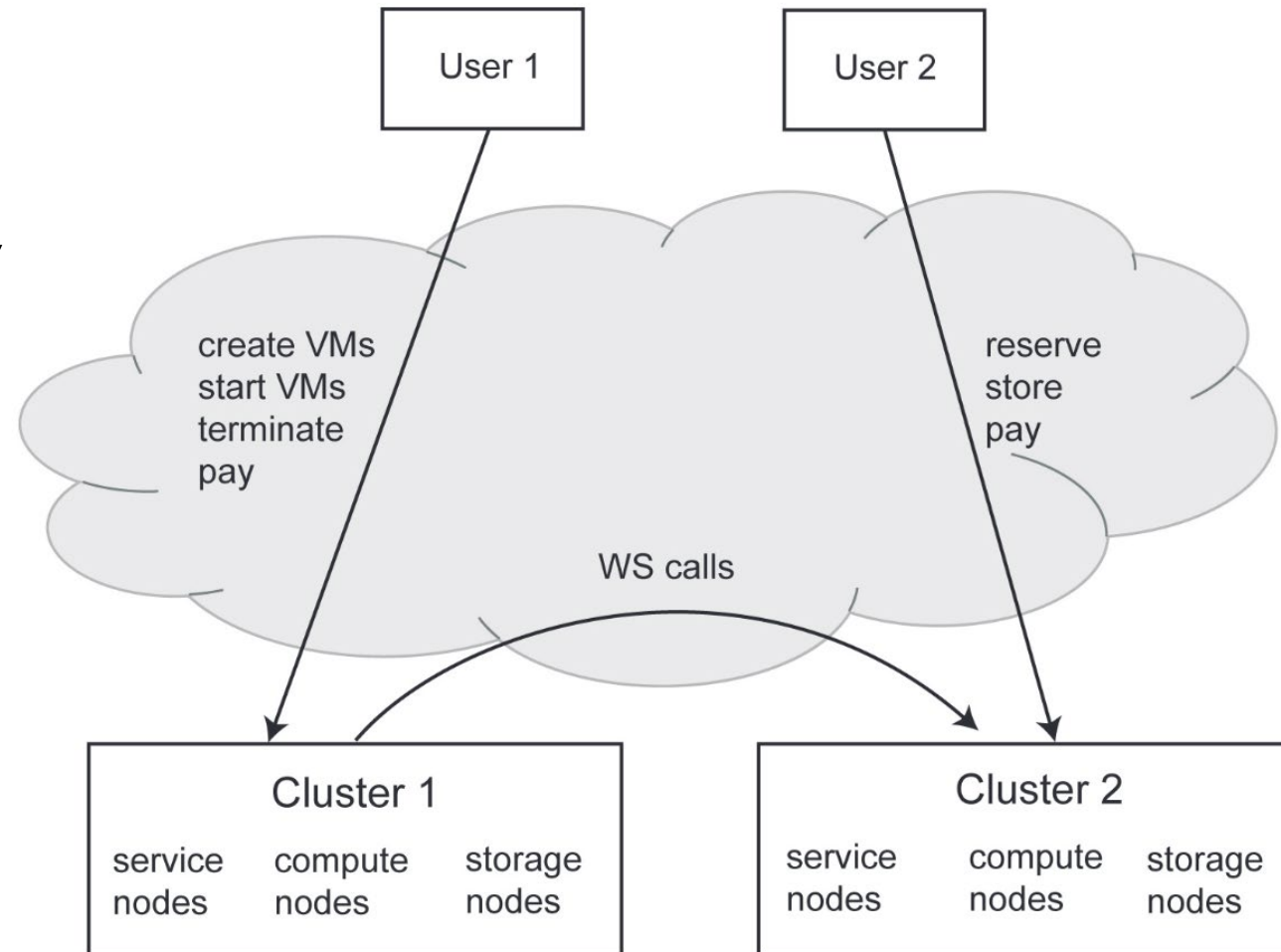


Cloud Data Management – Architecture

14

□ Cloud Architecture

- ▣ Access to resources using web services
 - Less distribution, more homogeneity and bigger clusters
- ▣ For different customers
 - Including individuals
- ▣ Replication across sites for high availability
- ▣ Scalability, Service Level Agreement (SLA), accounting and pricing essential



15

NoSQL

NoSQL – Taxonomy

16

- Not Only SQL
- Key/Value Stores
- Document Stores
- Column/Column Family Database
- Graph Databases
- Others
 - ▣ Geospatial Information Services
 - ▣ File Systems
 - ▣ Object Servers

NoSQL – Key/Value Stores

17

- Global collection of Key/Values pairs
- Originates from P2P Data Management, Distributed Hash Tables and Amazon's Dynamo
- Designed to handle massive load
- Multiple Types
 - ▣ In memory, i.e. Memcache
 - ▣ On Disk, i.e. Redis, SimpleDB
 - ▣ Eventual Consistency, i.e. Dynamo, Voldemort

NoSQL – Key/Value Stores

18

- Very simple key-value data access allows for high performance and availability
- Keys and values can be complex compound objects including lists or maps
- Only simple queries are supported:
 - ▣ .get(Key) to retrieve data
 - ▣ .put(key, value) to store data
 - ▣ .delete(key) to remove data
- Downsides:
 - ▣ No foreign key constraints
 - ▣ No complex filter queries
 - ▣ All joins must be done in code
 - ▣ No triggers

NoSQL – Key/Value Stores

19

□ P2P Data Storage Mechanism

- ▣ Two complementary methods for key based tuple store (e.g. in DHT)

- ▣ Tuple storage

- Each tuple of a relation is stored in the DHT using its tuple's identifier (e.g. primary key)
- Allows to retrieve tuples using their identifier

- ▣ Attribute storage

- Values of some attributes of a tuple are stored individually in the DHT
- Acts as secondary indices
- Good support for exact match queries

NoSQL – Tuple Storage

20

- Let
 - ▣ $R(a_1, \dots, a_n)$ be a relation
 - ▣ $t: \langle v_1, \dots, v_n \rangle$ be a tuple of R
 - ▣ v_1 be the primary key of tuple t
 - ▣ h be a hash function which hashes its input into a DHT key
- To store tuple t in the DHT, a peer does as follows:
 - ▣ $ts_key_t = h(R, v_1);$
 - ▣ $put(ts_key_t, \langle v_1, \dots, v_n \rangle);$

NoSQL – Attribute Storage

21

- Domain partitioning
 - ▣ Let D_a be the domain of an attribute a
 - ▣ We partition D_a into n nonempty sub-domains $d_1, \dots, d_n$
 - Example: weight
 - Domain $[0,200]$ in kilograms, can be partitioned into $n = 40$ sub-domains $[0,5), [5,10), \dots, [195,200]$
- Exploitation by peers
 - ▣ All peers know the sub-domains of each attribute
 - ▣ Thus, each peer can locally execute the following function:
 - $sd(a, v)$: returns the sub-domain of attribute a to which value v belongs
- To store value v_2 in the DHT:
 - ▣ $d_i = sd(a_2, v_2)$
 - ▣ $avs_key_{a2} = h(R, a_2, d_i); put(avs_key_{a2}, < v_2, ts_key_t >);$
- Two important features
 - ▣ After retrieving an attribute value, we are able to retrieve its entire tuple
 - ▣ The values which are „relevatively close“ are stored at the same peer (except edges of sub-domains)

NoSQL – Document Databases

22

- Similar to Key/Value Stores but the value is a document
- Inspired by Lotus Notes
- Flexible Schema
 - ▣ Any number of field can be added
- Documents stored in JSON or BSON formats
- Examples: CouchDB, MongoDB

Sample JSON Document

```
{  
    "day": [ 2010, 01, 23 ],  
    "products": {  
        "apple": { "price": 10,  
                   "quantity": 6 },  
        "kiwi": { "price": 20,  
                  "quantity": 2 }  
    },  
    "checkout": 100  
}
```

NoSQL – Column Family Databases

23

- Each key is associated with multiple attributes (columns)
- Hybrid row/column stores
- Inspired by Google Bigtable
- Examples: Hbase, Cassandra
- Data model:
 - ▣ Key-value pair, key mapped to a value that is a set of columns
 - ▣ Column is a tuple (triplet) consisting of a column name, a value and a timestamp

NoSQL – Column Family Databases

24

□ A Bigtable Row

▣ Column family is a kind of timestamped, multi-valued attribute

■ Set of columns (of the same type), each identified by a key

■ Column key = attribute value, but used as a name

■ E.g. the Anchor column family has two keys `Anchor:inria.fr` and `uwaterloo.ca`, the others only a default key, e.g. `Language:column`

■ Unit of access control and compression

Row key	Contents:	Anchor:	Language:
"com.google.www"	"<html> ... </html>" t_1	inria.fr	"english" t_1
		"google.com" t_2	
	"<html> ... </html>" t_5	"Google" t_3	
		uwaterloo.ca	
		"google.com" t_4	

NoSQL – Graph Databases

25

- Inspired by Euler Graph Theory $G = (E, V)$
- Focused on modeling the structure of data
 - ▣ Nodes/Vertices
 - ▣ Relationships/Edges
 - ▣ (Properties of Nodes/Edges)
- More details in the 3rd part of the course

26

GFS, HDFS, MapReduce

Google File System – Overview

27

- Used by many Google applications
 - ▣ Search engine, Bigtable, MapReduce, ...
- Basis for popular Open Source implementations: Hadoop Distributed File System (HDFS)
- Optimized for specific needs
 - ▣ Shared-nothing clusters of thousand nodes, built from inexpensive hardware
 - Node failure is expected
 - ▣ Very large files of typically several GB, containing many objects such as web documents
 - ▣ Mostly read and append (random updates are rare)
 - Large reads of bulk data (~MB) and small random reads (~KB)
 - Append operations are also large and there may be many concurrent client which append the same file
 - High throughput (for bulk data) more important than low latency

Google File System – Design Choices

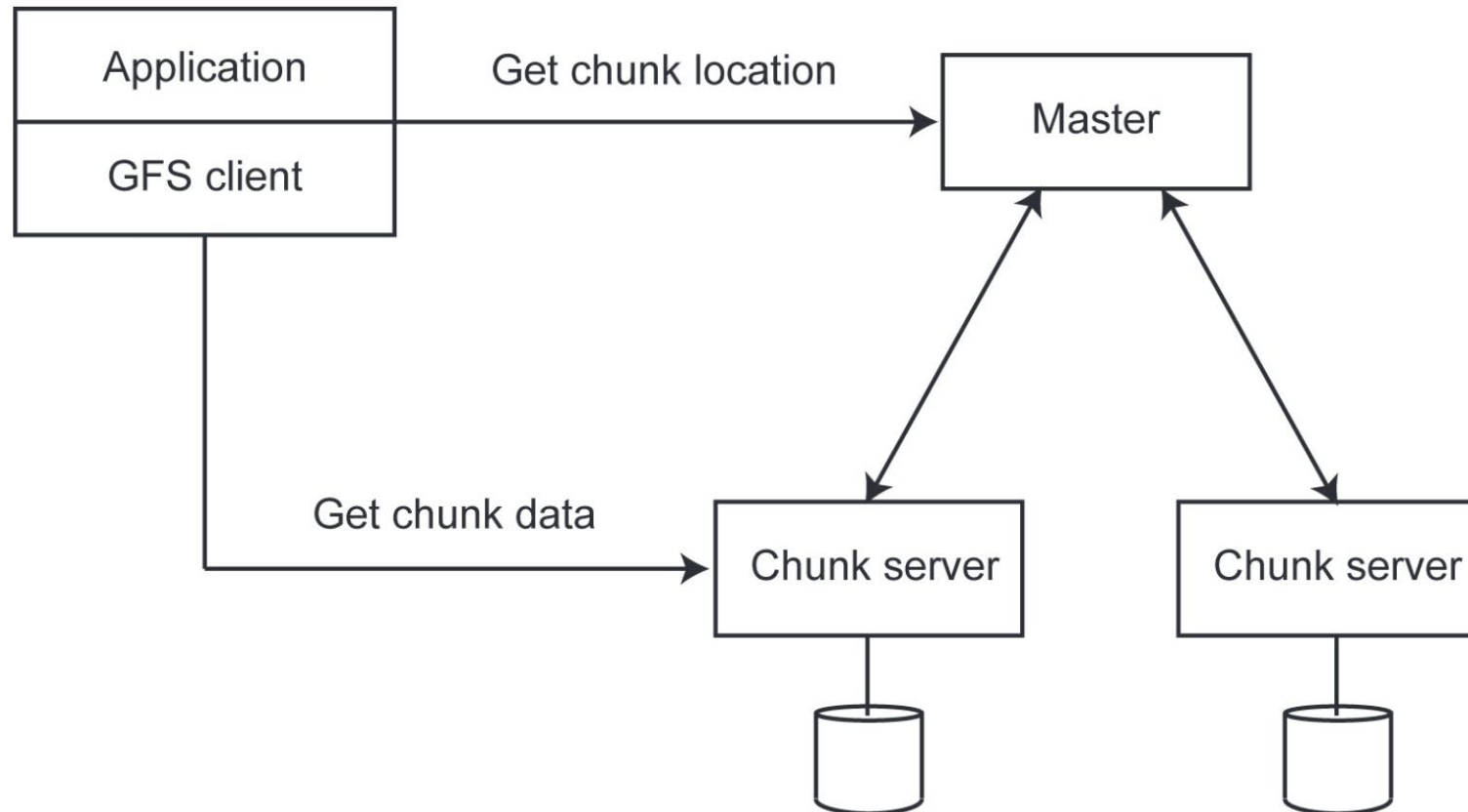
28

- Traditional file system interface (create, open, read, write, close, delete files)
 - ▣ Two additional operations: snapshot and record append
- Relaxed consistency, with atomic record append
 - ▣ No need for distributed lock management
 - ▣ Up to the application to use techniques such as checkpointing and writing self-validating records
- Single GFS master
 - ▣ Maintains file metadata like namespace, access control information, data placement information, ...
 - ▣ Simple, lightly loaded, fault-tolerant
- Fast recovery and replications strategies

Google File System – Architecture

29

- Files are divided in fixed-size partitions, called chunks, of large size, i.e. 64MB, each replicated at sever nodes



MapReduce – Overview

30

- Framework for data analysis of very large data sets
 - ▣ Highly dynamic, irregular, schemaless, ...
 - ▣ SQL or Xquery too heavy
- Simple parallel programming model
 - ▣ Data structured as (key,value) pairs
 - E.g. (doc-id,content), (word,count), ...
 - ▣ Functional programming style with two functions to be given
 - $Map(k_1, v_1) \rightarrow list(k_2, v_2)$
 - $Reduce(k_2, list(v_2)) \rightarrow list(v_3)$
- Implemented on GFS on very large clusters

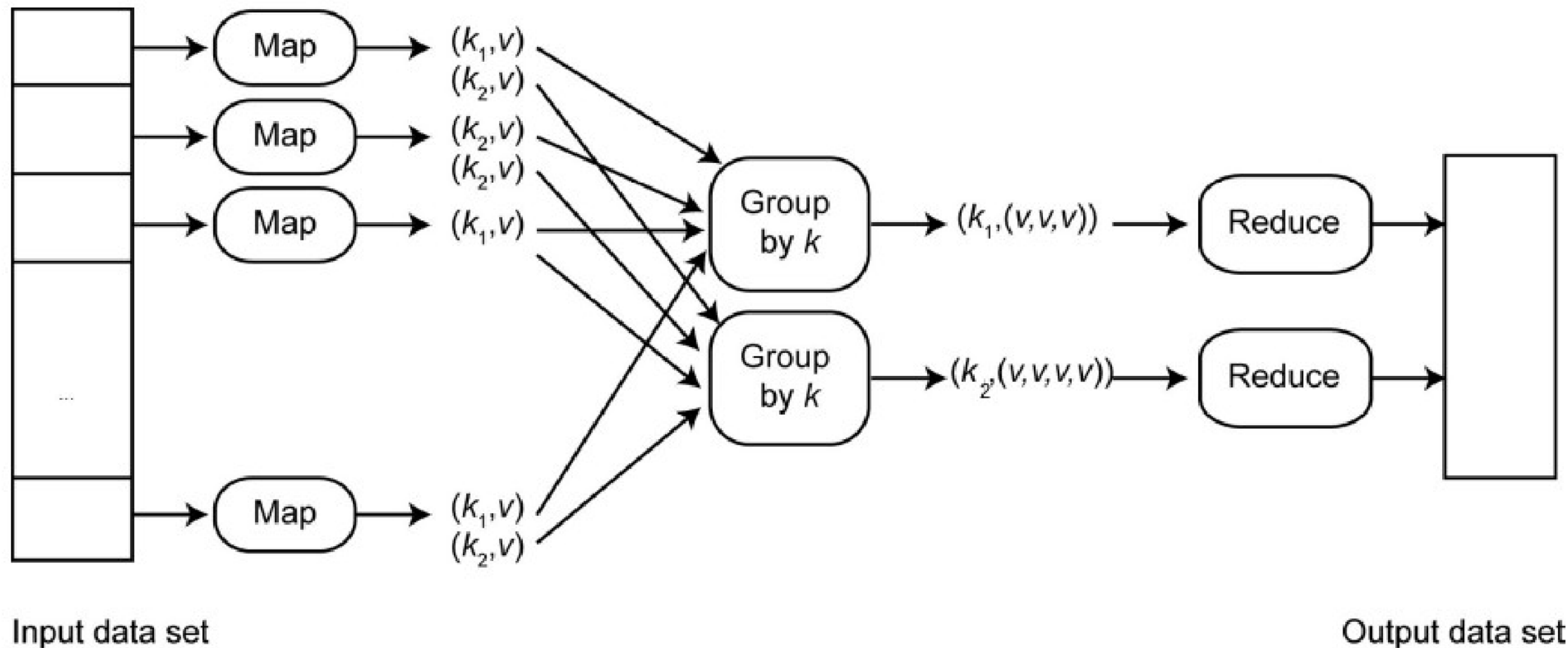
MapReduce – Use Cases

31

- Counting the number of some words in a set of documents
- Distributed grep: text pattern matching
- Counting URL access frequencies in Web logs
- Computing a reverse Web-link graph
- Computing the term-vectors (summarizing the most important words) in a set of documents
- Computing an inverted index for a set of documents
- Distributed sorting
- ...

MapReduce – Processing

32



MapReduce – Example Word Count

33

- Map (Input: String w, Output: (String w,1))
 - ▣ return (w.toLowerCase().remove('!', ',', '?', ...),1)
- Reduce (Input: (String w, list::Integer l, Output: (w, Integer s))
 - ▣ Return (w, SUM(l))

MapReduce – Fault-tolerance

34

- Fault-tolerance is fine-grain and well suited for large jobs
- Input and output data stored in GFS or HDFS
 - ▣ Already provides high fault-tolerance
- All intermediate data is written to disk
 - ▣ Helps checkpointing Map operations and thus provides fault tolerance from soft failures
- If one Map node or Reduce node fails during execution (hard failure)
 - ▣ Tasks are made eligible by the master for scheduling onto other nodes
 - ▣ It may be necessary to re-execute completed Map tasks, since the input data on the failed node disk is inaccessible

35

Analytics

Analytics – Overview

36

□ Data Cleaning

▣ Knowledge of data needed

■ How do we deal with missing or erroneous values?

- Ignore them (if no impact on actual analyses is likely)
- Default value (if realistic)
- Correct them
 - Regression
 - Average of surrounding values

■ How do we know data is erroneous?

- Other attributes indicating bad quality (used by DWD for instance)
- Impossible values, e.g. temperature of 6000°C
- Outlier detection

Regression – Overview

37

- Useful both for replacing missing values and trend analysis
 - ▣ Requires knowledge of relationship between statistical variables
- Is meant to predict values based on samples of dependant values
 - ▣ Used in economics, environmental science, weather reports, ...
- Relationship between variables determines which kind of regression is suitable for predictions

Regression – Linear Regression

38

- Used if there is a linear relationship between two variables
 - ▣ $\hat{y} = bx + a + \epsilon$
- E.g. Least Squares Regression Line
 - ▣ Minimize vertical distance between data points and regression line

$$■ b = \frac{S_{xy}}{S_{xx}} = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{\sum(xy) - \frac{\sum x \sum y}{n}}{\sum(x^2) - \frac{(\sum x)^2}{n}}$$

$$■ \bar{x} = \frac{\sum x}{n}, \bar{y} = \frac{\sum y}{n}$$

$$■ a = \bar{y} - b\bar{x}$$

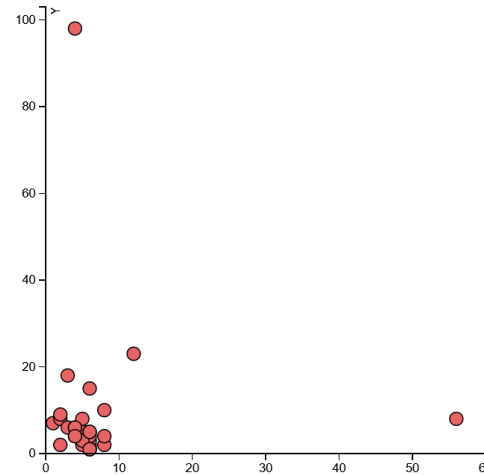
Outlier Detection – Overview

39

□ Types of outliers

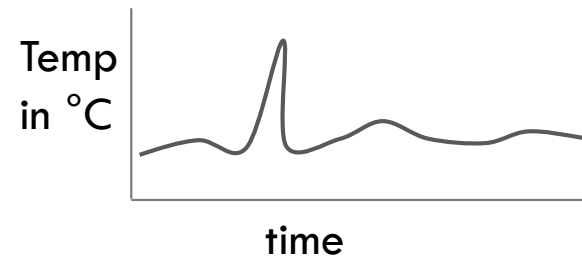
▣ Point outliers

- Simple Form of Outliers
- Vastly different from other values in the data set
- Caused by measurement errors or very unusual events



▣ Contextual outliers

- Data differs is not anomalous in the entirety of the data set, but in a specific context or group
- Causes are manifold, possibly unexpected circumstances during measurement or unusual behavior of a subgroup

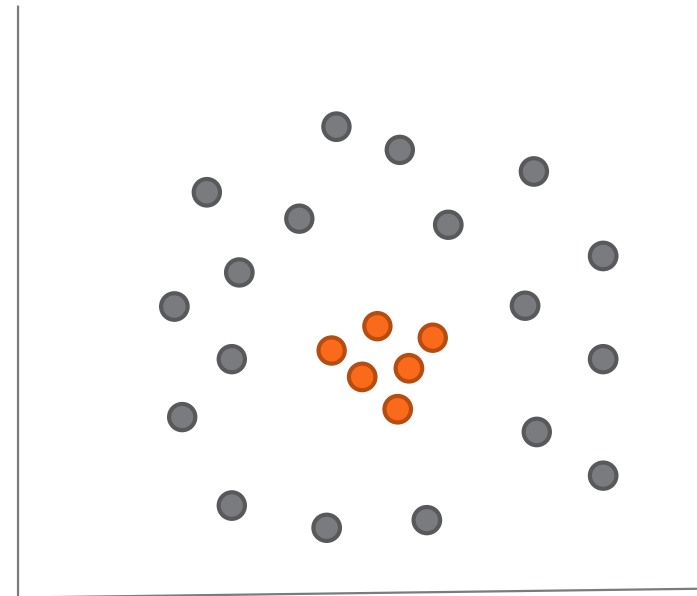


Outlier Detection – Overview

40

▣ Collective outliers

- Group of multiple data points which deviate from overall distribution as a group
- Possibly not unusual when considered individually, but are unusual as a group
- Can represent special patterns or anomalies which may require a closer look



Outlier Detection – Overview

41

□ Handling outliers

▣ Consider the following

■ Impact on analyses

- Do outliers skew results?
- Is it important to consider outliers?

■ Interpretation of outliers

- What caused the outliers?
- Further investigation of outliers (different analyses)

▣ Based on the above we can

- omit, i.e. not consider them for evaluations
- correct them to fit the expected distribution, e.g. using regression
- ignore them, i.e. not omit or correct them for our evaluations
- look more closely at them in further analyses

Outlier Detection – z-Score

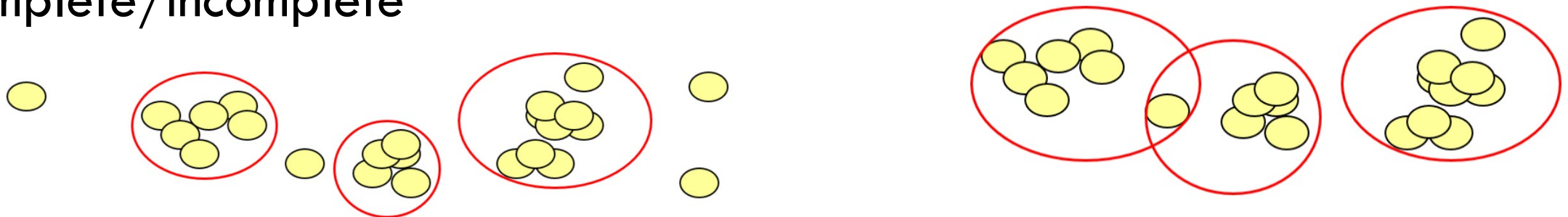
42

- Statistical methods to detect outliers
- Uses mean and standard deviation of the dataset
 - ▣ $Z_i = \left| \frac{x_i - \bar{x}}{\sigma_X} \right|$
 - ▣ Calculate z-Score for each point in the data set
 - ▣ Data points with high z-Score are marked as outliers
- Easy method to find point outliers
- Gives numerical evaluation of abnormality
- Only useful for normally distributed data sets
- Parameter tuning of threshold necessary

Clustering - Overview

43

- Group dataset into sets (clusters) of similar data points
- Clustering is unsupervised classification
 - ▣ Clusters are not predetermined, but inferred from the data itself
- Requires a similarity/distance function
 - ▣ Strongly depends on data types
 - ▣ Must adhere to properties of metric space
- Clustering algorithms can be disjunctive/overlapping and complete/incomplete

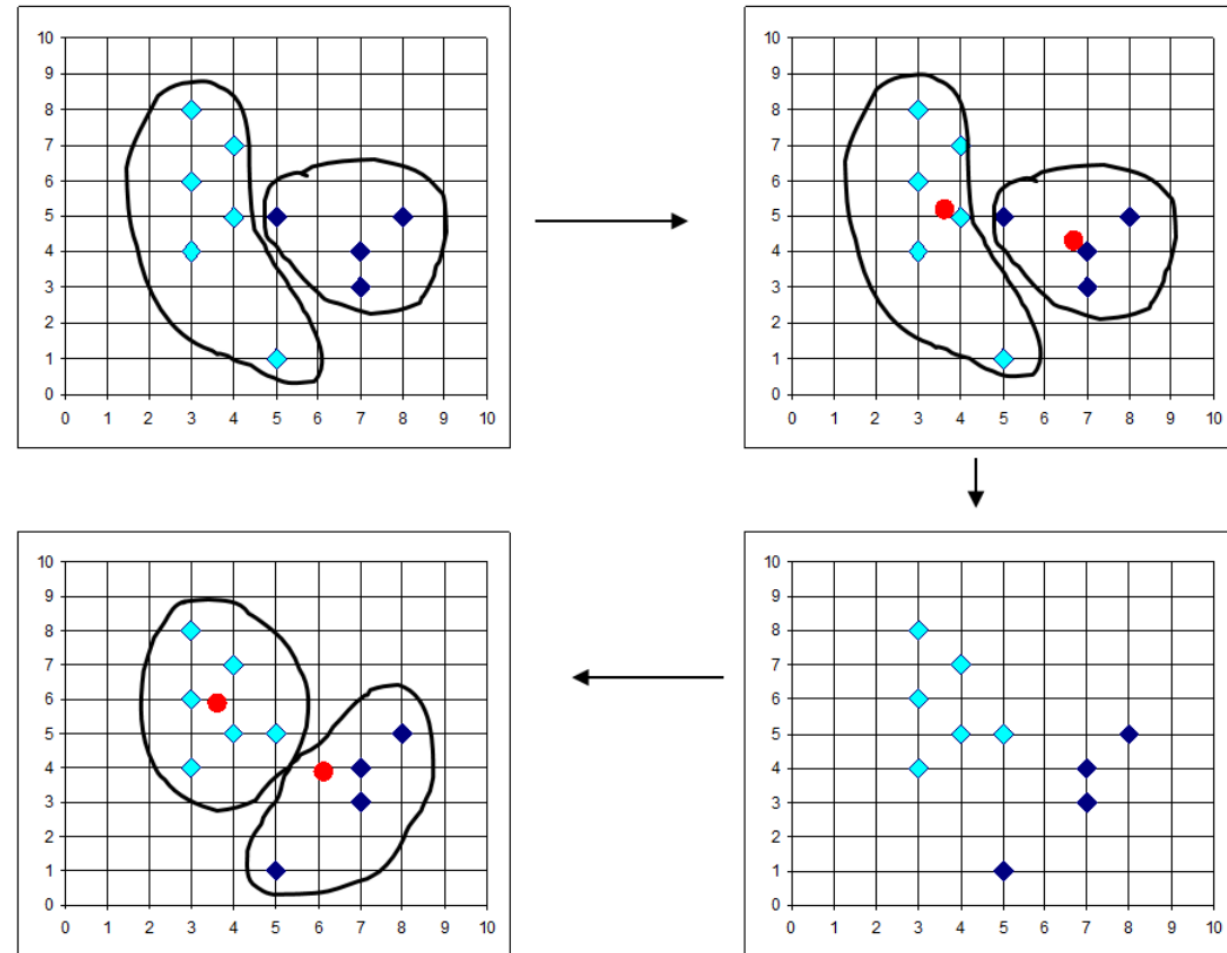


Clustering – k-Means

44

□ Predetermined number of clusters k

1. Partition dataset in k non-empty subset (disjunctive)
 2. Calculate cluster centroids
 3. Reassign data points to the cluster with the nearest centroid
 4. Repeat from 2 until
 - a. Clusters no longer change
 - b. A certain number of iterations is met
- In case of alternating assignments



Clustering – DBSCAN

45

- DBSCAN = Density-Based Spatial Clustering of Applications with Noise
- Clustering algorithm without predetermined number of clusters
- Uses density over distance alone
- Can find clusters of arbitrary shapes
- Can handle outliers with respect to density (classifies outliers as noise)
- Does not handle clusters of varying density



Source: github.com/NSHipster/DBSCAN

Clustering – DBSCAN algorithm

46

- Parameters: Dataset D , Distance ϵ , Number of points $minPts$
- DBSCAN algorithm
 - ▣ Foreach unvisited point $p \in D$:
 - Mark p as visited
 - $N = [q \in D \mid distance(p, q) \leq \epsilon]$
 - If $sizeof(N) < minPts$
 - Mark p as Noise
 - Else
 - Allocate new Cluster C
 - $expandCluster(P, N, C, \epsilon, minPts)$

Clustering – DBSCAN cluster expansion

47

- Parameters: Point p , Neighborhood N , Cluster C , Distance ϵ , Number of points $minPts$
- Cluster expansion
 - ▣ Add p to C
 - ▣ Foreach $q \in N$
 - If q is unvisited
 - Mark q as visited
 - $N' = [q' \in D \mid distance(q, q') \leq \epsilon]$
 - If $sizeof(N') \geq minPts$
 - $N = N \cup N'$
 - If q is not member of any cluster
 - Add q to C
 - (Unmark q as Noise)

48

Project Work

Literature Review

49

- M. Breuning, H. Kriegel, R. Ng, J. Sander “LOF: identifying density-based local outliers”.
- L. Ertöz, M. Steinbach and V. Kumar "A New Shared Nearest Neighbor Clustering Algorithm and its Applications".

Project tasks – Particulate Matter

50

- Implementing and using clustering algorithms
 - ▣ E.g. spatial clustering
 - ▣ Tuning parameters, adjusting density definition
- Outlier Detection
 - ▣ E.g. finding times of high pollution
 - ▣ Intersect outliers with other data
 - Research where outliers came from
- Trend analysis
 - ▣ Compare data from the same months of following years
 - ▣ Implement regression methods for particulate matter

51

Apache Spark and Zeppelin

Spark introduction, using Apache Zeppelin, ...

Spark

52

- „Apache Spark is a unified analytics engine for large-scale data processing“
 - ▣ Unified
 - Support for multiple programming languages using a similar API
 - Python, Scala, SQL, ...
 - ▣ Analytics engine
 - Developer focus should be the analytics, not implementing established algorithms
 - Libraries for machine learning, streaming data, graph analytics, ...
 - ▣ Large-scale:
 - Intended for use in Big Data
 - Supports connections to database systems aswell as remote file systems (HDFS, ...)
 - Automated load balancing and parallelization, i. e. Map/Reduce strategies

Spark

53

- Abstract API

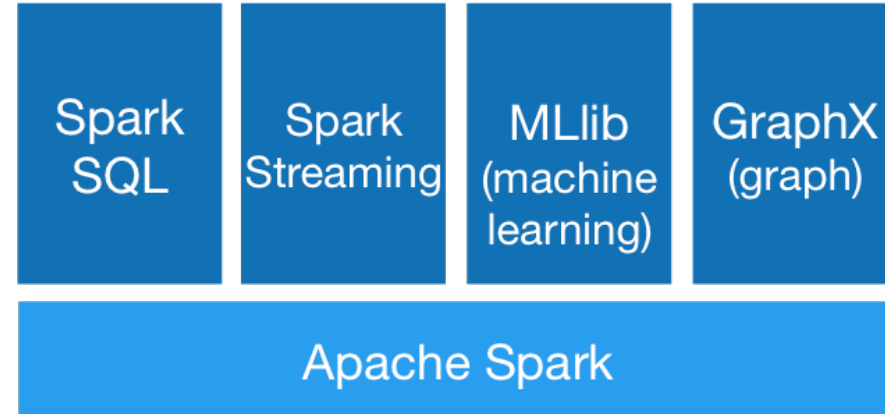
- ▣ Independent from Application

- Utility

- ▣ Little to implement by the developer
 - ▣ Most common algorithms can be found in provided libraries

- Lazy Evaluation

- ▣ Calculations are only performed if the result is required or queried directly
 - ▣ Saves execution time for unnecessary operations



Example – Wordcount (Scala)

54

```
val spark = SparkSession
    .builder()
    .master("spark:pgxlgtm:7077")
    .appName("Spark Test")
    .getOrCreate()

import spark.implicits._

val input = spark.SparkContext.textFile("hdfs://pgxlgtm:8020/input.txt")
val count = input.flatMap(line => line.split(" "))
    .map(word => (word,1))
    .reduceByKey(_ + _)

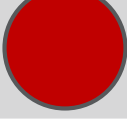
count.collect().foreach(c=> println(c))
spark.close()
```

Not
required
when using
Zeppelin

```
(Hello, 3)
(World, 2)
(Welt, 1)
```

Data Structures

55

	RDD Resilient Distributed Dataset (Java, Scala, Python, R)	DF Dataframe (Java, Scala, Python, R)	DS Dataset (Java, Scala)
Structured Data			
Optimization			
Type Support			
Fluent-API			
SQL			

Zeppelin Notebooks

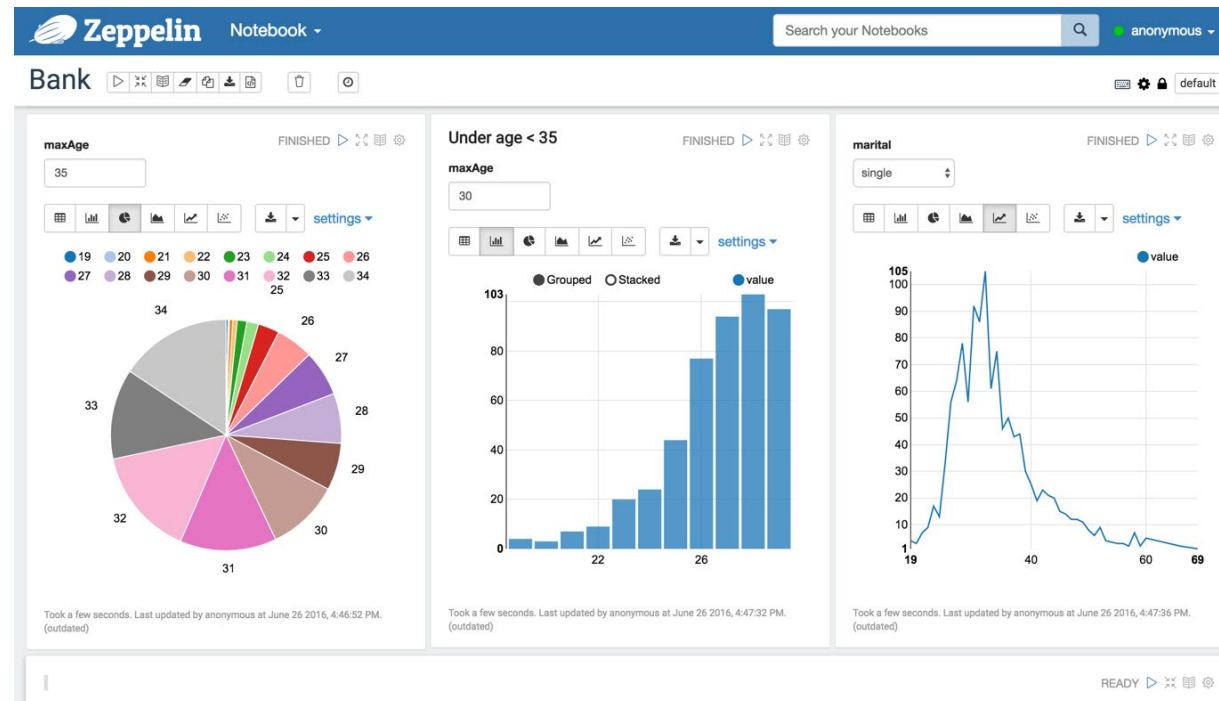
56

- ❑ Web based Notebook for data oriented, interactive analytics and collaborative coding with SQL, Scala, Python, ...
- ❑ Integration of new Environments (Languages, Frameworks) using Interpreters
- ❑ Support for Apache Spark and Hadoop

Zeppelin Notebook

57

- Native support for data visualization
- External libraries can also be installed and used (requires cluster access)



Zeppelin Notebooks

58

- Programming is done in paragraphs (code snippets which are interpreted at execution time)
- Paragraphs can be organized in notes (one note can contain multiple paragraphs)
- Notes can be stored in a folder hierarchy
- Access to folders and notes can be restricted by the owner (see cogwheel in the top right, default: 777)
- Interpreters are bound by notes
 - ▣ No interference between groups
 - ▣ No cross-note sharing of variables

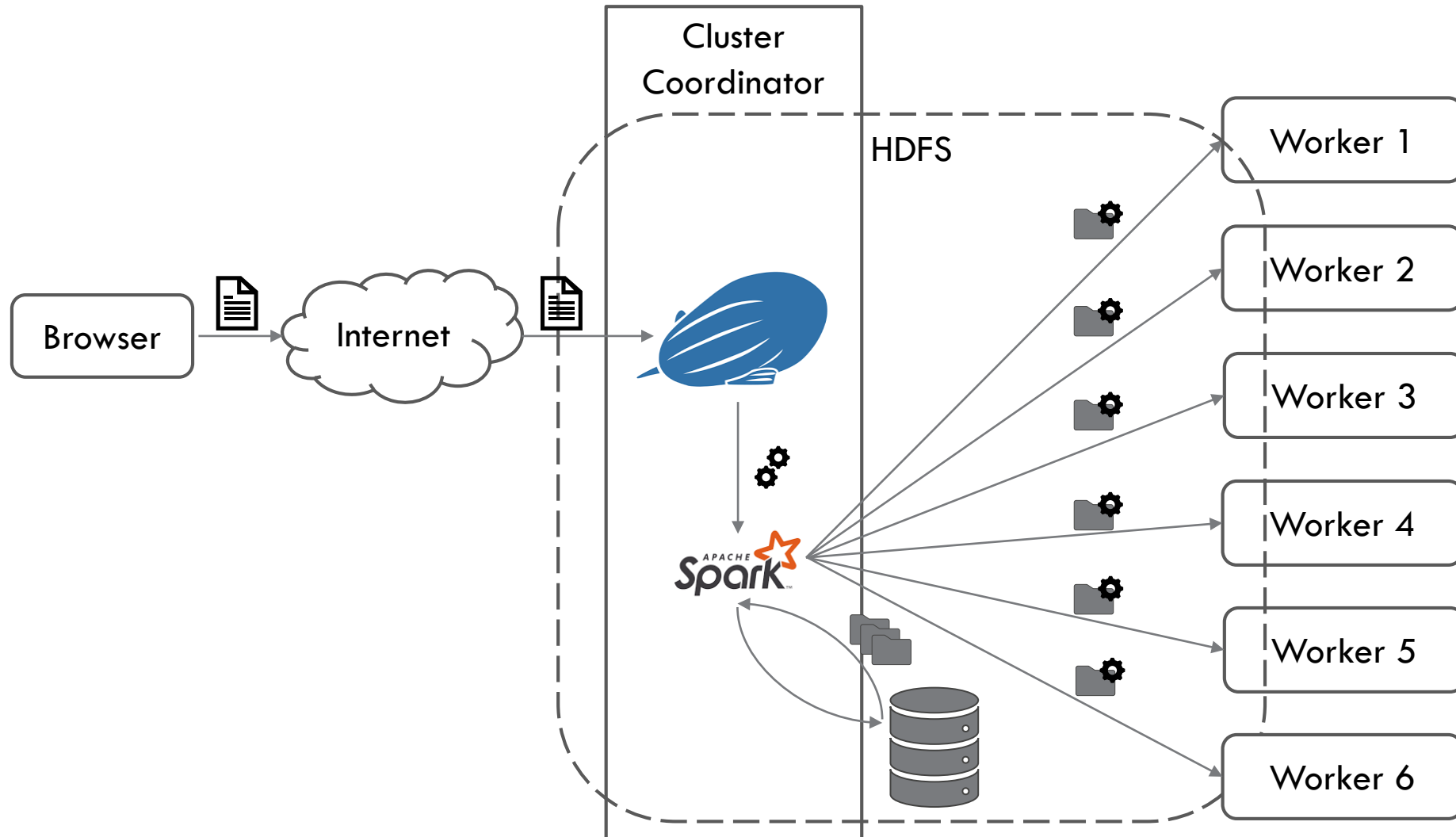
Cluster

59

- Zeppelin runs on our Cluster
- Cluster consists of Coordinator und 6 Worker nodes
 - ▣ VMs with 64GB RAM
 - ▣ Each with 4 logical CPUs @ 2GHz
 - ▣ Connected by highspeed network
 - ▣ Shared HDFS file system with ~300GB space and replication factor 3
 - Your files also go here (available under bdgtm:8020)
 - About ~60GB free

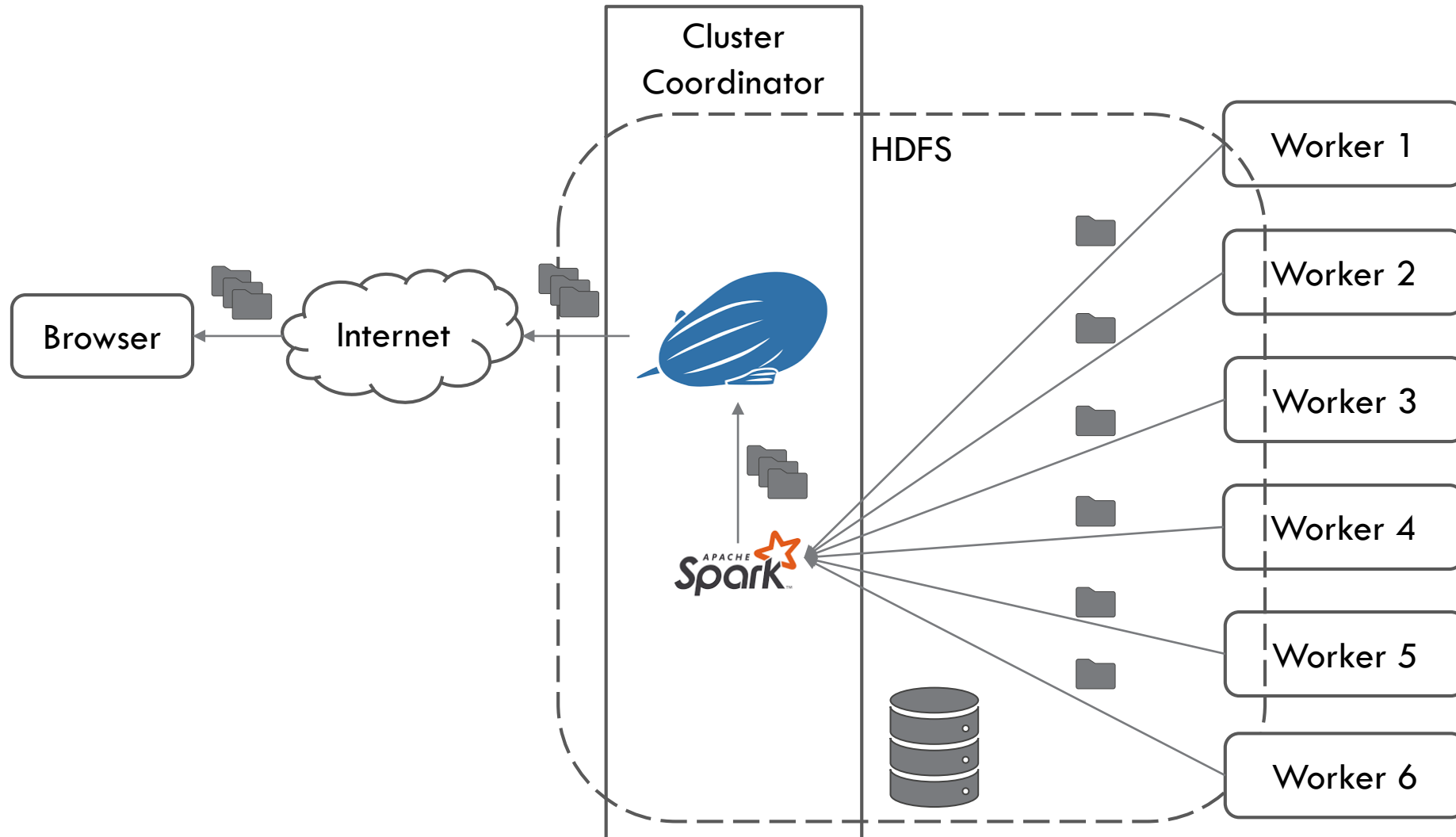
Lifetime of a Spark-Job

60



Lifetime of a Spark-Job

61



Further Material

62

- Apache Spark & Zeppelin websites
 - ▣ Tutorials
 - ▣ Docs
 - ▣ ...

63

Live Demo

Connection details

64

- Login via command line

SSH -L 8082:bdgtm:8082 xy123@honshu.informatik.uni-rostock.de

(xy123 is your IT account)

- Open localhost:8082 in your web browser

- Login to zeppelin using your IT account (same as Git)

- Create a Node and set permissions (cogwheel)

- HDFS Location to save/load data

- ▣ "hdfs://bdgtm:8020/pm_data/<file_name>"

- ▣ File names are structured as follows: DE_<yyyy>-<mm>_sds011.csv